

MỤC LỤC

LỜI NÓI ĐẦU	1
DANH MỤC TỪ VIẾT TẮT	2
PHẦN 1. LÝ THUYẾT	3
Chương 1. TỔNG QUAN.....	3
1.1. Kiến trúc ứng dụng web.....	3
1.1.1. Khái niệm	3
1.1.2. Một số kiến trúc ứng dụng web phổ biến.....	5
1.1.3. Mô hình lập trình dự án web.....	9
1.2. Cấu hình máy chủ web.....	14
1.2.1. Cài đặt Xampp.....	14
1.2.2. Tạo tên miền ảo trên localhost.....	17
CÂU HỎI ÔN TẬP CHƯƠNG 1	19
Chương 2. CƠ SỞ DỮ LIỆU MYSQL	20
2.1. Cơ sở dữ liệu quan hệ	20
2.1.1. Tổng quan	20
2.1.2. Structured Query Language - SQL	21
2.1.3. MySQL.....	23
2.2. Các lệnh cơ bản trong MySQL	23
2.2.1. Định nghĩa dữ liệu	23
2.2.2. Truy vấn dữ liệu - Select	25
2.2.3. Thao tác dữ liệu	29
2.3. Sử dụng phpMyAdmin	30
2.3.1. Tạo cơ sở dữ liệu.....	30
2.3.2. Tạo bảng.....	31
2.3.3. Xem cấu trúc bảng	34
2.3.4. Tạo quan hệ.....	34
2.3.5. Quản lý dữ liệu bảng.....	39
2.3.6. Xuất dữ liệu ra tệp.....	40
2.3.7. Nhập dữ liệu từ tệp.....	41
CÂU HỎI ÔN TẬP CHƯƠNG 2	42

Chương 3. LẬP TRÌNH PHP	44
3.1. Tổng quan	44
3.1.1. Cấu trúc của PHP	44
3.1.2. Hằng, biến.....	44
3.1.3. Các toán tử và biểu thức.....	48
3.1.4. Truyền biến giữa các trang web	49
3.1.5. Sử dụng chỉ thị include	51
3.2. Các kiểu dữ liệu cơ bản	52
3.2.1. Kiểu số nguyên.....	52
3.2.2. Kiểu số thực	52
3.2.3. Kiểu logic	52
3.2.4. Kiểu chuỗi ký tự	53
3.3. Các kiểu dữ liệu phức tạp	54
3.3.1. Kiểu mảng.....	54
3.3.2. Kiểu rỗng	57
3.3.3. Kiểu đối tượng	57
3.4. Các câu lệnh điều khiển.....	58
3.4.1. Cấu trúc if ... else ... elseif	58
3.4.2. Cấu trúc switch	60
3.4.3. Cấu trúc for, foreach	61
3.4.4. Cấu trúc while và do.. while	62
3.4.5. Cấu trúc try.....	63
3.5. Làm việc với cơ sở dữ liệu MySQL	65
3.5.1. Kết nối với cơ sở dữ liệu.....	65
3.5.2. Thao tác với cơ sở dữ liệu.....	67
3.5.3. Đóng kết nối.....	76
3.6. Lập trình hướng đối tượng.....	76
3.6.1. Hàm.....	76
3.6.2. Lớp và đối tượng.....	87
3.7. Làm việc với Session và Cookie	99
3.7.1. Session.....	99

3.7.2. <i>Cookie</i>	101
CÂU HỎI ÔN TẬP CHƯƠNG 3	103
Chương 4. MÔ HÌNH MVC.....	105
4.1. Giới thiệu mô hình MVC	105
4.2. Tầng model	108
4.3. Tầng controller.....	111
4.3.1. <i>Xây dựng BaseController</i>	111
4.3.2. <i>Xây dựng PagesController</i>	112
4.3.3. <i>Xây dựng PostsControllers</i>	113
4.4. Tầng view	114
4.4.1. <i>Module Pages</i>	114
4.4.2. <i>Module Posts</i>	114
CÂU HỎI ÔN TẬP CHƯƠNG 4.....	116
PHẦN 2. THỰC HÀNH.....	117
Bài 1. Cài đặt Xampp và cấu hình tên miền ảo	117
Bài 2. Quản trị cơ sở dữ liệu trong phpMyAdmin.....	117
Bài 3. Lập trình các câu lệnh điều khiển	118
Bài 4. Lập trình với kiểu chuỗi ký tự và mảng	120
Bài 5. Lập trình tương tác với cơ sở dữ liệu MySQL.....	121
Bài 6. Lập trình hướng đối tượng	130
Bài 7. Lập trình theo mô hình MVC	144
Bài 8. Lập trình website thương mại điện tử	159
TÀI LIỆU THAM KHẢO	160

LỜI NÓI ĐẦU

Lập trình các ứng dụng trên nền tảng web là một trong số các mảng của lĩnh vực lập trình, trong đó ngôn ngữ lập trình PHP và cơ sở dữ liệu MySQL được sử dụng phổ biến để phát triển các ứng dụng web. Với những đặc điểm trên, chúng tôi biên soạn tài liệu học tập Phát triển ứng dụng web nhằm giúp sinh viên tiếp cận với những kiến thức cơ bản về thiết kế và phát triển các ứng dụng web dựa trên ngôn ngữ lập trình PHP và cơ sở dữ liệu MySQL.

Nội dung tài liệu gồm 2 phần:

- Phần 1. Lý thuyết: Trình bày các nội dung lý thuyết gồm các chương:

+ Chương 1 – Tổng quan: Trình bày những kiến thức cơ bản về kiến trúc ứng dụng web, mô hình lập trình cho các ứng dụng web và cài đặt môi trường để phát triển các ứng dụng web.

+ Chương 2 – Cơ sở dữ liệu MySQL: Tập trung giới thiệu các khái niệm cơ bản về cơ sở dữ liệu, các lệnh cơ bản với cơ sở dữ liệu MySQL và sử dụng công cụ phpMyAdmin để quản trị cơ sở dữ liệu MySQL.

+ Chương 3 – Lập trình PHP: Trình bày các kiến thức cơ bản của ngôn ngữ lập trình PHP ứng dụng trong phát triển các ứng dụng web, đi từ cách tiếp cận lập trình tuần tự đến lập trình hướng đối tượng. Cùng với từng nội dung là ví dụ thực tế để người đọc tiếp cận.

+ Chương 4 – Mô hình MVC: Trình bày kỹ thuật lập trình ứng dụng web theo mô hình MVC. Đây là mô hình lập trình rất phổ biến không chỉ với các ứng dụng web mà còn phổ biến trong lập trình các ứng dụng trên các nền tảng khác. Nội dung chương cũng được minh họa và làm rõ bởi các ví dụ thực tế.

- Phần 2. Thực hành: Gồm các bài thực hành, luyện tập để ôn luyện kiến thức lý thuyết và rèn luyện kỹ năng thực hành. Các bài thực hành được bố trí theo trình tự các chương trong phần lý thuyết, có yêu cầu tổng hợp kiến thức trong toàn bộ tài liệu.

Với những nội dung trên, tài liệu sẽ rất hữu ích để nghiên cứu và tiếp cận với lập trình, phát triển các ứng dụng web. Đây là tài liệu lưu hành nội bộ, trong quá trình biên soạn, chúng tôi có sử dụng tài liệu của các đồng nghiệp và bố cục lại cho phù hợp đối tượng đào tạo. Khi biên soạn tài liệu này nếu có thiếu sót, rất mong nhận được những ý kiến đóng góp của bạn sinh viên và đồng nghiệp để tài liệu hoàn chỉnh hơn.

Tác giả

DANH MỤC TỪ VIẾT TẮT

STT	Từ viết tắt	Giải nghĩa
1	CMS	Content Manager System
2	CSDL	Cơ sở dữ liệu
3	CSS	Cascading Style Sheets
4	DDOS	Distributed Denial of Server
5	HTML	HyperText Markup Language
6	HTTP	Hyper Text Transfer Protocol
7	JS	JavaScript
8	MVC	Model - View - Controller
9	MVP	Model - View - Presenter
10	MVVM	Model - View - ViewModel
11	PDO	PHP Data Objects
12	PHP	Personal Home Page/Hypertext Preprocessor
13	SQL	Structured Query Language
14	UI/UX	User Interface/User Experience

PHẦN 1. LÝ THUYẾT

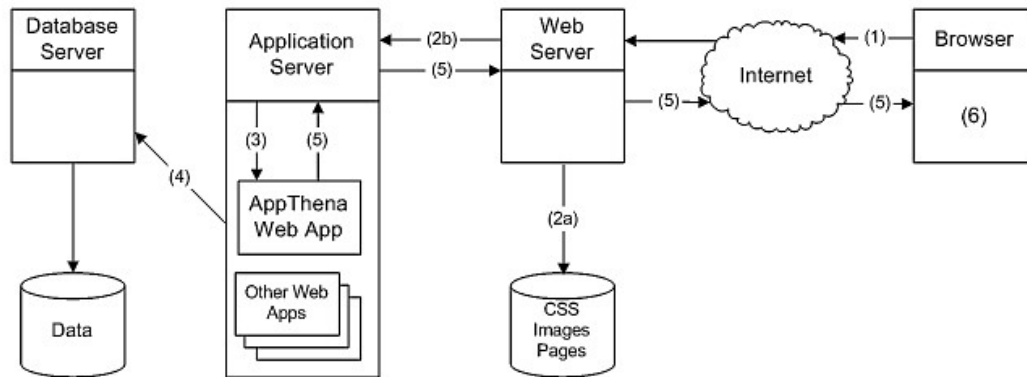
Chương 1. TỔNG QUAN

1.1. Kiến trúc ứng dụng web

1.1.1. Khái niệm

Trước khi đầu tư vào một dự án phát triển ứng dụng web, điều quan trọng là phải chọn được kiểu kiến trúc ứng dụng web (web application architecture) cũng như mô hình của các thành phần ứng dụng web. Việc đưa ra lựa chọn phù hợp rất quan trọng đối với sự thành công của một ứng dụng web.

Kiến trúc ứng dụng web mô tả sự tương tác giữa các ứng dụng, cơ sở dữ liệu (database) và các hệ thống phần mềm trung gian trên web. Kiến trúc này đảm bảo nhiều ứng dụng có thể hoạt động đồng thời và trơn tru. Dưới đây là một ví dụ đơn giản về kiến trúc ứng dụng web.



Hình 1.1. Kiến trúc ứng dụng web đơn giản

* Các thành phần của kiến trúc ứng dụng web

Các thành phần trong ứng dụng web là bất kỳ thành phần nào trong số các thành phần sau:

- Thành phần UI/UX của ứng dụng web: Bao gồm nhật ký hoạt động, trang tổng quát, thông báo, cài đặt, thống kê,... Các thành phần này không liên quan gì đến hoạt động của kiến trúc ứng dụng web. Thay vào đó, chúng là một phần của sơ đồ bố cục giao diện của ứng dụng web.

- Các thành phần cấu trúc: Hai thành phần cấu trúc chính của web là mã phía máy khách và máy chủ.

Thành phần phía máy khách: Các thành phần phía máy khách được phát triển trong CSS, HTML và JS. Vì nó tồn tại trong trình duyệt web của người dùng nên không cần có những điều chỉnh liên quan đến hệ điều hành hoặc thiết bị. Phần frontend đại diện cho chức năng của ứng dụng web mà người dùng tương tác cùng.

Thành phần phía máy chủ: Phần backend có thể được xây dựng bằng cách sử dụng một hoặc kết hợp nhiều ngôn ngữ lập trình như Java, .NET, NodeJS, PHP,

Python, Ruby on Rails,... cùng các framework. Thành phần phía máy chủ phải có ít nhất hai thành phần là logic ứng dụng và cơ sở dữ liệu. Logic ứng dụng là trung tâm điều khiển chính của ứng dụng web trong khi cơ sở dữ liệu là nơi lưu trữ tất cả các dữ liệu một cách liên tục.

* Hoạt động của kiến trúc ứng dụng web

Ngay sau khi người dùng truy nhập địa chỉ của trang web trên thanh tìm kiếm của bất kỳ trình duyệt (Browser) nào. Máy chủ sẽ gửi các file thông tin phản hồi đến trình duyệt để đáp lại yêu cầu, trình duyệt sau đó sẽ chạy các file này để hiển thị trang web được yêu cầu. Cuối cùng, người dùng có thể tương tác với trang web. Các đoạn mã (code) trong các file thông tin sẽ được trình duyệt phân tích cú pháp. Các đoạn mã này có thể có hoặc không cung cấp các hướng dẫn cụ thể để báo lại với trình duyệt về cách phản hồi với các yêu cầu khác nhau của người dùng. Do vậy, trong kiến trúc ứng dụng web sẽ phải bao gồm tất cả các thành phần phụ cũng như các ứng dụng bên ngoài thay thế cho toàn bộ ứng dụng phần mềm (nếu có), trong trường hợp nói trên là một trang web. Cụ thể với kiến trúc ứng dụng web đơn giản trên (hình 1.1) có thể mô tả các luồng thông tin như sau:

- (1): Trình duyệt gửi yêu cầu đến máy chủ web.
- (2a): Máy chủ web truy xuất các file thông tin tĩnh.
- (2b): Máy chủ web gửi yêu cầu đến máy chủ ứng dụng (ví dụ: MySQL, Apache, NodeJs...).
- (3), (4): Máy chủ ứng dụng truy xuất thông tin cơ sở dữ liệu hoặc các Template khác.
- (5): Máy chủ ứng dụng trả kết quả về cho máy chủ web. Sau đó máy chủ web trả kết quả về cho trình duyệt.
- (6): Trình duyệt hiển thị kết quả đáp ứng yêu cầu của người dùng.

Với bất kỳ ứng dụng web điển hình nào cũng có hai phần mã khác nhau chạy song song. Đó là:

- Mã từ phía máy khách (frontend): Mã này nằm trên trình duyệt và phản hồi yêu cầu đầu vào (input) của người dùng.
- Mã phía máy chủ (backend): Mã nằm trên máy chủ và phản hồi các yêu cầu HTTP của trình duyệt.

Một nhà phát triển web sẽ quyết định xem mã trên phía máy chủ sẽ làm gì với mã trong trình duyệt - phía máy khách. Để viết mã từ phía máy chủ, chúng ta sử dụng các ngôn ngữ lập trình như C#, Java, JavaScript, Python, PHP, Rub...

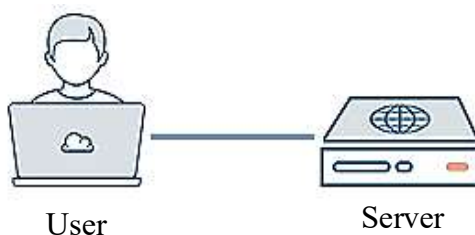
Bất kỳ mã nào có thể đáp ứng các yêu cầu HTTP đều có khả năng chạy trên phía máy chủ. Mã phía máy chủ chịu trách nhiệm tạo trang mà người dùng yêu cầu cũng như lưu trữ các loại dữ liệu khác nhau, bao gồm hồ sơ người dùng và thông tin yêu cầu đầu vào của người dùng.

Sự kết hợp của CSS, HTML, JavaScript được sử dụng để viết mã phía máy khách. Mã này sẽ được trình duyệt web phân tích cú pháp. Không giống mã phía máy chủ, mã phía máy khách có thể được người dùng nhìn thấy và chỉnh sửa. Mã phía máy khách chỉ giao tiếp thông qua các yêu cầu HTTP và không thể đọc trực tiếp các tệp từ máy chủ.

1.1.2. Một số kiến trúc ứng dụng web phổ biến

1.1.2.1. Kiến trúc một server duy nhất (Single Server)

Toàn bộ môi trường hoạt động của ứng dụng web đều nằm trên một server duy nhất. Đối với một ứng dụng web thông thường, sẽ bao gồm: web server, application server và database server. Một ví dụ của cách thiết lập này đó là LAMP stack (viết tắt của Linux, Apache, MySQL và PHP) có môi trường hoạt động đều nằm trên một server duy nhất.



Hình 1.2. Kiến trúc Single Server

* Ưu điểm: Thiết lập một ứng dụng nhanh chóng, bởi vì đây là thiết lập đơn giản nhất.

* Nhược điểm:

- Ứng dụng và database sử dụng cùng tài nguyên máy chủ (CPU, Memory, I/O,...), do đó bên cạnh việc gây ra giảm hiệu năng của hệ thống, nó còn có thể gây khó khăn trong việc xác định thành phần nào gây ra điều đó.
- Không dễ dàng mở rộng ứng dụng theo chiều ngang.

1.1.2.2. Kiến trúc tách riêng cơ sở dữ liệu

Cách thiết lập này sẽ tách riêng hệ thống cơ sở dữ liệu với những phần còn lại để loại bỏ việc tranh chấp tài nguyên giữa ứng dụng và database, đồng thời tăng tính bảo mật hệ thống.



Hình 1.3. Kiến trúc tách riêng cơ sở dữ liệu

* Ưu điểm:

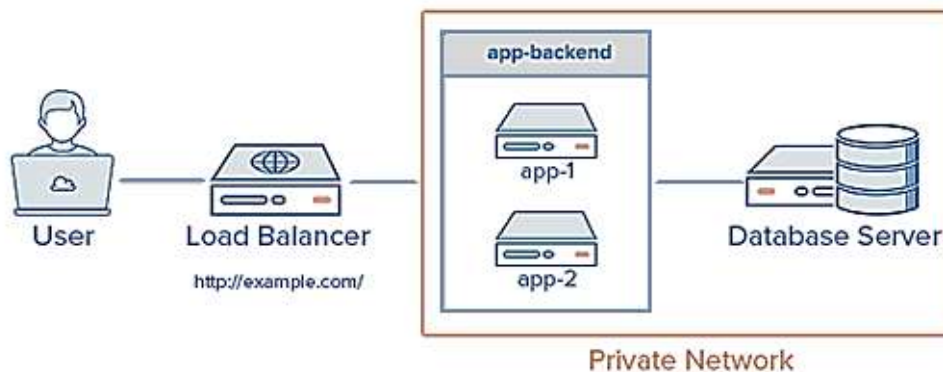
- Thiết lập một ứng dụng nhanh chóng.
- Các tầng ứng dụng và database không bị tranh chấp tài nguyên hệ thống (CPU, Memory, I/O,...)
- Có thể mở rộng theo chiều dọc mỗi tầng riêng biệt, bằng cách thêm tài nguyên cho server nào cần (CPU, Memory)
- Tùy thuộc vào thiết lập để tăng tính bảo mật hệ thống.

* Nhược điểm:

- Thiết lập phức tạp hơn so với cách chỉ sử dụng một server duy nhất.
- Có thể phát sinh các vấn đề về hiệu năng nếu kết nối mạng giữa 2 server có độ trễ cao (do khoảng cách địa lý giữa các server) hoặc băng thông quá thấp cho việc truyền tải dữ liệu.

1.1.2.3. Kiến trúc cân bằng tải

Trong kiến trúc cân bằng tải, kỹ thuật cân bằng tải (Load Balancer) được đưa vào hệ thống để tăng thời gian đáp ứng đối với người dùng (performance và uptime) của hệ thống bằng cách phân tải ra nhiều server. Nếu một server bị lỗi, các server còn lại sẽ phân chia nhau chịu tải cho đến khi server kia hoạt động trở lại. Kiến trúc cân bằng tải có thể được sử dụng để cung cấp cho nhiều ứng dụng thông qua cùng một domain và port. Một số phần mềm cung cấp khả năng phân tải cho các server: HAProxy, Nginx và Varnish,...



Hình 1.4. Kiến trúc cân bằng tải

* Ưu điểm:

- Cho phép mở rộng hệ thống theo chiều ngang, bằng cách thêm nhiều server
- Có thể bảo vệ chống lại những tấn công từ chối dịch vụ DDOS (Distributed Denial Of Service) bằng cách giới hạn kết nối client với số lượng và tần số hợp lý.

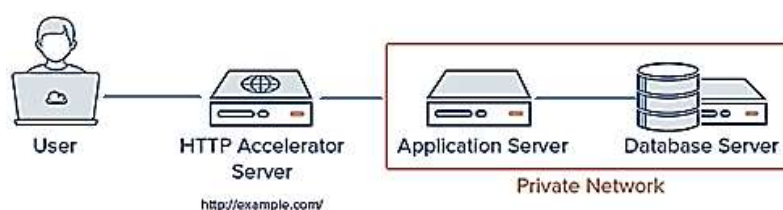
* Nhược điểm:

- Load balancer có thể gây ra tình trạng “nghẽn cổ chai” nếu server không được cung cấp đủ tài nguyên hoặc không được cấu hình tốt.
- Yêu cầu phức tạp khi áp dụng (đồng bộ session giữa 2 server,...)

- Load balancer là một "điểm lỗi duy nhất", nếu nó sập, toàn bộ hệ thống cũng sẽ sập theo. Bằng cách thiết lập hệ thống máy chủ hoặc thiết bị có độ sẵn sàng cao (high availability - HA) sẽ khắc phục được hạn chế này.

1.1.2.4. Kiến trúc HTTP Accelerator (hay Caching Reverse Proxy)

Một HTTP Accelerator, hay caching reverse proxy, có thể được sử dụng để giảm thời gian phục vụ của hệ thống đối với người dùng sử dụng một vài kỹ thuật khác nhau. Kỹ thuật chính của phương pháp này là cache lại response từ một web server hoặc application server vào trong bộ nhớ (memory), và những request trong tương lai với cùng nội dung sẽ được cung cấp một cách nhanh chóng mà không cần tương tác với web server hoặc application server. Một số phần mềm cung cấp HTTP acceleration: Varnish, Squid, Nginx.



Hình 1.5. Kiến trúc HTTP Accelerator

* Ưu điểm:

- Tiện dụng trong những hệ thống dynamic web nặng về nội dung hoặc với nhiều tệp được truy cập thường xuyên.

- Tăng hiệu suất bằng cách giảm CPU load trên web server, thông qua caching.

- Có thể đóng vai trò như một reverse proxy Load Balancer.

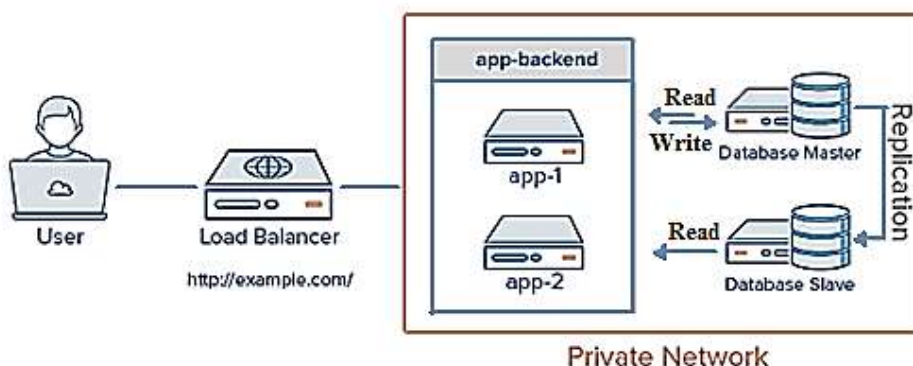
- Có thể sử dụng phần mềm caching để bảo vệ chống lại các tấn công DDOS.

* Nhược điểm:

- Cần phải tinh chỉnh để có hiệu suất tốt nhất.

- Nếu tốc độ truy cập vào cache chậm, nó có thể làm giảm hiệu năng.

1.1.2.5. Kiến trúc Master-Slave Replication



Hình 1.6. Kiến trúc Master-Slave Replication

Một cách để tăng hiệu suất của hệ thống cơ sở dữ liệu thực hiện nhiều thao tác đọc hơn ghi, ví dụ như các hệ thống CMS, đó là sử dụng kiến trúc Master-Slave Replication. Kiến trúc Master-Slave Replication yêu cầu một máy chủ cơ sở dữ liệu gọi là Master và một hoặc nhiều máy chủ cơ sở dữ liệu gọi là Slave. Master đóng vai trò ghi dữ liệu còn các Slave đóng vai trò đọc dữ liệu ra. Khi có dữ liệu được ghi trên Master, những dữ liệu này sau đó sẽ được đồng bộ dần lên các Slave. Kiến trúc này giúp chia tải (các thao tác đọc dữ liệu) ra nhiều server.

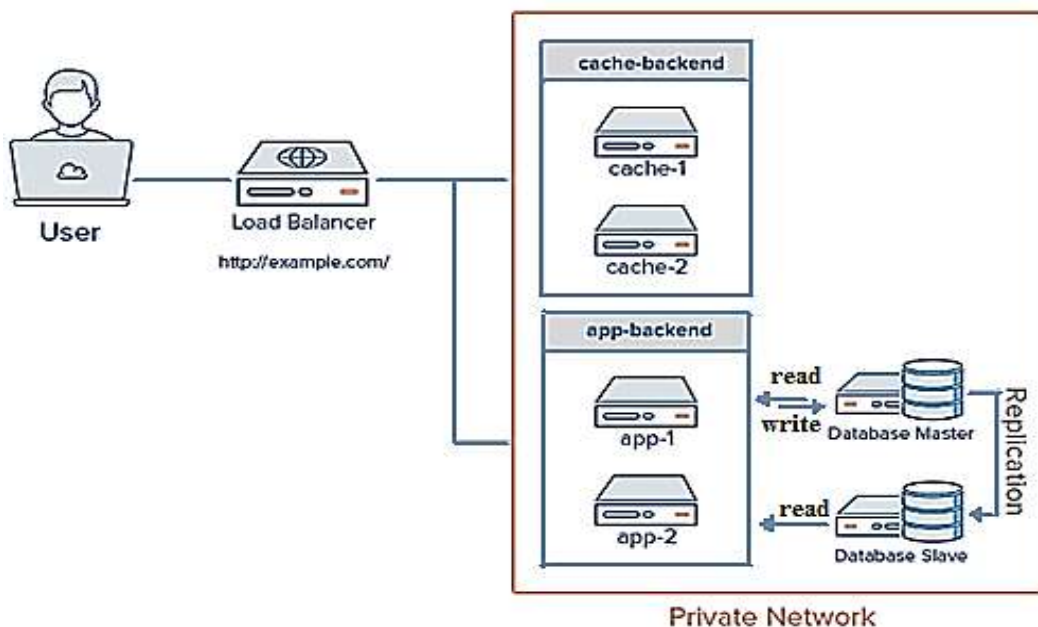
* Ưu điểm:

- Tiện dụng cho các hệ thống mà cần tăng tốc độ đọc dữ liệu.
- Tăng hiệu năng đọc dữ liệu từ database bằng cách phân tải request ra nhiều Server Slave.
- Có thể tăng hiệu năng ghi dữ liệu bằng cách chỉ sử dụng Server Master cho việc ghi (Master không cần phải thực hiện các thao tác đọc).

* Nhược điểm:

- Cài đặt phức tạp.
- Cập nhật lên các Server Slave diễn ra bất đồng bộ, do vậy mà nội dung trên đó có thể sẽ chưa được cập nhật mới nhất trong những thời điểm nhất định.
- Nếu Server Master lỗi, không thể ghi được dữ liệu.

1.1.2.6. Kiến trúc kết hợp



Hình 1.7. Kiến trúc kết hợp

Có thể thêm Load Balancer cho các caching server, ngoài những application servers và sử dụng kiến trúc Master-Slave Replication cho databases. Kiến trúc này kết hợp ưu điểm của tất cả những kiến trúc trên và hạn chế những vấn đề còn tồn tại của những kiến trúc đó.

Giả sử rằng Load Balancer sẽ được cấu hình để nhận biết các static requests (ví dụ như ảnh, css, javascript,...) và gửi những request đó tới các caching servers, gửi những requests khác tới application servers. Sau đây là mô tả hoạt động của kiến trúc này:

Khi người dùng (user) gửi một request dynamic content:

- (1) Người dùng gửi một request dynamic content tới máy chủ web (Load Balancer). Ví dụ: `http://example.com/`.
- (2) Load Balancer gửi request tới mã phía máy chủ (backend).
- (3) Mã phía máy chủ đọc dữ liệu từ các Database Slave và trả về dữ liệu cho Load Balancer.
- (4) Load Balancer sau đó sẽ trả về dữ liệu cho người dùng.

Khi người dùng gửi một request static content:

- (1) Load Balancer kiểm tra trong cache-backend xem nội dung được yêu cầu đã được cache hay chưa.
- (2) Nếu đã được cache: Trả dữ liệu về cho Load Balancer và chuyển tới bước 7. Nếu chưa được cache: cache-backend sẽ chuyển request này cho mã phía máy chủ, thông qua Load Balancer (bước 3).
- (3) Load Balancer sẽ chuyển request tới cho mã phía máy chủ.
- (4) Mã phía máy chủ đọc dữ liệu từ các Database Slave sau đó trả dữ liệu về cho Load Balancer.
- (5) Load Balancer chuyển response cho cache-backend.
- (6) Cache-backend thực hiện cache lại nội dung bên trong response.
- (7) Load Balancer sau đó sẽ trả về dữ liệu cho người dùng.

1.1.3. Mô hình lập trình dự án web

Đối với các dự án web, các nhà cung cấp dịch vụ máy chủ web sẽ thực hiện các yêu cầu liên quan đến kiến trúc ứng dụng web để cài đặt ứng dụng web lên máy chủ. Đối với các nhà quản trị dự án, lập trình viên,... họ quan tâm đến mô hình lập trình của dự án web để triển khai thực hiện.

Trong quá trình thực hiện dự án web, các lập trình viên thường xây dựng và bố cục các tệp, mã lệnh theo một cấu trúc nào đó. Các cấu trúc đó được gọi là mô hình lập trình hay kiến trúc lập trình. Sau đây là một số mô hình lập trình phổ biến.

a. Mô hình Module hóa

Đây là dạng mô hình cơ bản nhất và được sử dụng trong hầu hết các ứng dụng web và trong các mô hình lập trình khác. Trong mô hình này các mã lệnh thực hiện các chức năng nào đó được chia thành các file riêng biệt để sử dụng khi cần, tránh việc phải viết lại mã lệnh.

Ví dụ, trong thiết kế giao diện thường tách các đoạn mã cho phần Header, Footer, Main,... thành các file riêng để nhúng (include) trở lại khi sử dụng. Trong lập

trình xử lý thường tách mã xử lý cơ sở dữ liệu (mã kết nối, chọn cơ sở dữ liệu ...) thành file riêng,...

b. Mô hình Function hóa

Mô hình Function hóa là nâng cấp của mô hình module hóa, trong đó các file xử lý các chức năng được tổ chức lập trình thành các hàm (function) và gọi hàm khi sử dụng. Mô hình này rất phù hợp với các dự án nhỏ, dễ lập trình và kiểm soát.

c. Mô hình Class hóa

Mô hình Class hóa là nâng cấp mô hình function thành class, lập trình viên đã tổ chức các file chức năng thành file chứa các class để tận dụng lợi thế của lập trình hướng đối tượng. Thường mô hình class được tổ chức trong file tương tác cơ sở dữ liệu bao gồm các hàm kết nối, chọn cơ sở dữ liệu, thực thi câu truy vấn,... Mô hình này rất phù hợp với các dự án vừa và nhỏ, dễ lập trình, kiểm soát và phát triển dự án.

d. Mô hình MVC

MVC là từ viết tắt của 3 thuật ngữ: Model – View – Controller. Mô hình MVC được ứng dụng phổ biến trong kỹ thuật phần mềm, đặc biệt là lập trình web. Mô hình MVC được chia thành 3 phần, mỗi thành phần sẽ đảm nhận một nhiệm vụ riêng biệt và hoạt động độc lập với các thành phần khác.

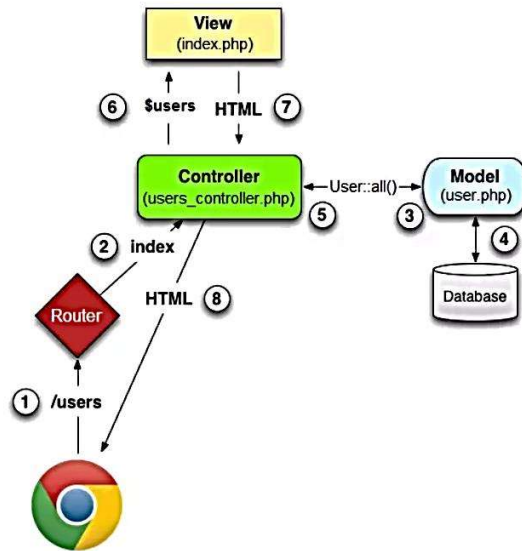
- Model: Có nhiệm vụ thao tác với cơ sở dữ liệu, nghĩa là nó sẽ chứa tất cả các hàm, các phương thức truy vấn trực tiếp với dữ liệu. Controller sẽ thông qua các hàm, phương thức của Model để lấy dữ liệu rồi gửi cho View.

- View: Có nhiệm vụ tiếp nhận dữ liệu từ Controller và hiển thị nội dung sang các đoạn mã HTML, có thể hiểu View là thành phần giao diện.

- Controller: Đóng vai trò trung gian giữa Model và View. Nó có nhiệm vụ tiếp nhận yêu cầu từ client sau đó xử lý request, load model tương ứng và gửi data cho view tương ứng rồi trả kết quả về cho client.

3 bộ phận Model – View – Controller của mô hình MVC tương tác qua lại với nhau tạo thành một hình tam giác. Cách thức hoạt động và luồng thông tin của mô hình này có thể hiểu đơn giản như sau:

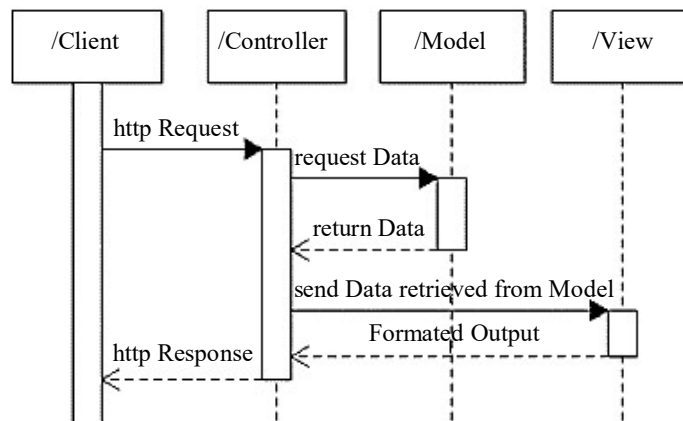
- Trình duyệt gửi một request lên server (1).
- Server nhận được request sẽ phân tích và gửi dữ liệu cho Controller dựa vào router điều hướng (2).
- Trong một số trường hợp thì Controller sẽ kết xuất (render) luôn ra View (một template được chuyển thành HTML) (6) và gửi trả về cho trình duyệt (7).
- Thông thường đối với các trang web động, Controller sẽ tương tác với một Model (3). Model đại diện cho một phần tử, chịu trách nhiệm giao tiếp với cơ sở dữ liệu (4), và trả kết quả cho Controller (5).
- Sau khi gọi Model, Controller sẽ render ra View với dữ liệu lấy được và trả kết quả về cho trình duyệt để hiển thị (8).



Hình 1.8. Mô hình MVC

Như vậy, qua mô hình hoạt động trên, ta có thể thấy phương thức xử lý các yêu cầu của người dùng được quyết định bởi Controller. Phương thức xử lý dữ liệu (logic nghiệp vụ) để hiển thị trên View thì được Model quyết định. Còn View thì chỉ có chức năng tiếp nhận yêu cầu người dùng và hiển thị dữ liệu cuối cùng.

Mô hình tuần tự sau sẽ mô tả luồng thông tin của một http request trong mô hình MVC.



Hình 1.9. Mô hình tuần tự luồng thông tin của một http request

* *Ưu điểm*

- Nhẹ, tiết kiệm băng thông: MVC không tiêu tốn nhiều viewstate nên rất tiết kiệm băng thông.

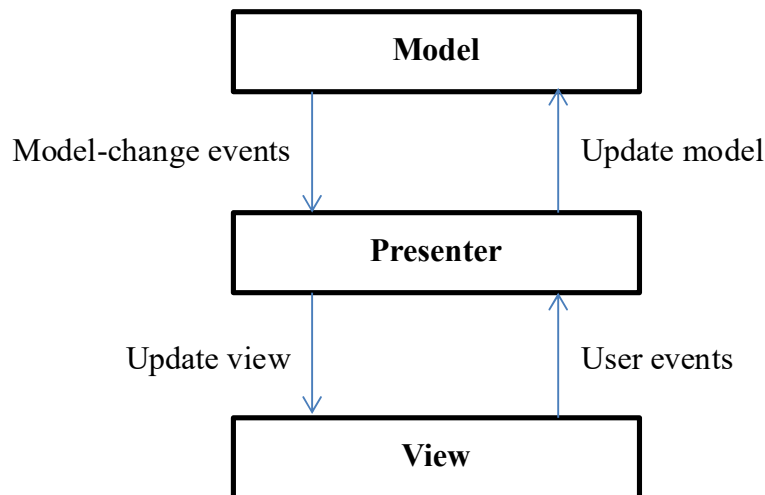
- Các thao tác gửi, nhận dữ liệu được diễn ra liên tục. Vì vậy ứng dụng hoạt động ổn định hơn.

- Có thể kiểm tra, phát hiện lỗi phần mềm dễ dàng.

- Phân tách các phần Model và View, thuận lợi cho việc lập trình.
 - Mô hình có kết cấu đơn giản.
 - * Nhược điểm
 - Khó thực hiện kiểm thử đơn vị (unit testing) do Controller và các API có sự liên hệ chặt chẽ với nhau.
 - Controller và View có liên quan với nhau. Do đó, khi thay đổi ở View thì đồng nghĩa sẽ phải thay đổi ở Controller.
 - Theo thời gian, Controller sẽ trở nên khó kiểm soát vì càng ngày càng có nhiều code được viết thêm vào.
 - MVC chỉ thích hợp với các dự án lớn. Với các dự án nhỏ, mô hình này khá cồng kềnh và tốn nhiều thời gian trong việc trung chuyển dữ liệu.
- Mặc dù có những hạn chế trên, so với những mô hình khác, MVC vẫn là sự lựa chọn hàng đầu cho lập trình ứng dụng nói chung và lập trình web nói riêng.

e. Mô hình MVP

MVP là một mô hình kiến trúc lập trình gần giống như MVC. MVP có nhiều điểm được kế thừa từ MVC nhưng Controller được thay bằng Presenter. Mô hình này gồm có 3 phần chính, bao gồm.



Hình 1.10. Mô hình MVP

- Model: Giống như MVC, Model bao gồm các lớp mô tả logic nghiệp vụ (business logic), định nghĩa quy tắc nghiệp vụ (business rules) cho dữ liệu.
- View: Là thành phần mà người dùng có thể trực tiếp tương tác, gửi yêu cầu. View không bao gồm bất kỳ hoạt động xử lý logic nào.
- Presenter: Đóng vai trò như một bộ phận trung tâm. Bộ phận này sẽ nhận input của người dùng thông qua thành phần View, xử lý dữ liệu với sự giúp sức của Model và cuối cùng trả kết quả lại cho View. Presenter và View giao tiếp với nhau thông qua các interface.

Trong mô hình MVP, View là tầng duy nhất tương tác với người dùng (khác với MVC, cả 2 tầng View và Controller đều có thể ghi nhận tương tác). Luồng đi của thông tin trong mô hình này như sau:

- View nhận tương tác của người dùng. Sau đó, yêu cầu được truyền đến Presenter và xử lý thông qua Model.
- Khi Model thực hiện thay đổi, xử lý dữ liệu xong, Presenter lấy dữ liệu từ Model để truyền đến cho View.

Mô hình MVP có sự phân tách rõ ràng giữa Model và View. Mỗi liên kết duy nhất giữa View và Presenter là thông qua các dữ liệu. Trong MVP, View là bộ phận có chức năng ủy thác đầu vào cho Presenter. Mỗi View đều có một liên kết với Presenter tương ứng thông qua giao diện. Mô hình này thường được sử dụng khi không thể binding dữ liệu qua DataContext.

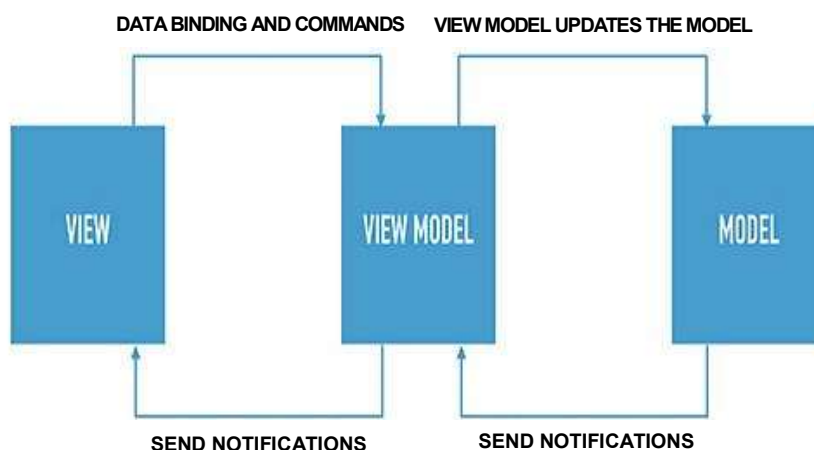
* Ưu điểm

- MVP có cấu trúc code rõ ràng hơn so với MVC nên khá dễ hiểu và dễ dùng.
- Dễ dàng để viết kiểm thử đơn vị cho Presenter vì nó hoạt động độc lập với View và không gắn với bất cứ API nào.

* Nhược điểm: Nhược điểm lớn nhất của mô hình MVP là theo thời gian, Presenter sẽ dần lớn lên do bị thêm các business logic rải rác. Người dùng sẽ rất khó để kiểm soát và chia nhỏ code khi Presenter đã quá lớn.

f. Mô hình MVVM

Là mô hình bao gồm các thành phần: Model, View và ViewModel. Mô hình này có khả năng ràng buộc dữ liệu giữa 2 thành phần của hệ thống là View và ViewModel. Đây là chức năng giúp tách biệt rõ ràng giữa giao diện của người dùng và ứng dụng một cách logic.



Hình 1.11. Mô hình MVVM

- Model: Giống như mô hình MVP và MVC, Model trong MVVM là bộ phận đại diện cho các tầng business logic, định nghĩa business rules cho dữ liệu.

- View: Đây là nơi cho phép người dùng tương tác, đưa ra yêu cầu.

- ViewModel: Đây là lớp trung gian giữa View và Model và xem như thành phần thay thế cho Controller trong mô hình MVC.

Ở mô hình MVVM, ViewModel là nơi thực hiện hầu hết tất cả các business logic, quản lý methods, properties khác để duy trì viewstate. Điều này đồng nghĩa với việc View trở thành một ánh xạ hiển thị trạng thái của ViewModel tại mọi thời điểm.

Lưu ý, trong MVVM, các tầng bên dưới sẽ không biết được thông tin, dữ liệu của tầng bên trên. Điều này giúp đem đến sự tách biệt rõ ràng giữa giao diện người dùng và ứng dụng logic.

* Ưu điểm: Người dùng có thể thực hiện unit testing dễ dàng do không phải phụ thuộc vào View. Khi test, không cần phải tạo giả lập như mô hình MVP mà chỉ cần xác nhận biến quan sát được (observable) thích hợp.

* Nhược điểm: MVVM có nhược điểm về khả năng duy trì. Khi gán biến và biểu thức vào View, các logic rải rác sẽ tăng dần theo thời gian. Điều này khiến việc kiểm soát và thêm code dần trở nên khó khăn hơn.

1.2. Cấu hình máy chủ web

1.2.1. Cài đặt Xampp

1.2.1.1. Giới thiệu về XAMPP

XAMPP (hoặc WAMP) luôn là sự lựa chọn hàng đầu của các lập trình viên website khi họ cần tới môi trường để chạy giả lập website trên trình duyệt. XAMPP hoạt động dựa trên sự tích hợp của 5 phần mềm chính gồm: Cross-Platform (X), Apache (A), MariaDB (M), PHP (P) và Perl (P). Tên gọi XAMPP là chữ viết tắt từ chữ cái đầu của 5 phần mềm này.

- Chữ X đầu tiên là viết tắt của hệ điều hành mà nó hoạt động như: Linux, Windows và Mac OS.

- Apache: Web Server mã nguồn mở Apache là máy chủ được sử dụng rộng rãi nhất trên toàn thế giới để phân phối nội dung web. Ứng dụng được cung cấp dưới dạng phần mềm miễn phí bởi tổ chức Apache Software Foundation.

- MySQL/MariaDB: Trong MySQL, XAMPP chứa một trong những hệ quản trị cơ sở dữ liệu quan hệ phổ biến nhất đó là MySQL. Kết hợp với Web Server Apache và ngôn ngữ lập trình PHP, MySQL cung cấp khả năng lưu trữ dữ liệu cho các dịch vụ web. Các phiên bản XAMPP hiện tại đã thay thế MySQL bằng MariaDB (một nhánh của MySQL).

- PHP: Ngôn ngữ lập trình phía máy chủ PHP cho phép người dùng tạo các trang web hoặc ứng dụng động. PHP có thể được cài đặt trên tất cả các nền tảng và hỗ trợ một số hệ thống cơ sở dữ liệu đa dạng.

- Perl: Ngôn ngữ kịch bản Perl được sử dụng trong quản trị hệ thống, phát triển ứng dụng web và lập trình mạng. Giống như PHP, Perl cũng cho phép người dùng lập trình các ứng dụng web động.

Ưu điểm lớn nhất của XAMPP là không phải trả phí bản quyền và sử dụng mã nguồn mở, bên cạnh đó cấu hình của web server này tương đối đơn giản, gọn nhẹ nên được sử dụng ngày càng phổ biến hiện nay.

Không thể sử dụng XAMPP hay bất cứ phần mềm tạo Web Server nào để đưa website vào vận hành thực tế trên mạng. Để làm điều đó, cần mua hosting hoặc sử dụng các dịch vụ cho thuê server của các đơn vị cung cấp dịch vụ web.

1.2.1.2. Cài đặt XAMPP

- Bước 1: Download XAMPP.

- Bước 2: Nhấp vào file có đuôi .exe trên file tải xuống.

- Bước 3: Trên cửa sổ Setup, tích chọn các phần mềm muốn cài đặt. Thông thường để cài đặt máy chủ web trên máy cá nhân, các phần mềm bắt buộc phải chọn là MySQL, Apache, PHP, phpMyAdmin. Sau khi chọn xong, nhấn Next.

- Bước 4: Chọn thư mục cài đặt và nhấn Next.

- Bước 5: Chờ để cài đặt, sau khi cài đặt hoàn tất nhấn Finish để kết thúc.

Lưu ý:

- Trước khi cài đặt XAMPP phải xóa hết tất cả phần mềm liên quan đến việc tạo localhost như PHP, MySQL. Không dùng đồng thời các công cụ tạo localhost khác nhau vì trong quá trình sử dụng các phần mềm tạo localhost có thể xung đột với nhau.

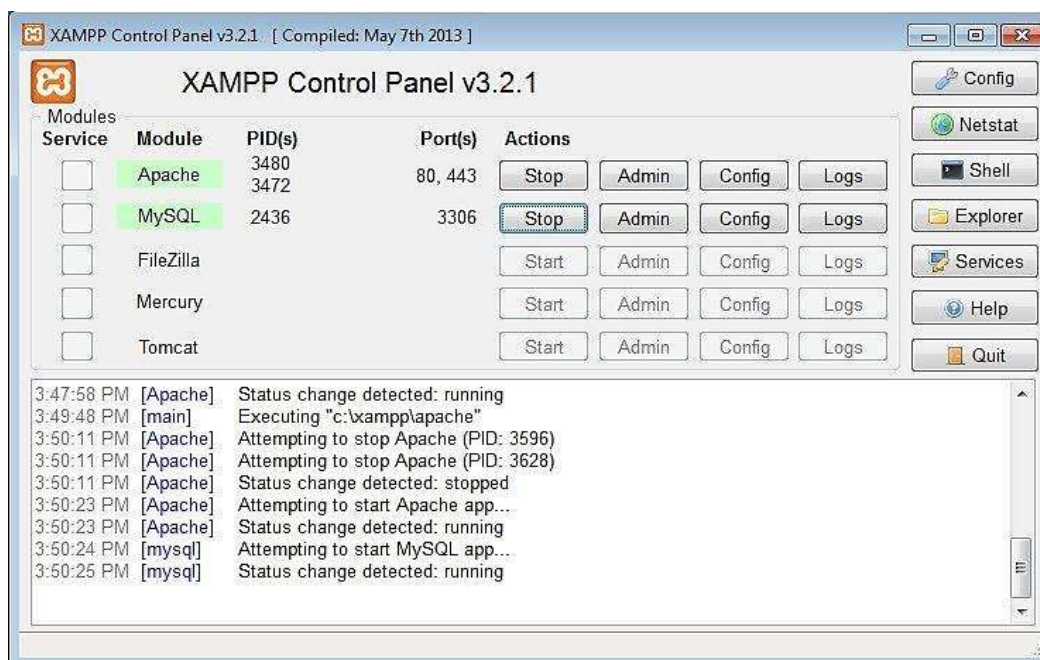
- Nếu dùng Skype: Trong trường hợp máy tính đang cài Skype, mở Skype\Tools\Connection Options, bỏ chọn phần “Use port 80 and 443.....” và thay thế bởi một cổng bất kỳ. Nếu không thực hiện thao tác này, XAMPP sẽ không thể chạy được do cổng mạng 80 đã bị Skype sử dụng.

- Tắt tường lửa: Nên tắt cài đặt tường lửa trên Windows, cũng như tắt cả các phần mềm Antivirus khác, vì những phần mềm này có thể sẽ chặn cổng 80 hoặc các ứng dụng web server, khiến cho XAMPP không thể hoạt động.

- Tắt UAC Windows: Nên tắt User Account Control trên Windows trước khi cài đặt XAMPP để tránh bị giới hạn quyền truy cập.

* Khởi động localhost

Mở thư mục cài đặt XAMPP (mặc định là c:\xampp) và chạy file xampp-panel.exe để bật bảng điều khiển của XAMPP. Trên bảng điều khiển có hai ứng dụng Apache và MySQL có nút Start, hãy ấn vào nút Start của từng ứng dụng để khởi động các dịch vụ Webserver Apache và MySQL Server. Nếu cả hai ứng dụng chuyển sang màu xanh như hình dưới là đã khởi động thành công.



Hình 1.12. Khởi động localhost

Sau khi khởi động xampp, truy cập địa chỉ là <http://localhost> để kiểm tra hoạt động của XAMPP. Trình duyệt hiển thị trang giới thiệu XAMPP, khi đó đã cài đặt xampp thành công.



Hình 1.13. Giao diện chính của XAMPP

* Cấu trúc thư mục làm việc và thao tác trên localhost

Thư mục thao tác mặc định là `C:\xampp\htdocs\`. Mở thư mục `C:\xampp\htdocs\` và tạo một thư mục (ví dụ tên “myproject”), thư mục này sẽ chứa website của dự án.

Để chạy được các file có trong dự án này, cần truy cập địa chỉ <http://localhost/myproject/> trên trình duyệt, nếu thư mục dự án không có file có tên `index.*` thì các file khác và thư mục con bên trong thư mục dự án sẽ được liệt kê.

Tiếp tục tạo file `hello-world.php` với nội dung bên trong là đoạn code PHP sau:

```
1. <?php
2. echo "Hello World!";
3. ?>
```

Mở trình duyệt, truy cập địa chỉ <http://localhost/myproject/hello-world.php> ta sẽ có được kết quả hiển thị như sau:

Hello World!

Trong các bước trên đã tạo một thư mục có tên myproject trong thư mục htdocs và có thể sử dụng đường dẫn <http://localhost/myproject> để chạy ứng dụng. Thư mục `C:\xampp\htdocs\myproject` chính là thư mục gốc của tên miền <http://localhost/myproject>. Đường dẫn trên website sẽ phân cấp theo cấu trúc thư mục và file trong thư mục gốc.

1.2.2. Tạo tên miền ảo trên localhost

Tên miền ảo là một tính năng của Virtual Host của phần mềm XAMPP. Việc tạo tên miền ảo giúp các nhà phát triển quản lý code tốt và dễ nhớ hơn. Để tạo được tên miền ảo thì máy tính phải cài đặt XAMPP. Khi phát triển một dự án và tạo dự án trên localhost, cách thông thường là với mỗi dự án thì sẽ tạo một thư mục riêng trong thư mục `C:\xampp\htdocs\`. Ví dụ một dự án mới tên là laptop247, đặt trong thư mục `"C:\xampp\htdocs\laptop247\"`, như vậy để có thể truy cập tới ứng dụng này trên localhost, đường dẫn trên trình duyệt là <http://localhost/laptop247>. Đây là đường dẫn ở localhost đến ứng dụng laptop247. Tuy nhiên, đường dẫn dưới localhost này gặp phải một số khuyết điểm sau:

- Đường dẫn dài và khó nhớ dẫn đến khó khăn khi tìm.
- Không hiển thị một cách chân thật được kiến trúc URL đã định ban đầu cho website vì không phản ánh được đúng tên miền của website.
- Đường dẫn không ấn tượng khi demo cho khách hàng.

Để giải quyết các vấn đề hạn chế trên, chúng ta tạo tên miền ảo cho localhost với XAMPP. Để tạo tên miền ảo trên localhost, tiến hành 2 bước sau:

Bước 1: Cập nhật file hosts của Windows

Tìm đến file hosts ở máy tính tại `C:\Windows\system32\drivers\etc\` sau đó ấn chuột phải và chọn Properties. Lưu ý, nếu dùng hệ điều hành Mac, đường dẫn đến file hosts là `/private/etc/hosts` hoặc `/etc/hosts` đối với Linux.

Hộp thoại Properties hiện lên với 4 tùy chọn khác nhau, hãy chọn tab Security, và ấn nút Edit để sửa quyền. Tại đây, chọn phần User và đánh dấu Full Control. Sau đó ấn Ok để lưu lại toàn bộ các bước thay đổi.

Bước tiếp theo cần mở file hosts ra, trở tên miền ảo cần thêm vào về địa chỉ IP của localhost là 127.0.0.1, sau đó lưu lại file hosts. Ví dụ: 127.0.0.1 laptop247.com

Bước 2: Cấu hình VirtualHost

Mở thư mục `C:\xampp\apache\conf\extra\` và tìm đến file `httpd-vhosts.conf` rồi mở nó lên. Sau đó tìm đến dòng `##nameVirtualhost *:80` và xóa hai dấu thăng (#) đi. Nếu có sử dụng cổng khác cho Apache thì hãy đổi số 80 trở thành cổng tương ứng.

Chèn thêm đoạn mã dưới đây vào cuối của file `httpd-vhosts.conf` để khai báo thêm tên miền `laptop247.com`:

```
1. <VirtualHost *:80>  
2. Document Root: "C:/xampp/htdocs/laptop247"  
3. ServerName laptop247.com  
4. </VirtualHost>
```

Trong đó:

- Document Root: Là đường dẫn đến thư mục mà chúng ta muốn tên miền ảo trỏ vào.

- Server Name: Là tên miền ảo đã khai báo trong file hosts mà chúng ta muốn thêm vào localhost.

Bây giờ, hãy khởi động lại Apache và truy cập domain ảo để xem kết quả.

CÂU HỎI ÔN TẬP CHƯƠNG 1

Câu 1. Trình bày khái niệm, cách thức hoạt động và các thành phần của kiến trúc ứng dụng web.

Câu 2. Trình bày các kiến trúc ứng dụng web phổ biến.

Câu 3. So sánh ưu, nhược điểm của các kiến trúc ứng dụng web phổ biến.

Câu 4. Trình bày các mô hình lập trình web.

Câu 6. Giải thích vai trò các thành phần trong mô hình lập trình MVC. Cách thức hoạt động của mô hình MVC.

Câu 7. So sánh ưu, nhược điểm của các mô hình lập trình.

Câu 8. Trình bày các bước tạo tên miền ảo trên localhost.

Câu 9. Cài đặt XAMPP và cấu hình tên miền ảo cho ứng dụng web.

Chương 2. CƠ SỞ DỮ LIỆU MYSQL

2.1. Cơ sở dữ liệu quan hệ

2.1.1. Tổng quan

Cơ sở dữ liệu (CSDL) là cách thức tổ chức lưu trữ dữ liệu hiệu quả để đảm bảo việc nhập và khai thác dữ liệu nhanh và chính xác [1]. Cơ sở dữ liệu có nhiều cách tổ chức, trong đó cách hiệu quả nhất là tổ chức theo mô hình quan hệ gọi đó là cơ sở dữ liệu quan hệ.

Cơ sở dữ liệu quan hệ (relational database) là một cơ sở dữ liệu dựa trên mô hình quan hệ dữ liệu, theo đề xuất của Edgar F. Codd vào năm 1970. Một hệ thống phần mềm sử dụng để duy trì cơ sở dữ liệu quan hệ là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS). Nhiều hệ thống cơ sở dữ liệu quan hệ có tùy chọn sử dụng ngôn ngữ truy vấn có cấu trúc (SQL) tiêu chuẩn để truy vấn và duy trì cơ sở dữ liệu. Cơ sở dữ liệu quan hệ tổ chức dữ liệu theo các bảng và có quan hệ với nhau để giảm thiểu sự dư thừa dữ liệu đồng thời vẫn đảm bảo sự hiệu quả trong lưu trữ và truy xuất dữ liệu.

* Các thành phần cơ bản của một cơ sở dữ liệu quan hệ

Bảng dữ liệu (Table): Là thành phần chính trong cơ sở dữ liệu quan hệ. Bảng chứa dữ liệu.



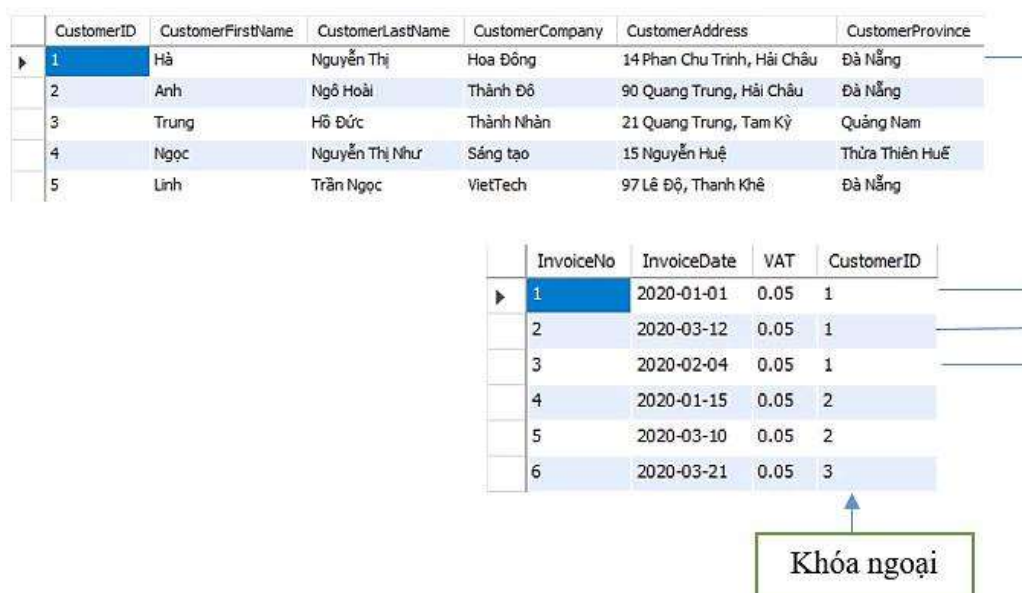
Hình 2.1. Bảng trong cơ sở dữ liệu quan hệ

Trong đó:

- Cột/Trường (Field): Thể hiện thuộc tính của bảng dữ liệu. ví dụ Tên, Địa chỉ,...
- Dòng (Row): Là một dòng dữ liệu gồm các dữ liệu có liên quan với nhau hay còn gọi là bản ghi (Record).
- Ô (Cell): Giao giữa dòng và cột và là nơi chứa dữ liệu.
- Khóa chính (Primary Key): Là một trường hoặc nhiều trường gộp lại được sử dụng để định nghĩa bản ghi. Khóa chính có hai thuộc tính là không được trùng và không được rỗng.

Một bảng có thể có khóa chính hoặc không, tuy nhiên để dễ quản lý người ta thường định nghĩa khóa chính cho các bảng.

Mối quan hệ (Relationship)



Hình 2.2. Minh họa mối quan hệ của bảng

Trong đó:

- Khóa ngoại (Foreign Key): Là trường ở bảng này (bảng Invoice) nhưng có trường tương ứng làm khóa chính ở bảng kia (bảng Customer) để tạo ra mối quan hệ giữa hai bảng.

- Mối quan hệ (Relationship): Tạo ra mối liên kết giữa hai bảng nhằm xác định mối liên quan giữa các trường dữ liệu của hai bảng. Ví dụ: nếu muốn biết khách hàng có mã khách hàng số 1 mua những đơn hàng nào thì phải dựa vào mối quan hệ trên. Trong cơ sở dữ liệu quan hệ mối quan hệ thể hiện ở 3 dạng sau:

- + Quan hệ 1-1: Trong quan hệ này mỗi bảng chỉ có một và chỉ một bản ghi tương ứng mà thôi.
- + Quan hệ 1-n: Là quan hệ phổ biến nhất trong cơ sở dữ liệu, trong quan hệ này 1 bản ghi ở bảng này có nhiều bản ghi tương ứng ở bảng kia. Trong ví dụ trên một bản ghi trong bảng Customer có nhiều bản ghi trong bảng Invoice.
- + Quan hệ n-n: Trong quan hệ này một bản ghi trong bảng này tương ứng với nhiều bản ghi trong bảng kia và ngược lại.

2.1.2. Structured Query Language - SQL

Ngôn ngữ truy vấn có cấu trúc (SQL) là một ngôn ngữ lập trình phục vụ việc lưu trữ và xử lý thông tin trong cơ sở dữ liệu quan hệ. Cơ sở dữ liệu quan hệ lưu trữ thông tin dưới dạng bảng có các hàng và cột đại diện cho những thuộc tính dữ liệu và nhiều mối quan hệ khác nhau giữa các giá trị dữ liệu. Có thể sử dụng các câu lệnh

SQL để lưu trữ, cập nhật, loại bỏ, tìm kiếm và truy xuất thông tin từ cơ sở dữ liệu. Có thể sử dụng SQL để duy trì và tối ưu hóa hiệu suất cơ sở dữ liệu [1].

SQL là một ngôn ngữ tiêu chuẩn để tạo và thao tác với cơ sở dữ liệu. MySQL là một chương trình quản lý cơ sở dữ liệu quan hệ sử dụng các lệnh SQL. Các lệnh SQL được quy định theo tiêu chuẩn quốc tế, còn chương trình MySQL thì được nâng cấp và cải tiến liên tục.

* Lệnh SQL

Lệnh SQL là những từ khóa hoặc câu lệnh SQL cụ thể được các nhà phát triển sử dụng để thao tác với dữ liệu được lưu trữ trong cơ sở dữ liệu quan hệ. Có thể phân loại các lệnh SQL như sau:

Ngôn ngữ định nghĩa dữ liệu (Data Definition Language - DDL): Gồm các lệnh SQL thiết kế cấu trúc cơ sở dữ liệu. Các kỹ sư cơ sở dữ liệu sử dụng DDL để tạo và điều chỉnh các đối tượng cơ sở dữ liệu dựa trên các yêu cầu nghiệp vụ.

Ví dụ:

Bảng 2.1. Một số lệnh định nghĩa dữ liệu.

Lệnh	Mô tả
CREATE	Tạo ra một bảng mới hoặc các đối tượng khác trong cơ sở dữ liệu.
ALTER	Sửa đổi một đối tượng cơ sở dữ liệu hiện có, chẳng hạn như một bảng.
DROP	Xoá toàn bộ một bảng hoặc các đối tượng khác trong cơ sở dữ liệu.

Ngôn ngữ truy vấn dữ liệu (Data Query Language - DQL): Gồm các lệnh hướng dẫn để truy xuất dữ liệu được lưu trữ trong cơ sở dữ liệu quan hệ. Các ứng dụng phần mềm sử dụng lệnh SELECT để lọc và trả về kết quả cụ thể từ một bảng SQL.

Ngôn ngữ thao tác dữ liệu (Data Manipulation Language - DML): Gồm các lệnh viết thông tin mới hoặc điều chỉnh các bản ghi hiện có trong cơ sở dữ liệu quan hệ. Ví dụ: một ứng dụng sử dụng lệnh INSERT để lưu trữ một bản ghi mới trong cơ sở dữ liệu.

Ví dụ:

Bảng 2.2. Một số lệnh thao tác dữ liệu.

Lệnh	Mô tả
INSERT	Tạo một bản ghi.
UPDATE	Chỉnh sửa bản ghi.
DELETE	Xoá bản ghi.

Ngôn ngữ kiểm soát dữ liệu (Data Control Language - DCL): Quản trị viên cơ sở dữ liệu sử dụng ngôn ngữ kiểm soát dữ liệu để quản lý hoặc cấp quyền truy cập cơ sở dữ liệu cho người dùng khác.

Ví dụ:

Bảng 2.3. Một số lệnh kiểm soát dữ liệu.

Lệnh	Mô tả
GRANT	Cung cấp một quyền cho người dùng.
REVOKE	Lấy lại các quyền được cấp từ người dùng.

Ngôn ngữ kiểm soát giao dịch (Transaction Control Language - TCL): Công cụ quan hệ sử dụng ngôn ngữ kiểm soát giao dịch gồm các lệnh để tự động thực hiện các thay đổi với cơ sở dữ liệu. Ví dụ: lệnh ROLLBACK để hoàn tác một thao tác trước đó đã thực hiện.

2.1.3. MySQL

MySQL là một hệ thống quản lý cơ sở dữ liệu quan hệ mã nguồn mở (Relational Database Management System - RDBMS) dựa trên ngôn ngữ truy vấn có cấu trúc (Structured Query Language - SQL) được phát triển, phân phối và hỗ trợ bởi tập đoàn Oracle. MySQL chạy trên hầu hết tất cả các nền tảng: Linux, UNIX và Windows. Các ứng dụng web thường sử dụng MySQL để quản lý cơ sở dữ liệu. Có thể tìm hiểu các thông tin về MySQL tại địa chỉ <http://www.mysql.com>.

MySQL là một phần thiết yếu của hầu hết mọi ứng dụng PHP mã nguồn mở như: WordPress, Joomla, Magento và Drupal. MySQL trở nên phổ biến vì nhiều lý do sau đây:

- MySQL được phát hành theo giấy phép nguồn mở. Vì vậy, người dùng không phải trả tiền để sử dụng nó.
- MySQL là một chương trình quản lý cơ sở dữ liệu mạnh mẽ.
- MySQL sử dụng một dạng chuẩn của ngôn ngữ dữ liệu SQL.
- MySQL hoạt động trên nhiều hệ điều hành và với nhiều ngôn ngữ bao gồm PHP, PERL, C, C ++, JAVA,...
- MySQL hoạt động rất nhanh và ổn định ngay cả với các tập dữ liệu lớn.
- MySQL rất thân thiện với PHP, ngôn ngữ được đánh giá cao nhất để phát triển web.
- MySQL hỗ trợ cơ sở dữ liệu lớn, lên tới 50 triệu bản ghi trong một bảng. Giới hạn kích thước tệp mặc định cho một bảng là 4GB và có thể tăng hạn mức này.
- MySQL là tùy biến. Giấy phép GPL mã nguồn mở cho phép người dùng sửa đổi phần mềm MySQL để phù hợp với môi trường cụ thể.

2.2. Các lệnh cơ bản trong MySQL

2.2.1. Định nghĩa dữ liệu

2.2.1.1. CREATE

Câu lệnh CREATE dùng để tạo cơ sở dữ liệu hoặc tạo bảng.

a. Tạo cơ sở dữ liệu

Cú pháp [3]:

```
CREATE DATABASE databasename;
```

Trong đó:

- CREATE, DATABASE là từ khóa.
- databasename là tên cơ sở dữ liệu cần tạo mới.

Để thống nhất trong cách viết cú pháp các lệnh, các từ khóa sẽ được viết in hoa.

b. Tạo bảng

Cú pháp [3]:

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    column3 datatype,  
    ....  
);
```

Trong đó:

- CREATE, TABLE là từ khóa.
- table_name là tên bảng cần tạo mới.
- column1, column2, ... là tên trường thông tin (cột) của bảng.
- datatype là kiểu dữ liệu của trường thông tin.

Ví dụ:

```
CREATE TABLE Persons (  
    PersonID int,  
    LastName varchar(255),  
    FirstName varchar(255),  
    Address varchar(255),  
    City varchar(255)  
);
```

2.2.1.2. ALTER TABLE

Câu lệnh ALTER TABLE được sử dụng để thêm, xóa hoặc sửa đổi các cột (trường) trong một bảng hiện có. Câu lệnh ALTER TABLE cũng được sử dụng để thêm và loại bỏ các ràng buộc khác nhau trên một bảng hiện có.

a. Thêm cột vào bảng

Cú pháp [3]:

```
ALTER TABLE table_name  
    ADD column_name datatype;
```

Ví dụ:

```
ALTER TABLE Customers  
    ADD Email varchar(255);
```

b. Xóa cột trong bảng

Cú pháp [3]:

```
ALTER TABLE table_name  
    DROP COLUMN column_name;
```

Ví dụ:

```
ALTER TABLE Customers  
    DROP COLUMN Email;
```

c. Đổi tên cột trong bảng

Cú pháp [3]:

```
ALTER TABLE table_name  
    RENAME COLUMN old_name TO new_name;
```

d. Thay đổi kiểu dữ liệu của một cột trong bảng

Cú pháp [3]:

```
ALTER TABLE table_name  
    MODIFY COLUMN column_name datatype;
```

2.2.1.3. DROP

Lệnh DROP dùng để xóa cơ sở dữ liệu hoặc xóa bảng hiện có.

a. Xóa cơ sở dữ liệu

Cú pháp [3]:

```
DROP DATABASE databasename;
```

b. Xóa bảng

Cú pháp [3]:

```
DROP TABLE table_name;
```

2.2.2. Truy vấn dữ liệu - Select

Câu lệnh SELECT được sử dụng để lấy dữ liệu từ cơ sở dữ liệu. Dữ liệu trả về được lưu trữ trong một bảng kết quả, được gọi là tập hợp kết quả.

Cú pháp [3]:

```
SELECT column1, column2, ...  
    FROM table_name;
```

Trong đó column1, column2, ... là tên cột (trường) của bảng. Nếu muốn chọn tất cả các trường có trong bảng thì sử dụng cú pháp sau:

```
SELECT * FROM table_name;
```

Ví dụ, cho bảng Customers:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
2	Ana Frujillo Emparedados y helados	Ana Frujillo	Avda. De la Constitución 2222	México D.F.	05021	Mexico
3	Antonio Moreno Taqueria	Antonio Moreno	Mataderos 2312	México D.F.	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	London	WA1 1DP	UK

Lấy ra thông tin cột "CustomerName" và "City" từ bảng "Customers":

```
SELECT CustomerName, City FROM Customers;
```

Kết quả:

CustomerName	City
Alfreds Futterkiste	Berlin
Ana Frujillo Emparedados y helados	México D.F.
Antonio Moreno Taqueria	México D.F.
Around the Horn	London
Frankenversand	München
Blauer See Delikatessen	Mannhelm

Kết hợp mệnh đề WHERE

Mệnh đề WHERE được sử dụng để lọc các bản ghi. Nó được sử dụng để chỉ trích xuất những bản ghi đáp ứng một điều kiện (condition) cụ thể.

Cú pháp [3]:

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Ví dụ với bảng Customers ở trên, chúng ta có thể lọc ra danh sách các bản ghi có CustomerID = 1:

```
SELECT * FROM Customers
WHERE CustomerID=1;
```

Kết quả:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany

Toán tử trong mệnh đề WHERE

Các toán tử sau có thể được sử dụng trong mệnh đề WHERE:

Bảng 2.4. Các toán tử trong mệnh đề Where

Toán tử	Mô tả
=	Bằng
>	Lớn hơn
<	Nhỏ hơn
>=	Lớn hơn hoặc bằng
<=	Nhỏ hơn hoặc bằng
<>	Không bằng. Chú ý, một số phiên bản của SQL sử dụng toán tử !=
BETWEEN	Giữa một phạm vi nhất định
LIKE	Tìm kiếm một mẫu
IN	Để chỉ định nhiều giá trị có thể có cho một cột (nhận các giá trị trong các giá trị nào đó)

Kết hợp với các toán tử AND, OR và NOT

- Toán tử AND và OR sử dụng để lọc các bản ghi dựa trên nhiều điều kiện:
 - + Toán tử AND hiển thị một bản ghi nếu **tất cả** các điều kiện được phân tách bằng AND là TRUE.
 - + Toán tử OR hiển thị một bản ghi nếu **bất kỳ** điều kiện nào được phân tách bằng OR là TRUE.
- Toán tử NOT hiển thị một bản ghi nếu (các) điều kiện KHÔNG ĐÚNG.

Cú pháp:

```

SELECT column1, column2, ...
    FROM table_name
    WHERE condition1 AND condition2 AND condition3 ...;

SELECT column1, column2, ...
    FROM table_name
    WHERE condition1 OR condition2 OR condition3 ...;

SELECT column1, column2, ...
    FROM table_name
    WHERE NOT condition;
    
```

Ví dụ 1:

```

SELECT * FROM Customers
    WHERE Country='Germany' AND City='Berlin';
    
```

Kết quả:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany

Ví dụ 2:

```
SELECT * FROM Customers
WHERE City='Berlin' OR City='München';
```

Kết quả:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
25	Frankenversand	Peter Franken	Berliner Platz 43	München	80805	Germany

Ví dụ 3:

```
SELECT * FROM Customers
WHERE Country='Germany' AND (City='Berlin' OR City='Leipzig');
```

Kết quả:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	Berlin	12209	Germany
52	Morgenstern Gesundkost	Alexander Feuer	Heerstr. 22	Leipzig	04179	Germany

* Sắp xếp dữ liệu trích xuất

Từ khóa ORDER BY được sử dụng để sắp xếp tập kết quả theo thứ tự tăng dần (ASC) hoặc giảm dần (DESC). Theo mặc định, từ khóa ORDER BY sắp xếp các bản ghi theo thứ tự tăng dần. Để sắp xếp các bản ghi theo thứ tự giảm dần, hãy sử dụng từ khóa DESC.

Cú pháp [3]:

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;
```

Ví dụ 1: Lấy ra toàn bộ các bản ghi trong bảng Customers và sắp xếp theo thứ tự tăng dần của trường thông tin Country.

```
SELECT * FROM Customers
ORDER BY Country;
```

Kết quả:

CustomerID	CustomerName	ContactName	Address	City	PostalCode	Country
12	Cactus Comidas para llevar	Patricio Simpson	Cerrito 333	Buenos Aires	1010	Argentina
54	Océano Atlántico Ltda.	Yvonne Moncada	Ing. Gustavo Moncada 8585 Plso 20-A	Buenos Aires	1010	Argentina
64	Rancho grande	Sergio Gutiérrez	Av. del Libertador 900	Buenos Aires	1010	Argentina
20	Ernst Handel	Roland Mendel	Kirchgasse 6	Graz	8010	Austria
59	Piccolo und mehr	Georg Pipps	Geislweg 14	Salzburg	5020	Austria
50	Maison Dewey	Catherine Dewey	Rue Joseph-Bens 532	Bruxelles	B-1180	Belgium
76	Suprêmes délices	Pascale Cartrain	Boulevard Tirou, 255	Charleroi	B-6000	Belgium
15	Comércio Mineiro	Pedro Afonso	Av. dos Lusíadas, 23	São Paulo	05432-043	Brazil
21	Familia Arquibaldo	Aria Cruz	Rua Orós, 92	São Paulo	05442-030	Brazil
31	Gourmet Lanchonetes	André Fonseca	Av. Brasil, 442	Campinas	04876-786	Brazil
34	Hanari Carnes	Mario Pontes	Rua do Paço, 67	Rio de Janeiro	05454-876	Brazil
61	Que Delícia	Bernardo Batista	Rua da Panificadora, 12	Rio de Janeiro	02389-673	Brazil
62	Queen Cozinha	Lúcia Carvalho	Alameda dos Canários, 891	São Paulo	05487-020	Brazil
67	Ricardo Adocicados	Janete Limeira	Av. Copacabana, 267	Rio de Janeiro	02389-890	Brazil
81	Tradição Hipermercados	Anabela Domingues	Av. Inês de Castro, 414	São Paulo	05634-030	Brazil
88	Wellington Importadora	Paula Parente	Rua do Mercado, 12	Resende	08737-363	Brazil
10	Bottom-Dollar Marketse	Elizabeth Lincoln	23 Tsawassen Blvd.	Tsawassen	T2F 8M4	Canada

Ví dụ 2: Lấy ra toàn bộ các bản ghi trong bảng Customers và sắp xếp trường thông tin Country theo thứ tự tăng dần, CustomerName theo thứ tự giảm dần.

```
SELECT * FROM Customers
ORDER BY Country ASC, CustomerName DESC;
```

2.2.3. Thao tác dữ liệu

2.2.3.1. INSERT INTO

Câu lệnh INSERT INTO được sử dụng để chèn/thêm các bản ghi mới vào một bảng trong cơ sở dữ liệu.

Cú pháp 1: Chỉ định tên cột và giá trị được chèn.

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

Ví dụ:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City,
PostalCode, Country) VALUES ('Cardinal', 'Tom B.
Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

Cú pháp 2: Thêm giá trị cho tất cả các cột của bảng, không cần chỉ định tên cột trong truy vấn SQL. Lưu ý thứ tự của các giá trị tương ứng với thứ tự của các cột trong bảng.

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Ví dụ:

```
INSERT INTO Customers VALUES ('Cardinal', 'Tom B.
Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```

2.2.3.2. UPDATE

Câu lệnh UPDATE sử dụng để sửa đổi các bản ghi hiện có trong một bảng.

Cú pháp:

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Ví dụ:

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City= 'Frankfurt'
WHERE CustomerID = 1;

UPDATE Customers
SET ContactName='Juan'
WHERE Country='Mexico';
```

2.2.3.3. DELETE

Câu lệnh DELETE được sử dụng để xóa các bản ghi hiện có trong một bảng.

Cú pháp:

```
DELETE FROM table_name WHERE condition;
```

Ví dụ:

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futtarkiste';
```

Nếu không có mệnh đề WHERE thì sẽ xóa toàn bộ dữ liệu của bảng.

Ví dụ:

```
DELETE FROM Customers;
```

2.3. Sử dụng phpMyAdmin

PhpMyAdmin là phần mềm mã nguồn mở được viết bằng ngôn ngữ PHP giúp quản trị cơ sở dữ liệu MySQL thông qua giao diện web. Đối với việc sử dụng dịch vụ từ nhà cung cấp Hosting có thể truy cập phpMyAdmin thông qua Cpanel bằng thông tin đăng nhập hoặc tài khoản Cpanel. Trường hợp sử dụng localhost bằng cách dùng các phần mềm như Xampp, Wampserver,... đường dẫn truy cập sẽ có dạng: <http://localhost/phpmyadmin>.

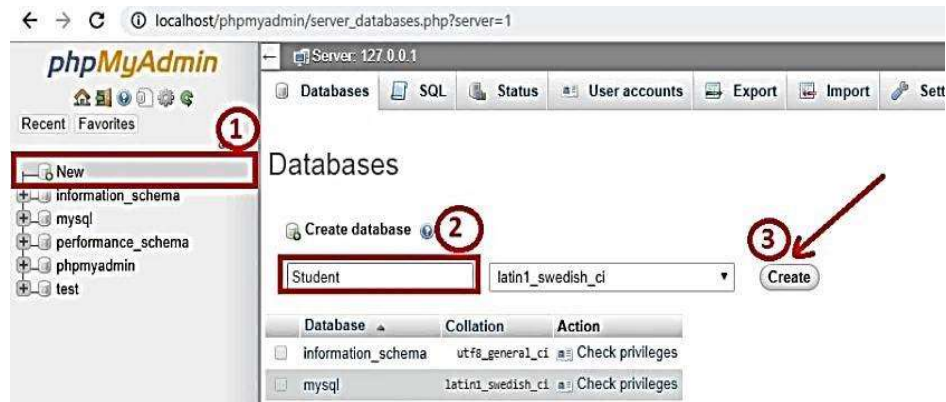
Nếu dùng localhost, thông thường sẽ được yêu cầu tạo tài khoản khi cài đặt phần mềm hoặc sử dụng tài khoản mặc định (đối với các phiên bản mới). Thông thường tài khoản mặc định trên localhost có tên đăng nhập là root, mật khẩu để trống (rỗng).

2.3.1. Tạo cơ sở dữ liệu

Để tạo cơ sở dữ liệu MySQL trên giao diện phpMyAdmin chúng ta thực hiện các bước như sau:

- Bước 1: Bấm vào New (1).
- Bước 2: Nhập tên cơ sở dữ liệu vào trường Create database (2).
- Bước 3: Bấm nút Create (3).

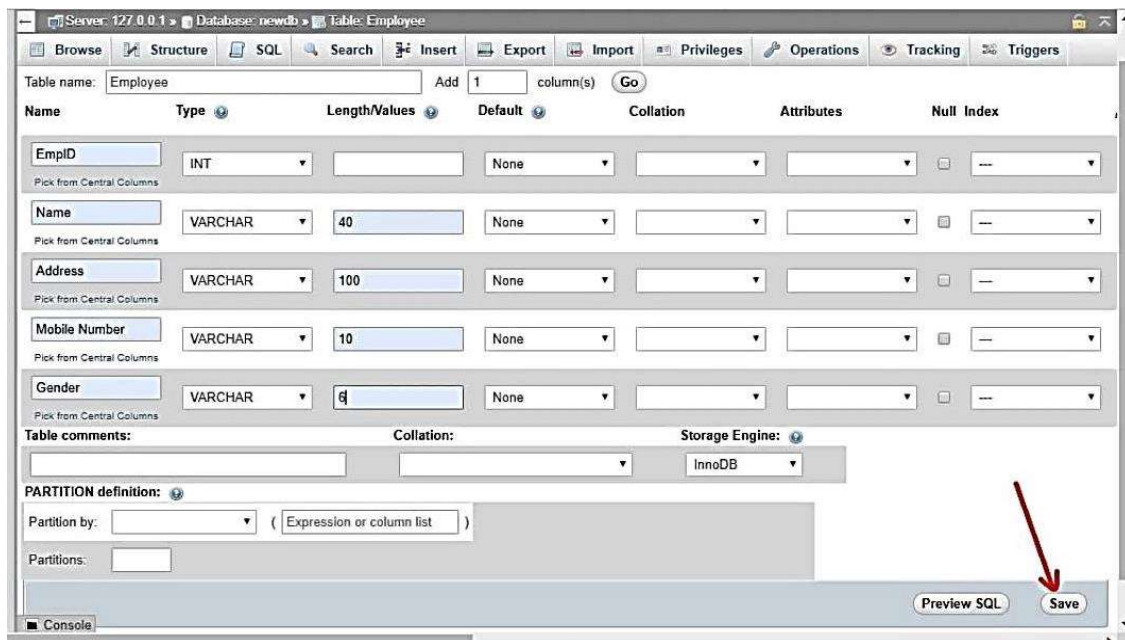
Kết quả sẽ xuất hiện một cơ sở dữ liệu mới trong cột bên trái giao diện nếu thao tác thành công.



Hình 2.3. Tạo cơ sở dữ liệu

2.3.2. Tạo bảng

- Chọn cơ sở dữ liệu cần tạo bảng trong cột bên trái của giao diện quản trị của phpMyAdmin.
- Nhập các thông tin như tên bảng, số cột và nhấn nút Go tại cột bên phải của giao diện. Xuất hiện thông báo tạo bảng thành công và tên bảng sẽ hiển thị trong cơ sở dữ liệu tại cột bên trái của giao diện quản trị.
- Nhập thông tin tên trường, loại, kích thước,... các trường (cột) của bảng.
- Nhấn nút Save để lưu.



Hình 2.4. Tạo trường thông tin của bảng

Các thông tin của trường dữ liệu trong bảng gồm:

- Name: Tên trường của bảng. Không sử dụng Tiếng Việt, dấu trắng và các ký tự đặc biệt.

- Type: Kiểu dữ liệu của trường thông tin. Mỗi trường thông tin sẽ có kiểu dữ liệu phù hợp với thông tin của nó. Ví dụ với trường thông tin về họ tên, địa chỉ nên sử dụng kiểu dữ liệu varchar để lưu thông tin dạng văn bản. Với trường thông tin giới tính có thể sử dụng kiểu dữ liệu varchar để lưu thông tin dưới dạng văn bản (Nam, Nữ) hoặc có thể sử dụng kiểu dữ liệu int/tinyint/bit/boolean,... để lưu thông tin dưới dạng số (0 hoặc 1), khi đó người lập trình sẽ phải xử lý quy ước Nam là 0 hoặc 1 và Nữ là giá trị còn lại. Các kiểu dữ liệu thường sử dụng cho trường thông tin của bảng trong cơ sở dữ liệu MySQL bao gồm:

Bảng 2.5. Kiểu dữ liệu trong MySQL

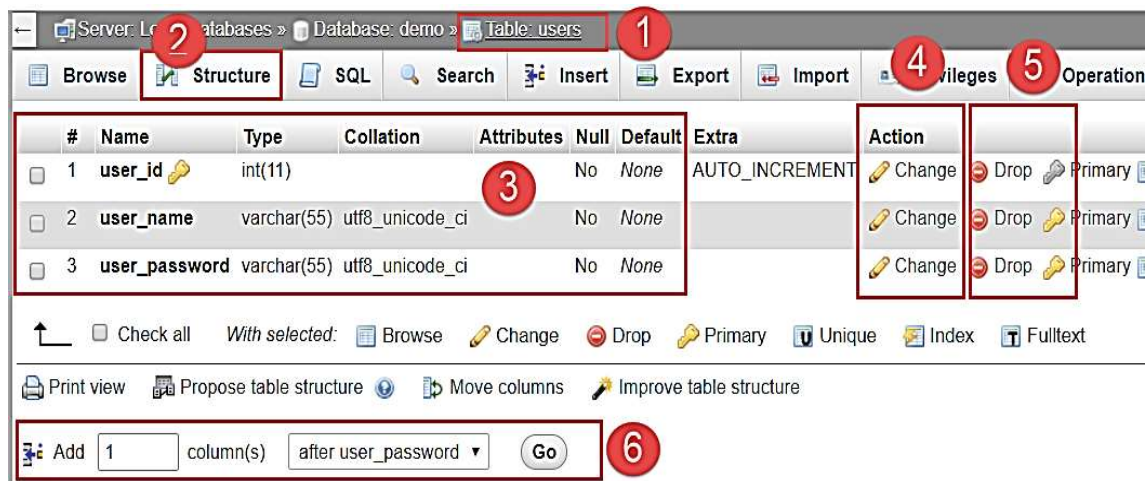
Kiểu dữ liệu	Mô tả
CHAR (size)	Dữ liệu kiểu chuỗi độ dài CỐ ĐỊNH (có thể chứa các chữ cái, số và ký tự đặc biệt). Tham số kích thước (size) chỉ định độ dài cột theo ký tự, có thể từ 0 đến 255 ký tự. Mặc định là 1 ký tự.
VARCHAR (size)	Dữ liệu kiểu chuỗi có độ dài THAY ĐỔI (có thể chứa các chữ cái, số và ký tự đặc biệt). Tham số kích thước chỉ định độ dài cột tối đa tính bằng ký tự, có thể từ 0 đến 65535 ký tự.
TEXT (size)	Lưu trữ một chuỗi có độ dài tối đa là 65.535 byte.
BIT (size)	Dữ liệu kiểu giá trị bit. Số lượng bit trên mỗi giá trị được chỉ định trong kích thước. Tham số kích thước có thể chứa giá trị từ 1 đến 64. Giá trị mặc định cho kích thước là 1.
TINYINT (size)	Dữ liệu kiểu số nguyên rất nhỏ. Phạm vi số có dấu từ -128 đến 127. Phạm vi số không dấu từ 0 đến 255. Tham số kích thước chỉ định chiều rộng hiển thị tối đa (là 255).
BOOL	Dữ liệu kiểu logic (đúng/sai). Giá trị 0 được coi là sai (False), giá trị khác 0 được coi là đúng (True).
BOOLEAN	Giống với BOOL.
SMALLINT (size)	Dữ liệu kiểu số nguyên nhỏ. Phạm vi số có dấu từ -32768 đến 32767. Phạm vi số không dấu từ 0 đến 65535. Tham số kích thước chỉ định chiều rộng hiển thị tối đa (là 255).
MEDIUMINT (size)	Dữ liệu kiểu số nguyên trung bình. Phạm vi số có dấu từ -8388608 đến 8388607. Phạm vi số không dấu từ 0 đến 16777215. Tham số kích thước chỉ định chiều rộng hiển thị tối đa (255).
INT (size)	Dữ liệu kiểu số nguyên. Phạm vi số có dấu từ -2147483648 đến 2147483647. Phạm vi số không dấu từ 0 đến 4294967295. Tham số kích thước chỉ định chiều rộng hiển thị tối đa (là 255).

Kiểu dữ liệu	Mô tả
INTEGER (size)	Giống với INT (size)
BIGINT (size)	Dữ liệu kiểu số nguyên lớn. Phạm vi có dấu từ -9223372036854775808 đến 9223372036854775807. Phạm vi không dấu từ 0 đến 18446744073709551615. Tham số kích thước chỉ định chiều rộng hiển thị tối đa (là 255).
FLOAT (p)	Dữ liệu kiểu số dấu phẩy động. Giá trị p xác định nên sử dụng FLOAT hay DOUBLE cho kiểu dữ liệu kết quả. Nếu p từ 0 đến 24, kiểu dữ liệu sẽ trở thành FLOAT(). Nếu p từ 25 đến 53, kiểu dữ liệu trở thành DOUBLE().
DOUBLE (size, d)	Dữ liệu kiểu số dấu phẩy động kích thước bình thường. Tổng số chữ số được chỉ định trong kích thước. Số chữ số sau dấu thập phân được chỉ định trong tham số d.
DECIMAL (size, d)	Đây là dạng dữ liệu số lưu trữ dưới dạng chuỗi và có dấu thập phân cố định. Trong đó, tham số size chỉ định số lượng chữ số tối đa. Còn tham số d chỉ định số lượng chữ số tối đa của số chữ số sau dấu thập. Số lượng tối đa cho kích thước là 65. Số lượng tối đa cho d là 30. Giá trị mặc định cho kích thước là 10. Giá trị mặc định cho d là 0.
DATE	Dữ liệu kiểu ngày. Định dạng: YYYY-MM-DD. Phạm vi được hỗ trợ là từ '1000-01-01' đến '9999-12-31'.
DATETIME (fsp)	Dữ liệu kiểu kết hợp ngày và thời gian. Định dạng: YYYY-MM-DD hh:mm:ss. Phạm vi được hỗ trợ từ '1000-01-01 00:00:00' đến '9999-12-31 23:59:59'. Thêm MẶC ĐỊNH và BẬT CẬP NHẬT trong định nghĩa cột để tự động khởi tạo và cập nhật ngày giờ hiện tại.
TIMESTAMP (fsp)	Dữ liệu kiểu thời gian. Các giá trị thời gian được lưu trữ dưới dạng số giây từ ('1970-01-01 00:00:00' UTC). Định dạng: YYYY-MM-DD hh:mm:ss. Phạm vi được hỗ trợ từ '1970-01-01 00:00:01' UTC đến '2038-01-09 03:14:07' UTC. Có thể chỉ định tự động khởi tạo và cập nhật ngày giờ hiện tại bằng cách sử dụng MẶC ĐỊNH CURRENT_TIMESTAMP và BẬT CẬP NHẬT CURRENT_TIMESTAMP trong định nghĩa cột.
TIME (fsp)	Dữ liệu kiểu thời gian. Định dạng: hh:mm:ss. Phạm vi được hỗ trợ là từ '-838:59:59' đến '838:59:59'
YEAR	Dữ liệu kiểu năm ở định dạng bốn chữ số. Các giá trị được phép ở định dạng bốn chữ số: 1901 đến 2155 và 0000.

- Length/Values: Giá trị tham số của dữ liệu (tham số size, d, p ...).
- Default: Chỉ định giá trị mặc định cho trường dữ liệu nếu không được thiết lập (nhập vào).
- Collation: Kiểu mã hóa dữ liệu.
- Attributes: Thuộc tính khác.
- Null: Cho phép dữ liệu rỗng hoặc không rỗng.
- Index: Chỉ định là trường khóa chính hay khóa phụ hoặc không.
- A_I: Chỉ định dữ liệu tự động tăng hoặc không khi thêm dữ liệu vào bảng (chỉ áp dụng với kiểu dữ liệu số, thường áp dụng cho trường khóa của bảng).

2.3.3. Xem cấu trúc bảng

Khi đã tạo bảng thành công, muốn xem lại cấu trúc của bảng, xem lại bảng đó gồm có những cột nào, xem lại bảng đó khóa chính là cột nào,... hoặc là muốn chỉnh sửa lại tên cột, giá trị,... thì thực hiện các lệnh sau:



Hình 2.5. Xem cấu trúc bảng

- (1) Chọn bảng cần xem cấu trúc. Chú ý, trước đó phải chọn cơ sở dữ liệu chứa bảng cần xem.
- (2) Chọn Structure.
- (3) Danh sách các cột đã tạo.
- (4) Sửa thông tin cột (tên cột, kiểu giá trị, khóa chính,...).
- (5) Xóa cột.
- (6) Thêm cột mới vào trong bảng.

2.3.4. Tạo quan hệ

PhpMyAdmin cho phép tạo các mối quan hệ (khóa ngoại - ràng buộc FOREIGN KEY). Có hai cách để tạo các mối quan hệ: Dạng xem mối quan hệ (Relation view) và trình thiết kế (Designer).

Hiện tại, loại bảng MySQL hỗ trợ các mối quan hệ là InnoDB được chỉ định trong lựa chọn Storage Engine khi tạo bảng.

Hình 2.6. Lựa chọn Storage Engine

Khi sử dụng bảng InnoDB, phpMyAdmin sẽ tạo các quan hệ InnoDB và sẽ được MySQL thực thi khi các ứng dụng truy cập cơ sở dữ liệu. Đối với loại bảng khác InnoDB, phpMyAdmin thực thi các mối quan hệ bên trong và những mối quan hệ đó không được áp dụng cho bất kỳ ứng dụng nào khác.

Để minh họa cho kỹ thuật tạo quan hệ, chúng ta có hai bảng Customers và Orders lần lượt như sau:

Bảng Customers lưu trữ thông tin của các khách hàng.

Bảng 2.6. Bảng Customers.

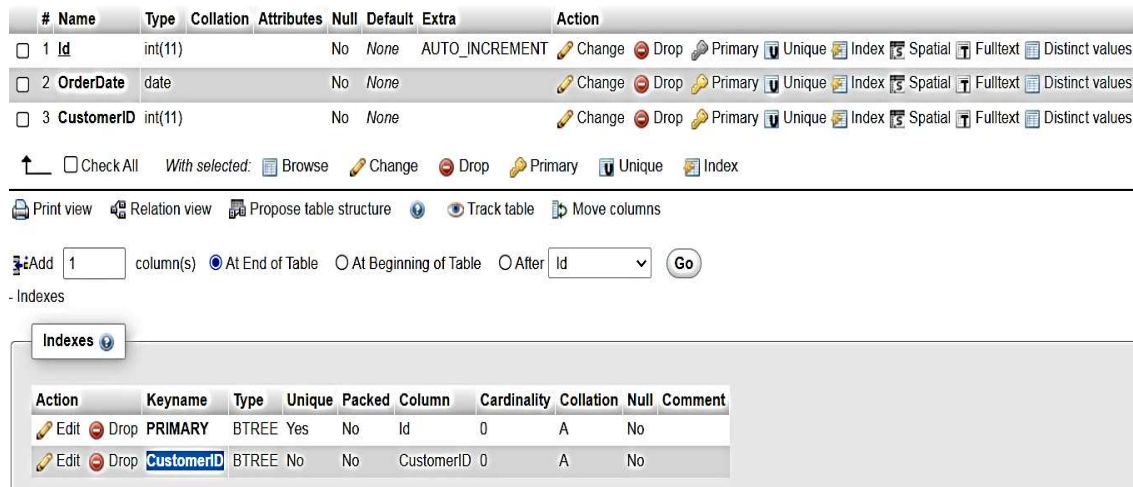
ID	Name	Address	City
1	Nguyễn Thành Nhân	63 Hoàng Văn Thụ	Cần Thơ
2	Dương Văn Gánh	16B Nguyễn Văn Cừ	Sóc Trăng
3	Trần Thị Huỳnh Như	365A Trần Quang Diệu	Vĩnh Long

Bảng Orders lưu trữ thông tin về các đơn đặt hàng của những khách hàng trong bảng Customers.

Bảng 2.7. Bảng Orders.

ID	OrderDate	CustomerID
1	2019-08-05	1
2	2019-09-17	2
3	2019-10-22	2
4	2019-11-08	3

Cột CustomerID trong bảng Orders tham chiếu dữ liệu từ cột ID trong bảng Customers, nó cho biết những khách hàng nào đã đặt những đơn hàng nào (nếu thêm dữ liệu vào bảng Order những đơn hàng mà ID của khách hàng không tồn tại thì những đơn hàng đó sẽ trở thành không hợp lệ). Để tham chiếu, phải thiết lập cột CustomerID trong bảng Orders là Index. Nếu chưa thiết lập có thể vào chế độ xem cấu trúc bảng để chỉnh sửa.



Hình 2.7. Cấu trúc bảng Orders

Để tránh việc bảng Orders bị chứa những đơn hàng không hợp lệ thì cần phải thiết lập ràng buộc FOREIGN KEY cho cột CustomerID, ràng buộc này đảm bảo việc ngăn chặn những hành động không hợp lệ, diễn hình như:

- Không cho phép thêm những đơn hàng mà ID của khách hàng không tồn tại trong bảng Customers.
- Không cho phép xóa những khách hàng mà ID của họ có tồn tại trong bảng Orders.
- Không cho phép xóa bảng Customers khi bảng Orders vẫn còn đang tồn tại.

Trong hai bảng, bảng chứa ràng buộc FOREIGN KEY (khóa ngoại) được gọi là bảng con, còn bảng chứa khóa chính thì được gọi là bảng tham chiếu hoặc bảng cha (Trong ví dụ phía trên, bảng Orders được gọi là bảng con, bảng Customers được gọi là bảng tham chiếu).

Để tạo một ràng buộc FOREIGN KEY khi tạo mới bảng chúng ta có thể sử dụng cú pháp câu lệnh SQL như sau:

```
CONSTRAINT tên_ràng_buộc FOREIGN
KEY (column1) REFERENCES table_name(column2)
```

Trong đó:

- column1 là tên của cột (bên trong bảng con) mà chúng ta muốn thiết lập ràng buộc FOREIGN KEY.
- table_name là tên của bảng tham chiếu.

- column2 là tên của cột (bên trong bảng tham chiếu) mà cột column1 tham chiếu đến, ngoài ra column2 còn phải là khóa chính (PRIMARY KEY) của cái bảng tham chiếu.

Ví dụ: Dưới đây là đoạn mã dùng để tạo hai bảng Customers và Orders giống như trong ví dụ phía trên.

```
CREATE TABLE Customers(  
    ID INT PRIMARY KEY,  
    Name VARCHAR(255),  
    Address VARCHAR(255),  
    City VARCHAR(255)  
);  
  
CREATE TABLE Orders(  
    ID INT PRIMARY KEY,  
    OrderDate DATE,  
    CustomerID INT,  
    CONSTRAINT lien_ket_01 FOREIGN KEY (CustomerID) REFERENCES  
    Customers(ID)  
);
```

Đối với trường hợp bảng đã được tạo trước, nhưng do quên thiết lập ràng buộc FOREIGN KEY cho cột nên bây giờ muốn bổ sung, để làm được điều đó thì sử dụng lệnh chỉnh sửa bảng như sau:

```
ALTER TABLE table1 ADD CONSTRAINT tên_ràng_buộc FOREIGN  
    KEY (column1) REFERENCES table2(column2);
```

Ví dụ:

```
ALTER TABLE Orders  
    ADD CONSTRAINT lien_ket_01 FOREIGN KEY (CustomerID)  
    REFERENCES Customers(ID);
```

Ngoài phương pháp sử dụng lệnh như trên, có thể sử dụng thiết lập Relation view cho bảng bằng cách kích chọn nút Relation view trong chế độ xem cấu trúc của bảng hoặc sử dụng trình thiết kế Designer.

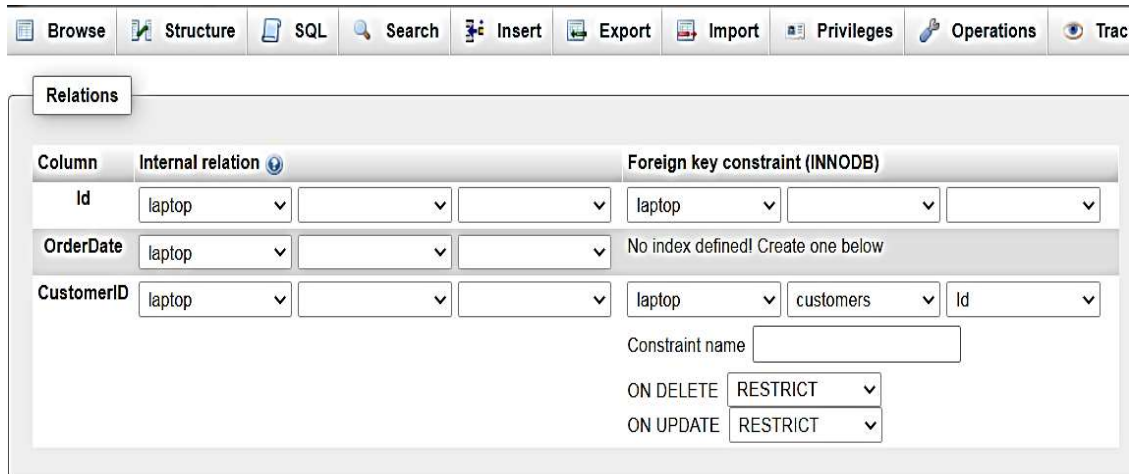
* Relation view

- Chọn bảng Orders, chọn chế độ xem cấu trúc bảng (Structure).
- Chọn Relation view trong chế độ xem cấu trúc bảng (phía dưới).
- Trong giao diện tiếp theo, bên trái là các cột của bảng Orders, bên phải là mục Foreign key constraint cần thiết lập. Tên cơ sở dữ liệu được chỉ định sẵn cho các thiết lập. Chỉ các cột của bảng Orders là trường khóa chính (Primary) hoặc chỉ số (Index) mới xuất hiện các ô lựa chọn cho thiết lập Foreign key.

- Chọn bảng Customers, cột Id tương ứng với cột CustomerID để thiết lập ràng buộc khóa ngoại.

- Thiết lập các tùy chọn cho khóa ngoại khi các sự kiện ON DELETE và ON UPDATE xảy ra. Một số phiên bản MySQL có thể lỗi khi thay đổi giá trị mặc định. Vì vậy hãy lựa chọn giá trị mặc định RESTRICT.

- Nhấn nút Save để lưu thiết lập.



Hình 2.8. Tạo quan hệ trong Relation view

* Designer

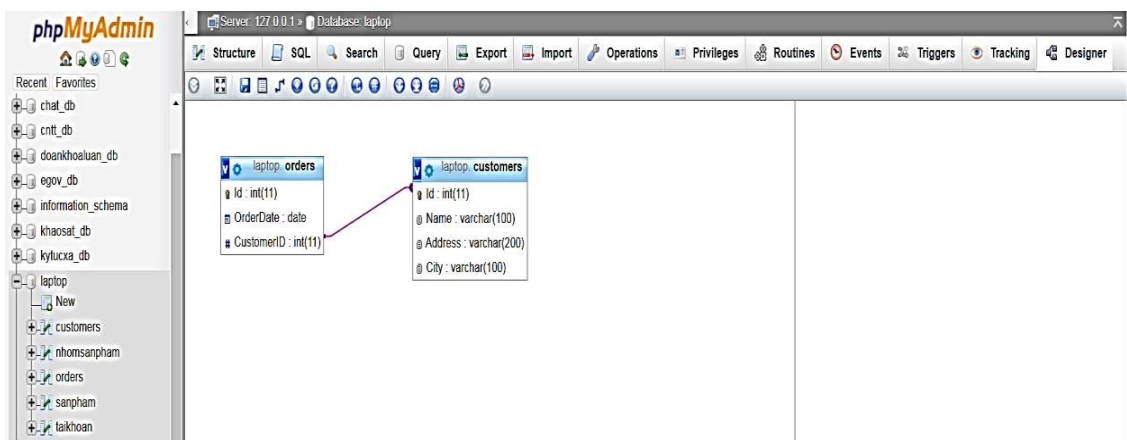
- Chọn cơ sở dữ liệu, chọn trình thiết kế (Designer).

- Chọn Create relation trên thanh công cụ.

- Kích chọn cột CustomerID của bảng Orders (select referenced key).

- Tiếp theo, kích chọn cột Id của bảng Customers (select foreign key). Xuất hiện hộp thoại Create relation FOREIGN KEY.

- Kích chọn OK để lựa chọn giá trị mặc định cho sự kiện ON DELETE và ON UPDATE đồng thời kết thúc quá trình thiết lập quan hệ sử dụng trình thiết kế Designer.



Hình 2.9. Tạo quan hệ trong Designer

2.3.5. Quản lý dữ liệu bảng

2.3.5.1. Thêm 1 dòng (bản ghi) dữ liệu vào bảng

Để thêm mới một dòng (bản ghi) vào bảng ta thực hiện các bước sau:

Server: Local Databases » Database: demo » Table: users

1

Column	Type	Function	Null	Value
user_id	int(11)			
user_name	varchar(55)			admin
user_password	varchar(55)			123456

2 3 4 Go

Hình 2.10. Thêm dữ liệu vào bảng

- (1) Chọn bảng cần thêm dữ liệu, chọn Insert để thêm dữ liệu.
- (2) + (3) Nhập giá trị cho cột.
- (4) Chọn Go để thêm dữ liệu.

2.3.5.2. Xem dữ liệu đã nhập vào trong bảng

Một bảng gồm nhiều dòng dữ liệu, hiển thị danh sách dữ liệu trong bảng được thực hiện theo các lệnh sau:

Server: Local Databases » Database: demo » Table: users

1

2 Browse Structure SQL Search Insert Export Import

Showing rows 0 - 0 (1 total, Query took 0.0004 seconds.)

SELECT * FROM `users`

Show all | Number of rows: 25 | Filter rows: Search this table

Options

	user_id	user_name	user_password
☐ Edit Copy Delete	1	admin	123456

3

Check all With selected: Edit Copy Delete Export

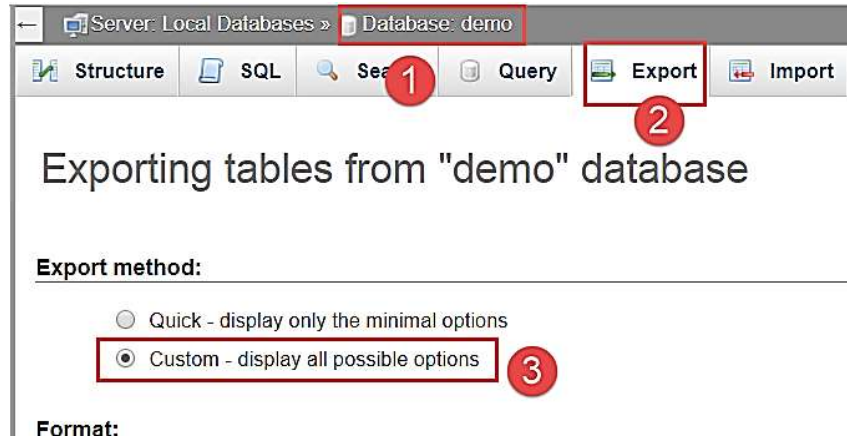
Show all | Number of rows: 25 | Filter rows: Search this table

Hình 2.11. Xem dữ liệu trong bảng

- (1) Chọn bảng cần xem dữ liệu.
- (2) Chọn Browse.
- (3) Vùng hiển thị các dòng trong bảng.

2.3.6. Xuất dữ liệu ra tệp

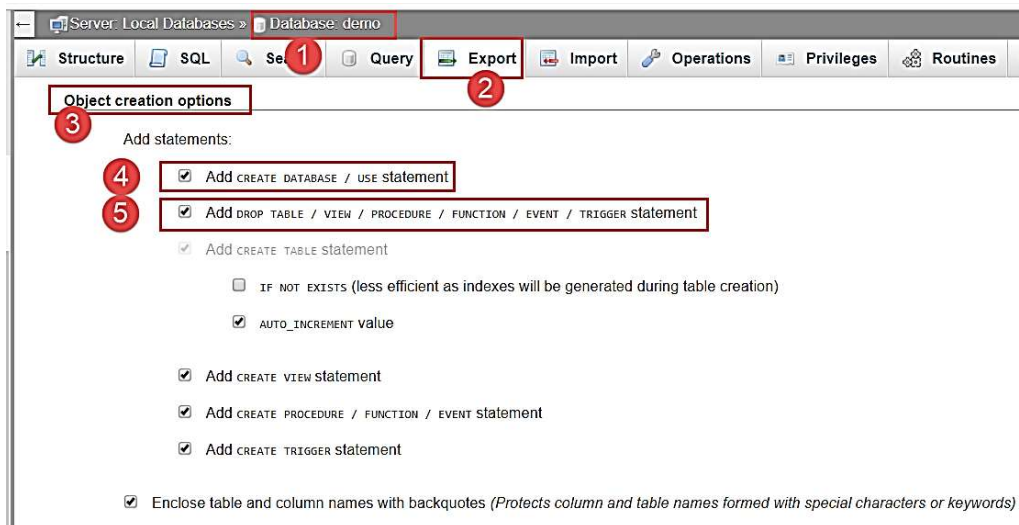
Xuất dữ liệu ra tệp nhằm nhiều mục đích như: backup dữ liệu, phòng tránh dữ liệu có bị mất mát,...và một việc quan trọng là trường hợp chúng ta cần chuyển toàn bộ dự án từ máy tính này sang máy tính khác. Chúng ta thực hiện các bước sau:



Hình 2.12. Xuất dữ liệu

- (1) Chọn cơ sở dữ liệu hoặc bảng.
- (2) Chọn Export.
- (3) Chọn mục Quick hoặc Custom. Chọn Custom để thiết lập thêm 1 số tùy chọn. Chọn Quick để thiết lập tùy chọn mặc định.

Trường hợp ở bước (3) chọn Quick thì nhấn nút Go để hoàn tất lệnh. Trường hợp bước (3) chọn Custom thì thiết lập thêm các tùy chọn khác sau:



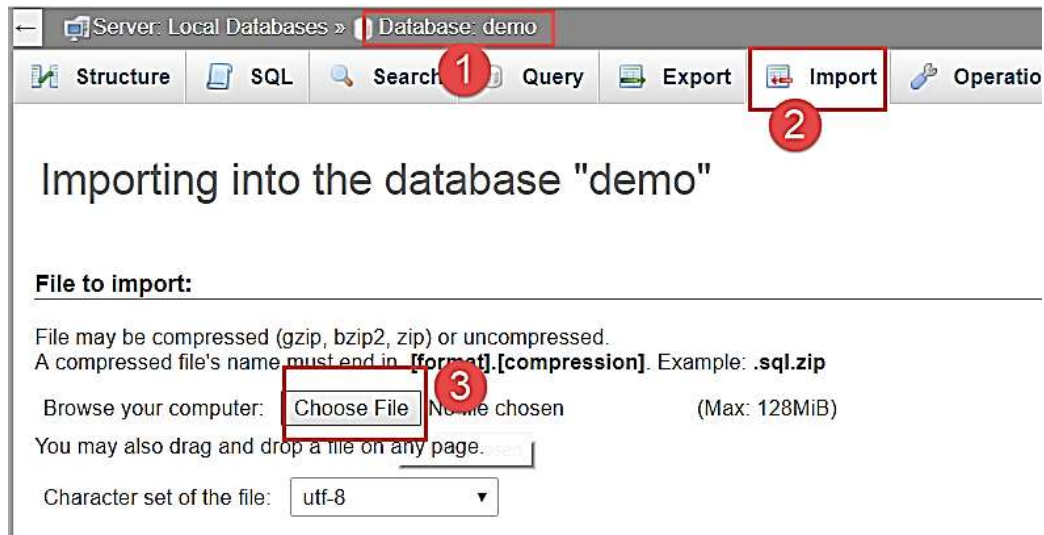
Hình 2.13. Tùy chọn xuất dữ liệu

- (4) Thêm câu lệnh tạo database.
- (5) Thêm câu lệnh tạo table.
- Sau cùng, nhấn nút Go để xuất dữ liệu. Tất cả dữ liệu trong cơ sở dữ liệu sẽ được lưu thành 1 file, file này có phần mở rộng là sql.

2.3.7. Nhập dữ liệu từ tệp

Để nhập dữ liệu từ tệp *.sql ta thực hiện các bước sau:

- (1) Tạo và chọn cơ sở dữ liệu cần nhập dữ liệu từ file.
- (2) Chọn Import.
- (3) Chọn file *.sql (file được tạo ra từ thao tác xuất dữ liệu trên).
- (4) Nhấn nút Go để thực hiện.



Hình 2.14. Nhập dữ liệu từ file

CÂU HỎI ÔN TẬP CHƯƠNG 2

Câu 1. Trình bày các thành phần cơ bản của một cơ sở dữ liệu quan hệ.

Câu 2. Trình bày các mối quan hệ của bảng trong cơ sở dữ liệu quan hệ. Phân loại.

Câu 3. Trình bày cú pháp các lệnh định nghĩa dữ liệu phổ biến. Ví dụ minh họa.

Câu 4. Trình bày cú pháp các lệnh truy vấn dữ liệu phổ biến. Ví dụ minh họa.

Câu 5. Trình bày cú pháp các lệnh thao tác dữ liệu phổ biến. Ví dụ minh họa.

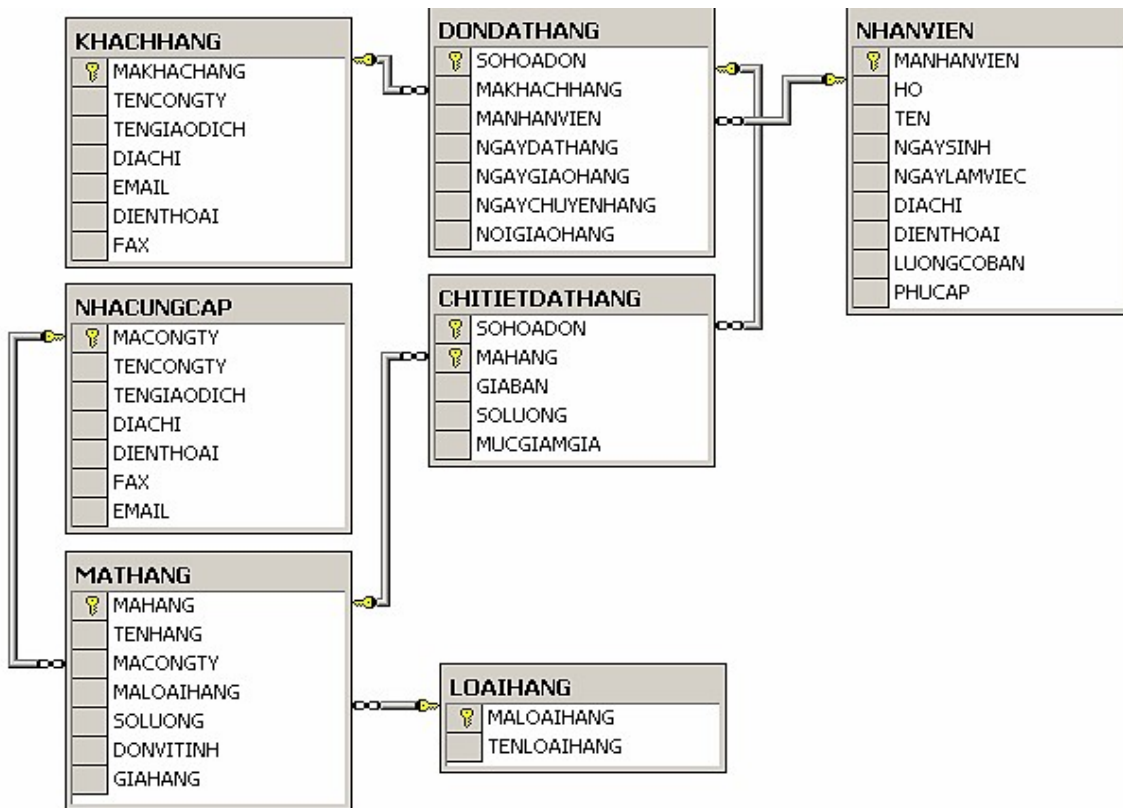
Câu 6. Trình bày cú pháp lệnh Select sử dụng mệnh đề Where. Ví dụ minh họa.

Câu 7. Trình bày các kiểu dữ liệu cơ bản trong MySQL.

Câu 8. Trình bày các thao tác quản trị cơ sở dữ liệu MySQL trong phpMyAdmin.

Câu 9. Thực hiện các yêu cầu sau:

a. Tạo cơ sở dữ liệu, quan hệ và nhập thông tin minh họa cho các bảng như hình dưới (tự lựa chọn kiểu dữ liệu cho phù hợp).



Trong đó:

- Bảng NHACUNGCAP lưu trữ dữ liệu các nhà cung cấp hàng.
- Bảng MATHANG lưu trữ dữ liệu về các mặt hàng hiện có trong công ty.
- Bảng LOAIHANG phân loại các mặt hàng hiện có.
- Bảng NHANVIEN lưu trữ dữ liệu về các nhân viên làm việc trong công ty.
- Bảng KHACHHANG lưu trữ thông tin về các khách hàng của công ty.

- Bảng DONDATHANG lưu trữ thông tin về các đơn đặt hàng.
 - Bảng CHITIETDATHANG lưu trữ thông tin chi tiết của các đơn đặt hàng.
- b. Sử dụng câu lệnh SELECT, thực hiện các yêu cầu truy vấn dữ liệu sau đây:
- Danh sách các đối tác cung cấp hàng cho công ty.
 - Mã hàng, tên hàng và số lượng của các mặt hàng hiện có trong công ty.
 - Họ tên, địa chỉ và năm bắt đầu làm việc của các nhân viên trong công ty.
 - Địa chỉ và điện thoại của nhà cung cấp có tên giao dịch VINAMILK.
 - Mã và tên mặt hàng có giá lớn hơn 100000 và số lượng hiện có ít hơn 50.
 - Danh sách mặt hàng do Công ty Việt Tiến cung cấp.
 - Danh sách khách hàng đã đặt mua mặt hàng Sữa hộp XYZ của công ty.
 - Thông tin chi tiết đơn đặt hàng số 1.
- c. Sử dụng câu lệnh UPDATE để thực hiện các yêu cầu sau:
- Cập nhật lại giá trị trường NGAYCHUYENHANG của những bản ghi có NGAYCHUYENHANG chưa xác định (NULL) trong bảng DONDATHANG bằng với giá trị của trường NGAYDATHANG.
 - Tăng số lượng hàng của những mặt hàng do công ty VINAMILK cung cấp lên gấp đôi.
 - Cập nhật giá trị của trường NOIGIAOHANG trong bảng DONDATHANG bằng địa chỉ của khách hàng đối với những đơn đặt hàng chưa xác định được nơi giao hàng (giá trị trường NOIGIAOHANG bằng NULL).
- d. Thực hiện các yêu cầu dưới đây bằng câu lệnh DELETE.
- Xoá khỏi bảng NHANVIEN những nhân viên đã làm việc trong công ty quá 40 năm.
 - Xoá những đơn đặt hàng trước năm 2000 ra khỏi cơ sở dữ liệu.
 - Xoá khỏi bảng LOAIHANG những loại hàng hiện không có mặt hàng.
 - Xoá khỏi bảng KHACHHANG những khách hàng hiện không có bất kỳ đơn đặt hàng nào cho công ty.

Chương 3. LẬP TRÌNH PHP

3.1. Tổng quan

3.1.1. Cấu trúc của PHP

PHP (Personal Home Page hoặc Hypertext Preprocessor) là ngôn ngữ tiền xử lý siêu văn bản. PHP là một ngôn ngữ lập trình kiểu script, chạy trên Server và trả về mã HTML cho trình duyệt. PHP trở thành một ngôn ngữ lập trình web rất phổ biến hiện nay. Các mã lệnh PHP được nhúng vào các trang web, các trang này thường có phần mở rộng là .php, .php3, .php4. Khi client gửi yêu cầu "cần tải các trang này về" đến web server, đầu tiên web server sẽ phân tích và thi hành các mã lệnh PHP được nhúng trong tài liệu, sau đó trả về một trang web kết quả đã được xử lý cho client.

PHP được nhân dạng dưới 4 dạng sau :

1. `<?php print "hello there!"; ?>`
2. `<script language="php"> print "hello there!"; </script>`
3. `<? print "hello there!"; ?>`
4. `<% print "hello there!"; %>`

Chú ý: Các dòng mã của PHP đều được kết thúc bằng dấu chấm phẩy ";". Dấu chấm phẩy cho biết khi thực hiện câu lệnh PHP nếu một câu lệnh kết thúc nó có thể tiến hành câu lệnh tiếp theo. Ngoặc đóng ">" PHP sau khi câu lệnh kết thúc cũng có tác dụng tương tự như ";".

PHP cho phép đặt các dòng chú thích theo các cách như sau:

1. `//` dòng chú thích.
2. `#` dòng chú thích.
3. `/*` đoạn chú thích `*/`

Xuất giá trị ra trình duyệt: Để xuất dữ liệu ra trình duyệt chúng ta có những dòng cú pháp sau:

```
echo "Thông tin";
```

Hoặc

```
Printf "Thông tin";
```

3.1.2. Hằng, biến

3.1.2.1. Hằng

Hằng là một giá trị không thể chỉnh sửa được thông qua việc thực hiện chương trình. Có thể định nghĩa một hằng bằng cách dùng hàm `define()`.

```
define ("tên_hằng","giá trị của hằng");
```

Ví dụ:

1. `define ("COMPANY","Sao Do university");//Hằng COMPANY có giá trị "Sao Do University"`
2. `define ("diem_so",4.5);//định nghĩa hằng diem_so với giá trị 4.5`

Sau khi một hằng được tạo ra, có thể sử dụng thay cho giá trị của chúng:

```
echo ("Tên công ty: ".COMPANY);
```

Điều này tương đương với:

```
echo ("Tên công ty: Sao Do university");
```

Một khi hằng được định nghĩa, giá trị của hằng không bao giờ bị thay đổi. Chỉ có các kiểu dữ liệu *boolean*, *integer*, *float*, *string* mới có thể chứa các hằng. Có thể nhận giá trị của một hằng bằng cách chỉ định tên của hằng. Không giống như biến, không cần khai báo ký tự \$ trước tên hằng. Có thể dùng hàm *constant()* để đọc giá trị của hằng, sử dụng hàm *get_defined_constants()* để nhận một danh sách tất cả các hằng đã được định nghĩa.

Các hằng được định nghĩa trước: PHP cung cấp một lượng lớn các hằng đã được định nghĩa trước để sử dụng.

`_FILE_`: Tên của script file đang được thực hiện.

`_LINE_`: Số dòng của mã script đang được thực hiện trong script file hiện tại.

`_PHP_VERSION_`: Version của PHP.

`_PHP_OS`: Tên hệ điều hành mà PHP đang chạy.

`E_ERROR`: Báo hiệu có lỗi.

`E_PARSE`: Báo lỗi sai khi biên dịch.

`E_NOTICE`: Một vài sự kiện có thể là lỗi hoặc không.

3.1.2.2. Biến

a. Khai báo

Biến trong PHP được thể hiện bởi dấu dollar "\$" và theo sau là tên của biến. Tên biến có *phân biệt chữ hoa, chữ thường*. Tên biến phải bắt đầu bằng một chữ cái hay một dấu gạch nối _, theo sau là các chữ cái, chữ số hay là dấu gạch nối (Không sử dụng Tiếng Việt).

Ví dụ:

```
1. $var = "Bob";  
2. $Var = "Joe";  
3. echo "$var, $Var";
```

Ngoài cách gán giá trị cho biến khi khai báo. PHP còn cung cấp thêm một cách khác để gán giá trị cho biến - gán theo tham chiếu. Điều này có nghĩa là một biến mới có thể tham chiếu đến một biến nguyên thủy. Sự thay đổi của biến mới sẽ tác động đến biến nguyên thủy và ngược lại. Nó cũng có nghĩa là không có sao chép, do đó việc kết gán sẽ diễn ra nhanh hơn. Để thực hiện gán bằng tham chiếu, chỉ cần thêm & ở đầu tên của biến được gán.

Ví dụ, in ra dòng 'My name is Bob' hai lần:

```
1. <?php
2. $foo = 'Bob';           //gán giá trị 'Bob' cho $foo
3. $bar = &$foo;           //Tham chiếu $foo qua $bar.
4. $bar = "My name is $bar"; //thay đổi $bar...
5. echo $bar;
6. echo $foo;             // $foo cũng bị thay đổi.
7. ?>
```

b. Phạm vi sử dụng của biến

Các phạm vi sử dụng của biến trong PHP gồm:

- Biến cục bộ.
- Biến là tham số các hàm.
- Biến toàn cục.
- Biến static.

** Biến cục bộ*

Một biến được khai báo trong một hàm thì được xem như là một biến cục bộ và nó chỉ được tham chiếu trong hàm đó. Bất kể việc gán giá trị bên ngoài hàm đều xem như là biến hoàn toàn khác với biến trong hàm đó. Khi thoát khỏi hàm thì biến cục bộ và giá trị của nó sẽ bị huỷ bỏ. Biến cục bộ có thuận lợi bởi nó loại bỏ những khả năng của các tác động không dự đoán được làm thay đổi kết quả từ các biến có thể truy cập toàn cục.

Ví dụ:

```
1. $x = 4;
2. function assignx() {
3.     $x = 0;
4.     print "\$x inside function is $x. <br>";
5. }
6. assignx();
7. print "\$x outside of function is $x. <br>";
```

Sau khi thực hiện sẽ có các kết quả như sau:

\$x inside function is 0.

\$x outside of function is 4.

** Biến là tham số của hàm*

Cũng giống như nhiều ngôn ngữ lập trình khác, hàm trong PHP có chứa tham số phải khai báo các tham số trong phần đầu của hàm. Mặc dù, những tham số này nhận các giá trị đến từ bên ngoài hàm, nhưng chúng sẽ chỉ có thể truy cập một khi hàm tồn tại. Các tham số của hàm được khai báo sau tên hàm và bên trong dấu "()".

Ví dụ:

```
1. function x10 ($value) {  
2.     $value = $value * 10;  
3.     return $value;  
4. }
```

Chú ý: Mặc dù có thể truy cập và thao tác với các tham số trong hàm mà nó được khai báo, nhưng các tham số sẽ bị huỷ bỏ khi hàm kết thúc thực hiện.

* *Biến toàn cục*

Ngược lại với biến cục bộ, biến toàn cục có thể được truy cập từ bất cứ phần nào trong chương trình. Tuy nhiên, để có thể chỉnh sửa, biến toàn cục phải được khai báo toàn cục trong hàm mà nó được chỉnh sửa. Để khai báo một biến là toàn cục trong một hàm nào đó ta đặt từ khóa GLOBAL ở phía trước.

Ví dụ:

```
1. $somevar = 15;  
2. function addit() {  
3.     GLOBAL $somevar;  
4.     $somevar ++;  
5.     print "Somevar is $somevar";  
6. }  
7. addit();
```

Kết quả của *\$somevar* là 16. Tuy nhiên, nếu bỏ dòng lệnh *GLOBAL \$somevar*; thì biến *\$somevar* sẽ chỉ có giá trị là 1, bởi biến *\$somevar* được xem như là biến cục bộ trong hàm *addit()*.

Một cách khác để khai báo một biến là toàn cục là sử dụng mảng *\$GLOBALS* của PHP.

Ví dụ, khai báo biến *\$somevar* là toàn cục bằng cách sử dụng mảng *\$GLOBALS*.

```
1. $somevar = 15;  
2. function addit() {  
3.     $GLOBALS["somevar"];  
4.     $somevar++;  
5. }  
6. addit();  
7. print "Somevar is $somevar";
```

* *Biến static*

Tương phản với các biến được khai báo trong các tham số của hàm sẽ bị huỷ bỏ khi thoát khỏi hàm, biến static sẽ không mất giá trị của nó khi thoát khỏi hàm và sẽ giữ nguyên giá trị đó khi hàm được gọi lại lần nữa. Có thể khai báo một biến là static bằng cách dùng từ khóa STATIC đặt trước tên biến.

Ví dụ:

```

1. function keep_track() {
2.     STATIC $count = 0;
3.     $count++;
4.     print $count . "<br>";
5. }
6. keep_track();
7. keep_track();
8. keep_track();

```

Kết quả:

```

1
2
3

```

3.1.3. Các toán tử và biểu thức

Toán tử là một ký hiệu chỉ định một hoạt động trong biểu thức. Các toán tử có trong PHP như sau:

Bảng 3.1. Toán tử trong PHP

Toán tử	Mục đích
()	Thực hiện theo yêu cầu có thứ tự.
New	Minh họa một object mới.
! ~	Phép Not trong Boolean, Not trong Bitwise.
++ --	Tự động tăng, tự động giảm
@	Che dấu lỗi.
/ * %	Chia, nhân, chia lấy dư.
+ - .	Cộng, trừ, nối.
<< >>	Dịch sang trái, dịch sang phải trong Bitwise.
< <= > >=	So sánh nhỏ hơn, nhỏ hơn hay bằng, lớn hơn, lớn hơn hay bằng.
== != === <>	So sánh bằng, không bằng, giống nhau, khác nhau.
& ^	Phép AND, OR, XOR trong Bitwise.
&&	Phép AND, OR trong Boolean.
?:	Toán tử tam phân.
= += *= /= .= %= &= = ^=	Các toán tử gán.
AND XOR OR	Phép AND, XOR, OR trong Boolean.

3.1.4. Truyền biến giữa các trang web

Có bốn cách cơ bản để truyền biến giữa các trang web: Truyền biến trong URL, qua session, cookie hoặc với một HTML form.

Với mỗi cách truyền biến, chúng ta có thể truy cập đến chúng theo một cách tương ứng. Cú pháp và cách sử dụng như sau:

- \$_GET[tên biến]: Khi phương thức chuyển biến là “GET”, phương thức trong HTML forms.

- \$_POST[tên biến]: Khi phương thức chuyển biến là “POST”, phương thức trong HTML forms.

- \$_SESSION[tên biến]: Khi biến đã được ấn định giá trị từ một session đặc biệt.

- \$_COOKIE[tên biến]: Khi biến đã được ấn định giá trị từ một cookie.

- \$_REQUEST[tên biến]: Khi nó không quan trọng (\$_REQUEST bao gồm các biến chuyển từ bất cứ cách nào ở trên).

- \$_SERVER[tên biến]: Khi biến đã được ấn định giá trị từ một server.

- \$_FILES[tên biến]: Khi biến đã được ấn định giá trị từ một file upload.

- \$_ENV[tên biến]: Khi biến đã được ấn định giá trị từ một môi trường hệ điều hành.

Nếu không gọi ra các biến sử dụng cú pháp này thì giá trị của nó sẽ là rỗng, đây có thể là nguyên nhân làm cho chương trình dịch bị thất bại.

3.1.4.1. Truyền biến qua URL

Phương thức đầu tiên để truyền biến giữa các trang là thông qua một URL. Ví dụ một URL: <http://www.mydomain.com/news/articles/showart.php?id=12345>. Những phần sau dấu “?” gọi là chuỗi truy vấn (query string). Có thể nối giữa các biến trong URL bằng cách dùng dấu “&”.

Ví dụ:

<http://www.mydomain.com/news/articles/showart.php?id=12345&lang=en>

Nhược điểm:

- Người dùng có thể nhìn thấy giá trị của biến, những thông tin nhạy cảm thì không được bảo mật khi sử dụng cách thức này.

- Người dùng có thể đổi giá trị biến trong URL, mở những phần mà người lập trình không muốn hiển thị.

- Người sử dụng có thể gặp phải những thông tin không chính xác hoặc đã cũ khi dùng URL đã được lưu với những biến cũ trong đó.

Khi truyền biến qua URL, trong trường hợp biến đó chứa khoảng trắng, dấu & hay một ký tự đặc biệt nào đó, thì ta dùng hàm `urlencode()`.

3.1.4.2. Truyền biến qua form

Form là mã trong HTML được sử dụng để nhập dữ liệu cần truyền đi thông qua

các formfield (còn gọi là Input). Form được bao bởi một cặp thẻ mở và đóng: <form> </form>. Form có những thuộc tính sau:

- Action: Thuộc tính này chỉ ra đường dẫn URL của trang mà Form sẽ được gửi đến để xử lý, nó chứa đường dẫn URL tương đối hoặc URL tuyệt đối.

- Method: Thuộc tính này chỉ ra kiểu HTTP yêu cầu trình duyệt gửi thông tin đến server, nó phải được thiết lập là POST hoặc GET.

- Name: Đây là thuộc tính hữu hiệu nhất trong việc quản lý các thành phần của form. Tên form không được gửi đến server khi form được gửi đi.

Việc thực hiện truyền dữ liệu thông qua nhấn nút Submit. Trang web nhận dữ liệu form bằng cách sử dụng các biến toàn cục của PHP sau:

- \$_POST["FieldName"] (tương ứng với Method là POST)
- \$_GET["FieldName"] (tương ứng với Method là GET)
- \$_REQUEST["FieldName"] (Áp dụng với Method là POST hoặc GET)

Ví dụ: Trang tìm sách (timSach.php)

```
1. <html>
2. <body>
3. <h1>Tìm sách</h1>
4. <form action="xltimSach.php" Method="GET">
5.     Từ khóa: <input type="text" name="txtTukhoa"/>
6.     <input type="submit" value="Tìm"/>
7. </form>
8. </body>
9. </html>
```



Hình 3.1. Trang timsach.php

Trang xử lý tìm sách (xltimSach.php)

```
1. <html>
2. <body>
3. <?php $sTukhoa = $_REQUEST["txtTukhoa"]; ?>
4. <h1>Tìm sách</h1>
5. Từ khóa tìm sách là: <?php echo $sTukhoa;?> <br/>
```


6. Kết quả tìm là:

7. `</body>`

8. `</html>`

Nếu trong trang `timsach.php` ta sử dụng `Method="POST"` thì trong trang `xltimsach.php` ta phải dùng biến toàn cục `$_POST["txtTukhoa"]` để nhận dữ liệu.

Đối với cách truyền dữ liệu sử dụng Session và Cookies được trình bày trong các phần sau.

3.1.5. Sử dụng chỉ thị `include`

PHP cho phép tạo các phần tử và hàm khác nhau, được sử dụng nhiều lần trong nhiều trang tránh mất nhiều thời gian để viết các hàm này trong nhiều trang. Do đó, hãy sử dụng `include` hoặc `require` giúp nhúng file trong các chương trình khác nhau và tiết kiệm công sức viết code nhiều lần. Có hai cách để nhúng file vào trang PHP.

- Include (bao gồm).

- Require (yêu cầu).

Cả `include` và `require` đều giống nhau, ngoại trừ lỗi. `Include` chỉ tạo ra một cảnh báo, tức là `E_WARNING` và tiếp tục thực thi tập lệnh. `Require` tạo ra một lỗi nghiêm trọng, tức là `E_COMPILE_ERROR` và dừng việc thực thi tập lệnh.

Cú pháp:

```
include 'filename';
include ('filename');
require 'filename';
require ('filename');
```

Nên sử dụng `require` khi ứng dụng yêu cầu phải nhập vào một tập tin mang tính chất bắt buộc. Nếu tập tin này không tồn tại PHP sẽ chấm dứt hoạt động của chương trình đang chạy và hiển thị lỗi `E_COMPILE_ERROR`.

Ví dụ sau sẽ nhập vào tập tin `database_config.php`, file này chứa thông tin như `database username`, `password` hay địa chỉ `host` để kết nối tới database:

```
1. <?php
2. require('database_config.php');
3. ?>
```

Nếu file `database_config.php` không tồn tại thì PHP sẽ hiển thị lỗi và chấm dứt chương trình.

* `include_once` và `require_once`

Chúng ta sử dụng `include_once` hoặc `require_once` để nhúng nội dung file vào một lần đầu tiên, nếu file này tiếp tục được nhập vào từ các lệnh `require`, `include` hay `require_once`, `include_once` sau đó, PHP sẽ bỏ qua việc nhập vào.

Sử dụng `require_once` khi file nhập vào là bắt buộc phải có và tập tin này chỉ nên được nhập vào một lần đầu tiên. Tương tự như phân tích về sự khác nhau giữa

include và require ở phần trên, require_once thường được dùng nếu file nhưng là bắt buộc, nếu không chương trình sẽ dừng và báo lỗi.

Ví dụ sau sẽ nhập file connect_db.php chứa logic thực hiện việc kết nối tới database. Do đó trên mỗi trang có kết nối tới database, chúng ta chỉ nên thực hiện việc nhập vào file này một lần đầu duy nhất.

```
1. <?php
2. require_once('connect_db.php');
3. ?>
```

3.2. Các kiểu dữ liệu cơ bản

3.2.1. Kiểu số nguyên

Là một tập hợp bao gồm các số nguyên {..., -2, -1, 0, 1, 2,...}.

Cú pháp: Integer có thể được chỉ định trong cơ số 10, cơ số thập lục phân hay cơ số bát phân, tùy chọn đi trước bởi dấu - hay +. Nếu sử dụng với cơ số bát phân thì phải theo thứ tự với 0 đứng trước, còn đối với số thập lục phân thì 0x.

Ví dụ:

```
1. $a = 1234; # số thập phân
2. $a = -123; # số âm
3. $a = 0123; # số bát phân
4. $a = 0x1A; # số thập lục phân
```

Kích thước của kiểu dữ liệu số nguyên là 32bit, và PHP không hỗ trợ kiểu *unsigned integer*. Nếu chỉ định một số vượt qua giới hạn của kiểu dữ liệu *integer* thì sẽ được xem như kiểu *float*. Tương tự như vậy, khi thực hiện một phép toán mà kết quả trả về là một số vượt qua biên của kiểu *integer* thì kiểu *float* sẽ được trả về. Tuy nhiên, có một lỗi trong PHP mà không phải bao giờ điều này cũng đúng, nó liên quan đến các số âm. Chẳng hạn, khi thực hiện phép tính $-50000 * \$million$, kết quả sẽ là 429496728. Tuy nhiên, khi cả hai toán tử đều là số dương thì không có vấn đề gì xảy ra.

Để chuyển một giá trị sang kiểu *integer*, có thể dùng toán tử ép kiểu (*int*) hay (*integer*). Tuy nhiên, trong hầu hết các trường hợp không cần phải dùng toán tử ép kiểu bởi giá trị sẽ được tự động chuyển sang nếu toán tử, hàm hay cấu trúc điều khiển đòi hỏi một đối số *integer*.

3.2.2. Kiểu số thực

Có thể được chỉ định bằng cách sử dụng một trong các cú pháp sau:

```
$a = 1.234;
$a = 1.2e3;
$a = 7E-10;
```

Kích cỡ của kiểu float tùy thuộc vào *platform*, giá trị lớn nhất là xấp xỉ 1.8e308

3.2.3. Kiểu logic

Kiểu logic (boolean) biểu thị một giá trị là đúng (TRUE) hoặc sai (FALSE).

Cú pháp: Để chỉ định một giá trị boolean, có thể sử dụng từ khóa TRUE hay FALSE. Cả hai đều không phân biệt chữ hoa hay chữ thường.

Ví dụ:

```
1. $foo=True; // gán giá trị TRUE cho biến $foo.
```

Để có thể chuyển một giá trị sang kiểu boolean, có thể dùng ép kiểu (bool) hay (boolean). Tuy nhiên trong hầu hết các trường hợp không cần phải sử dụng việc ép kiểu bởi giá trị sẽ được tự động chuyển nếu toán tử, hàm hay là cấu trúc điều khiển đòi hỏi một tham số kiểu boolean.

Chú ý: Giá trị 1 được xem là TRUE, giống như các giá trị khác 0 khác (bất kể là số dương hay âm). Giá trị 0 được xem là FALSE.

3.2.4. Kiểu chuỗi ký tự

Trong PHP, một ký tự cũng tương tự như một byte, do đó nó có chính xác 256 ký tự khác nhau. Cú pháp có thể khai báo bằng các cách sau:

- *Dấu nháy đơn:* Cách dễ dàng nhất để chỉ định một chuỗi đơn giản là đóng nó trong một dấu nháy đơn. Ví dụ: `echo 'My test';`

- *Dấu nháy kép:* Nếu chuỗi được đóng trong dấu nháy kép, PHP hiểu sẽ có thêm các chuỗi cho các ký tự đặc biệt,... Ví dụ: `\n;\t;\;\$;`...

- * Một số hàm xử lý chuỗi cơ bản:

- `strlen($chuỗi)`: Trả về độ dài của chuỗi.

- `substr($chuỗi, $vị trí, $chiều dài)`: Trả về một phần của chuỗi.

- `strtoupper($chuỗi)`: Chuyển ký tự chữ thường thành chữ hoa.

- `strtolower($chuỗi)`: Chuyển ký tự chữ hoa thành chữ thường.

- `trim($chuỗi, 'ký tự')`: Loại bỏ 'ký tự' ở đầu và cuối mỗi chuỗi.

- * Các hàm tìm kiếm chuỗi

- `strpos($chuỗi, $chuỗi_con, $vị trí bắt đầu)`: Trả về vị trí xuất hiện của một chuỗi trong một chuỗi khác. Hàm này chỉ trả về vị trí của chuỗi đầu tiên xuất hiện trong trường hợp có nhiều chuỗi giống nhau tồn tại.

- `stripos($chuỗi, $chuỗi_con, $vị trí bắt đầu)`: Hàm tìm kiếm chuỗi với vị trí bắt đầu tìm kiếm được xác định trước, có tính năng tương tự như `strpos()`.

- * Các hàm so sánh chuỗi

- `strcmp($chuỗi_1, $chuỗi_2)`: Sự so sánh chuỗi bằng cơ sở nhị phân.

- `strncmp($chuỗi_1, $chuỗi_2, $chiều dài)`: Trả về giá trị mà chuỗi 1 dài hơn so với chuỗi 2.

- `strcasecmp($chuỗi_1, $chuỗi_2)`: So sánh chuỗi theo thứ tự abc, các ký tự được xét từ đầu đến cuối chuỗi.

- `strncasecmp($chuỗi_1, $chuỗi_2, $chiều dài)`: Là một hàm giống như `strcasecmp()`, khác ở chỗ hàm này có thể chỉ rõ số ký tự trong mỗi chuỗi đem so sánh.

3.3. Các kiểu dữ liệu phức tạp

3.3.1. Kiểu mảng

3.3.1.1. Khái niệm

Mảng dùng để lưu trữ bộ các giá trị có cùng kiểu dữ liệu. Các giá trị lưu trữ trong một mảng gọi là các phần tử. Mỗi phần tử mảng có một chỉ số kết hợp (index) hoặc khóa (key) được sử dụng để truy nhập phần tử mảng. Mảng trong PHP và phần lớn các ngôn ngữ lập trình khác có các chỉ số bắt đầu từ 0. PHP cung cấp mảng chỉ số, mảng kết hợp và mảng nhiều chiều. Mảng chỉ số được hiểu như mảng một chiều.

3.3.1.2. Mảng chỉ số

a. Khai báo mảng chỉ số

```
$tên_biến = array(danh_sách_các_giá_trị);
```

Ví dụ tạo mảng tên là products, bao gồm ba phần tử:

```
1. <?php
2. $products = array("Tires", "Oil", "Spark Plugs");
3. ?>
```

Có thể khai báo một mảng rỗng như sau:

```
$tên_biến = array();
```

Ngoài cách khai báo trên, ta cũng có thể khai báo bằng cách gán trực tiếp giá trị cho phần tử mảng:

```
$tên_biến[] = "Giá trị";
```

Chú ý: array() là một cấu trúc ngôn ngữ còn gọi là từ khóa, không phải là hàm. Tùy thuộc vào nội dung trong array, có thể không cần thao tác khởi tạo chúng. Chúng được tự động tạo khi sử dụng lần đầu.

Có thể copy một mảng vào mảng khác sử dụng toán tử “=”. Có thể sử dụng hàm range() để tự động tạo mảng. Ví dụ sau tạo một mảng numbers với các phần tử trong phạm vi từ 1 đến 10:

```
$numbers = range(1,10);
```

b. Truy nhập các phần tử mảng chỉ số

Để truy nhập phần tử mảng ta sử dụng tên biến mảng và key hay index. key hay index cho biết giá trị (thứ tự) lưu trữ nào mà chúng ta muốn truy nhập. Ví dụ:

```
echo $products[0]; //in giá trị phần tử thứ 0 của mảng products.
```

```
echo $products[1]; //in giá trị phần tử thứ 1 của mảng products.
```

Để thay đổi giá trị phần tử mảng, ta dùng lệnh gán. Ví dụ:

```
$products[0] = "Fuses"; //gán phần tử thứ 0 của mảng
                        //products giá trị Fuses.
```

Để thêm 1 phần tử vào mảng chúng ta cũng có thể sử dụng như sau:

```
$tên_biến[] = "Giá trị";
```

3.3.1.3. Mảng kết hợp

Mảng kết hợp còn có tên gọi là mảng băm. Là các mảng được tạo index bằng các chuỗi (được gọi là key) và gán giá trị (value) cho các key.

a. Khai báo mảng kết hợp

`$tên_biến = array(tên_khóa => "giá trị",...);`

Ví dụ:

```
1. <?php
2. $a= array (name => "Kenny", job => "Teacher", age=>"45", email => "webmaster@vietchuyen.com.vn";
3. ?>
```

b. Truy nhập các phần tử mảng kết hợp

Để truy cập giá trị của mảng kết hợp, chúng ta không thể sử dụng chỉ số vị trí phần tử mà phải chỉ rõ khóa của phần tử cần truy nhập.

Ví dụ: Để lấy giá trị của tuổi trong mảng trên ta sử dụng như sau:

```
1. echo $a[age] //Kết quả sẽ cho ra 45.
```

3.3.1.4. Mảng nhiều chiều

Các mảng không chỉ đơn giản là danh sách các key và value mà mỗi vị trí trong mảng có thể lưu giữ mảng khác. Bằng cách này ta có thể tạo mảng 2 chiều. Ví dụ sau mô tả các sản phẩm như là các mảng 2 chiều với mỗi dòng mô tả một sản phẩm riêng và mỗi cột mô tả một thuộc tính sản phẩm như sau.

Bảng 3.2. Minh họa bảng sản phẩm

Code	Description	Price
TIR	Tires	100
OIL	Oil	10
SPK	Spark Plugs	4

```
1. <?php
2. $sanpham = array( array("TIR","Tires",100), array("OIL","Oil",10), array("SPK","Spark Plugs",4));
3. ?>
```

Để truy nhập dữ liệu trong mảng hai chiều tương tự mảng một chiều, chỉ khác là mỗi phần tử của mảng có hai chỉ số - dòng và cột. Để hiển thị nội dung của mảng trên, chúng ta truy nhập mỗi phần tử theo thứ tự như sau:

```
1. <?php
2. echo $sanpham[0][0]. " | ". $sanpham[0][1]. " | ". $sanpham[0][2]. "<br>";
3. echo $sanpham[1][0]. " | ". $sanpham[1][1]. " | ". $sanpham[1][2]. "<br>";
4. echo $sanpham[2][0]. " | ". $sanpham[2][1]. " | ". $sanpham[2][2]. "<br>";
5. ?>
```

Kết quả:

TIR|Tires|100

OIL|Oil|10

SPK|Spark Plugs|4

Các hàm thường dùng trên mảng:

Bảng 3.3. Một số hàm thường dùng trên mảng

Hàm	Mô tả
array_column(array, column_key, index_key)	Trả về các giá trị từ một cột duy nhất trong mảng đầu vào.
array_combine(keys, values)	Tạo mảng bằng cách sử dụng các phần tử từ một mảng "keys" và một mảng "values".
array_count_values(array)	Đếm tất cả các giá trị của một mảng.
array_diff(array1, array2, array3,...)	So sánh các mảng và trả về sự khác biệt (chỉ so sánh các giá trị).
array_diff_assoc(array1,array2,array3,...)	So sánh các khóa và giá trị của hai mảng và trả về sự khác biệt.
array_fill(index, number, value)	Điền vào một mảng với các giá trị.
array_intersect(array1, array2, array3,...)	So sánh các giá trị của hai mảng và trả về kết quả khớp.
array_key_exists(key, array)	Kiểm tra xem khóa đã chỉ định có tồn tại trong mảng không.
array_keys(array, value, strict)	Trả về tất cả các khóa của một mảng.
array_merge(array1, array2, array3,...)	Hợp nhất một hoặc nhiều mảng thành một mảng.
array_pop(array)	Xóa phần tử cuối cùng của một mảng
array_push(array, value1, value2,...)	Chèn một hoặc nhiều phần tử vào cuối mảng.
array_sum(array)	Trả về tổng của các giá trị trong một mảng.
array_unique(array, sorttype)	Loại bỏ các giá trị trùng lặp khỏi một mảng.
array_values(array)	Trả về tất cả các giá trị của một mảng.
arsort(array, sorttype)	Sắp xếp một mảng kết hợp theo thứ tự giảm dần, theo giá trị.
asort(array, sorttype)	Sắp xếp một mảng kết hợp theo thứ tự tăng dần, theo giá trị.
count(array, mode)	Trả về số lượng phần tử trong một mảng.
in_array(search, array, type)	Kiểm tra nếu một giá trị được chỉ định tồn tại trong một mảng.
list(var1, var2,...)	Gán các biến như thể chúng là một mảng.
sort(array, sorttype)	Sắp xếp một mảng chỉ số theo thứ tự tăng dần.

3.3.2. Kiểu rỗng

Kiểu rỗng (Null) là một kiểu dữ liệu đặc biệt chỉ có thể có một giá trị: NULL. Biến có kiểu dữ liệu NULL là biến không có giá trị được gán cho nó.

Chú ý:

- Nếu một biến được tạo mà không có giá trị thì biến sẽ tự động được gán giá trị NULL.
- Các biến cũng có thể được làm trống bằng cách đặt giá trị thành NULL.

Ví dụ:

```
1. <?php
2. $x = "Hello world!";
3. $x = null;
4. var_dump($x);
5. ?>
```

3.3.3. Kiểu đối tượng

Lớp và đối tượng là hai khía cạnh chính của lập trình hướng đối tượng. Một lớp là khuôn mẫu cho các đối tượng và một đối tượng là thể hiện của một lớp. Khi các đối tượng riêng lẻ được tạo, chúng kế thừa tất cả các thuộc tính và hành vi từ lớp, nhưng mỗi đối tượng sẽ có các giá trị khác nhau cho các thuộc tính.

Giả sử chúng ta có một lớp tên là Xe hơi. Ô tô có thể có các thuộc tính như kiểu xe, màu sắc,... Chúng ta có thể định nghĩa các biến như \$model, \$color,... để giữ giá trị của các thuộc tính này. Khi các đối tượng riêng lẻ (Volvo, BMW, Toyota,...) được tạo, chúng kế thừa tất cả các thuộc tính và hành vi từ lớp, nhưng mỗi đối tượng sẽ có các giá trị khác nhau cho các thuộc tính.

Nếu tạo một hàm __construct(), PHP sẽ tự động gọi hàm này khi tạo một đối tượng từ một lớp.

Ví dụ:

```
1. <?php
2. class Car {
3.     public $color;
4.     public $model;
5.     public function __construct($color, $model) {
6.         $this->color = $color;
7.         $this->model = $model;
8.     }
9.     public function message() {
10.        return "My car is a " . $this->color . " " . $this->model . "!";
11.    }
12.}
13.
14.$myCar = new Car("black", "Volvo"); //Kiểu đối tượng
```



```

15.echo $myCar -> message();
16.echo "<br>";
17.$myCar = new Car("red", "Toyota");
18.echo $myCar -> message();
19. ?>

```

Kết quả:

My car is a black Volvo!

My car is a red Toyota!

3.4. Các câu lệnh điều khiển

3.4.1. Cấu trúc if ... else ... elseif

Câu lệnh if là một trong những đặc tính quan trọng nhất của mỗi ngôn ngữ lập trình. Nó cho phép thực thi chọn lựa các dòng mã lệnh chỉ khi thoả mãn các điều kiện cụ thể.

Cú pháp:

```

if (condition) {
    code to be executed if condition is true;
}

```

Ví dụ:

```

1. if ($country == "ca") echo ("Canada");
2. //Canada được in ra khi biến $country là ca

```

Nếu nhiều hơn một câu lệnh được thực hiện khi thoả mãn điều kiện thì sử dụng dấu {} để chỉ ra những dòng lệnh nào là nằm trong khối if:

```

1. if ($country=="ca"){
2.     echo("Canada");
3.     echo("Ottawa");
4. }
5. //Canada và Ottawa sẽ chỉ được in nếu biến $country là ca

```

Điều kiện được kiểm tra trong dấu () phải trả về giá trị *boolean*, hoặc là *true* hoặc là *false*. Cũng như bất kỳ điều kiện nào mà không được thoả mãn, zero hay là chuỗi rỗng (""), các giá trị không được định nghĩa thì tất cả đều trả về giá trị là *false*. Các điều kiện có thể được nối với nhau bằng các toán tử logic AND (&&), OR (||) và XOR.

Ví dụ:

```

1. if (((4 < 5) && (3 > 2)) xor (5 == 5)) echo ("This will print");

```

Các điều kiện phân nhánh:

Nếu điều kiện được kiểm tra mà trả về *false* thì PHP cho phép chỉ ra một khối lệnh khác cần được thực hiện bằng cách dùng từ khóa *else*. Mỗi khối mã lệnh thực thi điều kiện này được xem như là một phân nhánh và mỗi nhánh phải được định vị trong các dấu ngoặc nếu có nhiều hơn một dòng lệnh.

Cú pháp:

```
if (condition) {  
    code to be executed if condition is true;  
} else {  
    code to be executed if condition is false;  
}
```

Ví dụ:

```
1. if ($h < 0) {  
2.     echo ("Negative");  
3. } else {  
4.     echo ("Positive");  
5. }
```

PHP cũng cung cấp từ khóa *elseif* để kiểm tra các điều kiện lựa chọn nếu điều kiện trong câu lệnh *if* là không đúng. Một số câu lệnh *elseif* có thể được sử dụng với câu lệnh *if*. Nhánh *else* cuối cùng cho phép ta định vị đoạn mã được thực hiện nếu cả điều kiện sau *if* và *elseif* đều không đúng. Ví dụ:

```
1. if ($h < 0) {  
2.     echo ("Negative");  
3. } elseif ($h == 0) {  
4.     echo ("Zero");  
5. } else {  
6.     echo ("Positive");  
7. }
```

Có thể kiểm tra những điều kiện hoàn toàn khác nhau khi sử dụng *elseif*. Ví dụ:

```
1. if ($country == "ca") {  
2.     // do something ...  
3. } elseif ($position == "h") {  
4.     // do something else ...  
5. }
```

Chú ý: Có thể cả hai điều kiện trên đều là *true*, nhưng chương trình thực hiện tuần tự từng lệnh do đó chỉ có nhánh lệnh thứ nhất là được thực hiện.

Có thể sử dụng các câu lệnh *if* lồng nhau trong câu lệnh *if* khác.

Ví dụ:

```
1. if ($country == "ca") {  
2.     if ($position == "h") {  
3.         echo ("Human resources positions in Canada.");  
4.     } elseif ($position == "a") {  
5.         echo ("Accounting positions in Canada.");  
6.     }  
7. }
```

Các câu lệnh trên cũng tương tự như sau:

```
1. if ($country == "ca" && $position == "h") {  
2.     echo ("Human resources positions in Canada.");  
3. } elseif ($country == "ca" && $position == "a") {  
4.     echo ("Accounting positions in Canada.");  
5. }
```

3.4.2. Cấu trúc switch

Câu lệnh *switch* được sử dụng khi một biến riêng rẽ đang được kiểm tra so với các giá trị khác.

Cú pháp:

```
switch (n) {  
    case label1:  
        code to be executed if n=label1;  
        break;  
    case label2:  
        code to be executed if n=label2;  
        break;  
    ...  
    default:  
        code to be executed if n is different from all labels;  
}  
}
```

Ví dụ:

```
1. switch ($country) {  
2.     case "ca":  
3.         echo ("Canada");  
4.         break;  
5.     case "uk":  
6.         echo ("the United Kingdom");  
7.         break;  
8.     default:  
9.         echo ("the United States");  
10. }
```

Khi câu lệnh *switch* thực hiện kiểm tra giá trị của biến *\$country* và so sánh nó với mỗi một trong các giá trị trong các mệnh đề *case*. Khi một giá trị thích hợp được tìm thấy, các câu lệnh kết hợp với *case* được thực hiện cho đến khi gặp câu lệnh *break*. Nếu không tìm ra được giá trị thích hợp nào thì câu lệnh *default* sẽ được thực hiện.

Chú ý rằng lệnh *switch* trong PHP thì linh hoạt hơn nhiều so với hầu hết các ngôn ngữ khác. Không giống như ngôn ngữ C, Java và ngay cả JavaScript, các giá trị *case* cũng có thể là một trong các loại vô hướng, bao gồm tất cả các số, các chuỗi và ngay cả các biến.

Ví dụ:

```
1. $val = 6; $a = 5; $b = 6;
2. switch ($val) {
3.     case $a:
4.         echo ("five");
5.         break;
6.     case $b:
7.         echo ("six");
8.         break;
9.     default:
10.        echo ("$val");
11. }
```

Các mảng và các đối tượng là những loại dữ liệu mà không thể dùng làm nhãn của *case* trong *switch*.

3.4.3. Cấu trúc *for*, *foreach*

Cấu trúc của vòng lặp *for* còn gọi là vòng lặp xác định bởi ta có thể xác định chính xác số lần thực hiện của vòng lặp.

Cú pháp:

```
for (init counter; test counter; increment counter) {
    code to be executed for each iteration;
}
```

Ví dụ:

```
1. for ($i = 1; $i < 11; ++$i)
2. {
3.     echo ("$i <BR> \n"); //In từ 1 đến 10
4. }
```

Câu lệnh *for* chứa ba biểu thức bên trong dấu ngoặc đơn được phân tách với nhau bởi dấu chấm phẩy. Biểu thức thứ nhất là một câu lệnh gán để khởi tạo biến điều khiển vòng lặp. Câu lệnh này được thực thi chỉ một lần trước sự lặp lại lần đầu của vòng lặp. Biểu thức thứ hai là biểu thức *Boolean* mà được thực thi tại đầu mỗi lần lặp. Nếu giá trị trả về là *true* thì vòng lặp sẽ tiếp tục thực hiện. Nếu là *false* thì vòng lặp kết thúc. Biểu thức thứ ba là một câu lệnh mà thực thi tại giai đoạn cuối của mỗi lần lặp của vòng lặp, thường được dùng để tăng hay giảm các biến điều khiển vòng lặp.

Vòng lặp *foreach* dùng để lặp qua thực hiện một khối mã cho mỗi phần tử trong một mảng.

Cú pháp:

```
foreach ($array as $value) {
    code to be executed;
}
```

Ví dụ:

```
1. <?php
2. $colors = array("red", "green", "blue", "yellow");
3. foreach ($colors as $value) {
4.     echo "$value <br>";
5. }
6. ?>
```

3.4.4. Cấu trúc *while* và *do.. while*

Cấu trúc của vòng lặp *while* thực thi một khối mã miễn là điều kiện được chỉ định là đúng.

Cú pháp:

```
while (condition is true) {
    code to be executed;
}
```

Ví dụ:

```
1. <?php
2. $x = 1;
3. while($x <= 5) {
4.     echo "The number is: $x <br>";
5.     $x++;
6. }
7. ?>
```

Một vòng lặp *while* sẽ kiểm tra một biểu thức điều kiện *Boolean*. Nếu biểu thức là *false* thì đoạn mã bên trong dấu ngoặc móc sẽ được bỏ qua và kết thúc vòng lặp. Ngược lại, nếu có giá trị *true* thì đoạn mã bên trong dấu ngoặc móc sẽ được thực hiện và lặp lại cho đến khi biểu thức điều kiện là *false*. Chú ý rằng điều kiện chỉ được kiểm tra mỗi khi bắt đầu vòng lặp, bởi vậy ngay khi điều kiện thay đổi trong phần giữa của khối lệnh thì mã lệnh sẽ vẫn được thực thi cho đến hết. Để thoát khỏi vòng lặp vào thời điểm sớm hơn ta có thể sử dụng lệnh *break*.

Vòng lặp *do...while* thực hiện lặp qua một khối mã một lần, sau đó lặp lại vòng lặp miễn là điều kiện đã chỉ định là đúng.

Cú pháp:

```
do {
    code to be executed;
} while (condition is true);
```

Ví dụ:

```
1. <?php
2. $x = 1;
3. do {
```

```

4.   echo "The number is: $x <br>";
5.   $x++;
6. } while ($x <= 5);
7. ?>

```

Lưu ý: Trong vòng lặp *do...while*, điều kiện được kiểm tra sau khi thực hiện các câu lệnh trong vòng lặp. Điều này có nghĩa là vòng lặp *do...while* sẽ thực hiện các câu lệnh của nó ít nhất một lần ngay cả khi điều kiện sai.

Ví dụ:

```

1. <?php
2. $x = 6;
3. do {
4.   echo "The number is: $x <br>";
5.   $x++;
6. } while ($x <= 5);
7. ?>

```

Đoạn mã trên đặt biến $x = 6$, tiếp theo chạy vòng lặp và sau đó điều kiện mới được kiểm tra.

3.4.5. Cấu trúc try

Trong quá trình lập trình chúng ta thường sử dụng các câu lệnh, cấu trúc lệnh để xử lý các lỗi có thể xảy ra khi thực hiện. Nếu xử lý lỗi không chính xác, thì có thể dẫn tới nhiều kết quả không mong đợi. Tiến trình bắt các lỗi được tạo bởi chương trình và sau đó thực hiện các hành động thích hợp, khi đó chúng ta gọi là xử lý ngoại lệ.

Xử lý ngoại lệ (exception) được sử dụng để thay đổi quy trình thực thi mã bình thường nếu xảy ra tình trạng lỗi (ngoại lệ) cụ thể. Ngoại lệ là trường hợp được ném (throw) và bắt (catch) trong khi lỗi thường không thể thu hồi lại.

PHP có một Exception Model tương tự như trong các ngôn ngữ lập trình khác. Exception cung cấp các điều khiển tốt hơn thông qua xử lý lỗi. Để sử dụng Exception Model, chúng ta thường dùng cấu trúc *try*.

Cú pháp:

```

try {
    //Các lệnh cần thực hiện;
    throw ...; //Ném ra một ngoại lệ nếu cần.
}
catch (Exception $e) {
    //Các lệnh xử lý bắt ngoại lệ nếu xảy ra;
}
finally {
    //Các lệnh thực thi cuối cùng sau khi thực thi khối try, catch;
}

```

- *Try*: Là một khối `{}`. Các lệnh chúng ta mong muốn chương trình thực hiện sẽ đặt trong khối này. Nếu *exception* không xảy ra thì code sẽ tiếp tục như bình thường. Tuy nhiên nếu *exception* xảy ra thì một *exception* được ném ra (throw).

- *Catch*: Mỗi khối *catch* thu nhận một *exception* và tạo một đối tượng chứa thông tin *exception* đó. Khối *catch* là bắt buộc phải có sau khối *try*.

- *Finally*: Một khối *finally* có thể được sử dụng sau hoặc thay vì các khối *catch*. Mã trong khối *finally* sẽ luôn được thực thi sau khi khối *try* và *catch* thực thi, bất kể ngoại lệ có xuất hiện hay không. Khối *finally* không bắt buộc phải có.

Khi một *exception* xảy ra, các lệnh theo sau lệnh mà xuất hiện *exception* đó sẽ không được thực thi và PHP sẽ cố gắng tìm kiếm khối *catch* so khớp đầu tiên. Nếu một *exception* không được bắt thì một Fatal Error (lỗi nghiêm trọng) trong PHP sẽ được thông báo với dòng thông báo “Uncaught Exception...”

- Một *exception* có thể được ném, và bắt bên trong PHP. Code có thể được bao quanh trong một khối *try*.

- Mỗi khối *try* phải có ít nhất một khối *catch* tương ứng. Nhiều khối *catch* có thể được sử dụng để bắt các lớp *exception* khác nhau.

- Các *exception* có thể được ném (hoặc ném lại) bên trong một khối *catch*.

Ví dụ 1: Dưới đây là một đoạn code minh họa sử dụng cấu trúc *try*.

```
1. <?php
2. try {
3.     echo "try block<br />";
4.     throw new Exception("test");
5. } catch (Exception $ex) {
6.     echo "catch block<br />";
7. } finally {
8.     echo "finally block<br />";
9. }
10. ?>
```

Ví dụ 2:

```
1. <?php
2. try {
3.     $error = 'Luôn luôn ném lỗi này';
4.     throw new Exception($error);
5.     // Phần code theo sau một ngoại lệ sẽ không được thực thi.
6.     echo 'Phần code này không bao giờ được thực thi';
7. }
8. catch (Exception $e) {
9.     echo 'Bắt ngoại lệ: ', $e->getMessage(), "\n";
10. }
```

```
11.// Tiếp tục tiến trình thực thi
12.echo 'Hello World';
13. ?>
```

Trong ví dụ trên, hàm `$e->getMessage` được sử dụng để lấy error message. Dưới đây là một số hàm có thể được sử dụng từ lớp Exception trong PHP.

- `getMessage()`: Thông báo của ngoại lệ.
- `getCode()`: Hiển thị các mã số là đại diện của các loại ngoại lệ.
- `getFile()`: Hiển thị tên các tập tin và đường dẫn nơi xảy ra ngoại lệ.
- `getLine()`: Hiển thị các dòng xảy ra ngoại lệ.
- `getTrace()`: Hiển thị một loạt các dấu vết trước chương trình xảy ra ngoại lệ.
- `getPrevious()`: Hiển thị các ngoại lệ trước khi xảy ra trường hợp hiện tại.
- `getTraceAsString()`: Hiển thị một loạt các dấu vết của ngoại lệ như một chuỗi thay vì một mảng.
- `__toString()`: Hiển thị toàn bộ ngoại lệ như một chuỗi.

3.5. Làm việc với cơ sở dữ liệu MySQL

3.5.1. Kết nối với cơ sở dữ liệu

Trong PHP, khi lập trình tương tác với cơ sở dữ liệu, chúng ta thường sử dụng thông qua các hàm có sẵn.

Chú ý: Trước khi sử dụng PHP để lập trình với MySQL, hãy sử dụng chương trình quản lý `phpMyAdmin` để tạo trước một cơ sở dữ liệu, các bảng cũng như người dùng,... để thuận tiện thực hành.

PHP 5 trở lên có thể hoạt động với cơ sở dữ liệu MySQL bằng cách sử dụng:

- MySQLi extension (Tiện ích mở rộng MySQLi, chữ "i" là viết tắt của chữ cải tiến - improve).
- PDO (PHP Data Objects - Đối tượng dữ liệu PHP).

Tiện ích mở rộng MySQLi được cài đặt tự động trong hầu hết các phiên bản của PHP khi gói `mysql php5` được cài đặt.

* Sử dụng MySQLi hay PDO [2]

- PDO hoạt động trên 12 hệ thống cơ sở dữ liệu khác nhau, trong khi MySQLi chỉ hoạt động với cơ sở dữ liệu MySQL. Vì vậy, nếu phải chuyển dự án của mình sang sử dụng cơ sở dữ liệu khác thì PDO sẽ giúp quá trình này trở nên dễ dàng. Chúng ta chỉ phải thay đổi chuỗi kết nối và một vài truy vấn. Với MySQLi, chúng ta sẽ cần phải viết lại toàn bộ mã - bao gồm cả truy vấn.

- Cả hai đều hướng đối tượng, nhưng MySQLi còn cung cấp API thủ tục.
- Cả hai đều hỗ trợ các câu lệnh đã chuẩn bị. Các câu lệnh đã chuẩn bị bảo vệ khỏi SQL injection và rất quan trọng đối với bảo mật ứng dụng web.

* Mở kết nối, sử dụng MySQLi hướng đối tượng [3].

```
1. <?php
2. $servername = "localhost";
3. $username = "root";
4. $password = "";
5. $dbname = "myDB";
6. // Create connection
7. $conn = new mysqli($servername,$username,$password,$dbname);
8. // Check connection
9. if (mysqli_connect_error()) {
10.     die("Connection failed: " . mysqli_connect_error());
11.}
12.echo "Connected successfully";
13.?.>
```

* Mở kết nối, sử dụng MySQLi hướng thủ tục [3].

```
1. <?php
2. $servername = "localhost";
3. $username = "root";
4. $password = "";
5. $dbname = "myDB";
6. // Create connection
7. $conn = mysqli_connect($servername,$username,$password,$dbname);
8. // Check connection
9. if (!$conn) {
10.     die("Connection failed: " . mysqli_connect_error());
11.}
12.echo "Connected successfully";
13.?.>
```

* Mở kết nối, sử dụng PDO [3].

```
1. <?php
2. $servername = "localhost";
3. $username = "root";
4. $password = "";
5. $dbname = "myDB";
6.
7. try {
8.     $conn = new
        PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
9.     // set the PDO error mode to exception
10.     $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

```

11. echo "Connected successfully";
12.} catch(PDOException $e) {
13. echo "Connection failed: " . $e->getMessage();
14.}
15.?>

```

Lưu ý: Trong ví dụ kết nối sử dụng PDO ở trên, chúng ta cũng đã chỉ định một cơ sở dữ liệu (myDB). PDO yêu cầu cơ sở dữ liệu hợp lệ để kết nối. Nếu không có cơ sở dữ liệu nào được chỉ định thì một ngoại lệ sẽ được đưa ra.

PDO có một lớp ngoại lệ để xử lý mọi vấn đề có thể xảy ra trong các truy vấn cơ sở dữ liệu của chúng ta. Nếu một ngoại lệ được đưa ra trong khối try{} thì tập lệnh sẽ ngừng thực thi và chuyển trực tiếp đến khối catch(){} đầu tiên.

3.5.2. Thao tác với cơ sở dữ liệu

3.5.2.1. Chèn dữ liệu vào MySQL

Sau khi cơ sở dữ liệu và bảng đã được tạo, chúng ta có thể bắt đầu thêm dữ liệu vào chúng. Dưới đây là một số quy tắc cú pháp để làm theo:

- Truy vấn SQL phải được trích dẫn bằng PHP.
- Các giá trị chuỗi bên trong truy vấn SQL phải được trích dẫn (đặt trong dấu nháy đơn).
- Các giá trị số không được trích dẫn.
- Từ NULL không được trích dẫn.

Câu lệnh INSERT INTO được sử dụng để thêm các bản ghi mới vào bảng MySQL như sau:

```

INSERT INTO table_name (column1, column2, column3,...)
VALUES (value1, value2, value3,...)

```

Giả sử, chúng ta đã tạo một bảng trống có tên "MyGuests" với các cột: "id", "firstname", "lastname", "email" và "reg_date".

```

CREATE TABLE MyGuests (
    id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    firstname VARCHAR(30) NOT NULL,
    lastname VARCHAR(30) NOT NULL,
    email VARCHAR(50),
    reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
    CURRENT_TIMESTAMP
)

```

Bây giờ, hãy điền dữ liệu vào bảng này.

Lưu ý: Nếu một cột là AUTO_INCREMENT (như cột "id") hoặc TIMESTAMP có cập nhật mặc định là current_timestamp (như cột "reg_date"), thì không cần chỉ định trong truy vấn SQL, MySQL sẽ tự động thêm giá trị.

* Sử dụng MySQLi hướng đối tượng

```
1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";
6. // Create connection
7. $conn = new mysqli($servername, $username, $password, $dbname);
8. // Check connection
9. if ($conn->connect_error) {
10.     die("Connection failed: " . $conn->connect_error);
11.}
12.$sql = "INSERT INTO MyGuests (firstname, lastname, email)
13.VALUES ('John', 'Doe', 'john@example.com')";
14.
15.if ($conn->query($sql) === TRUE) {
16.    echo "New record created successfully";
17.} else {
18.    echo "Error: " . $sql . "<br>" . $conn->error;
19.}
20.
21.$conn->close();
22.??>
```

* Sử dụng MySQLi hướng thủ tục

```
1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";
6. // Create connection
7. $conn = mysqli_connect($servername,$username,$password,$dbname);
8. // Check connection
9. if (!$conn) {
10.     die("Connection failed: " . mysqli_connect_error());
11.}
12.$sql = "INSERT INTO MyGuests (firstname, lastname, email)
13.VALUES ('John', 'Doe', 'john@example.com')";
14.if (mysqli_query($conn, $sql)) {
15.    echo "New record created successfully";
16.} else {
```

```

17. echo "Error: " . $sql . "<br>" . mysqli_error($conn);
18.}
19.
20. mysqli_close($conn);
21. ?>

```

* Sử dụng PDO

```

1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDBPDO";
6. try {
7.     $conn = new
        PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
8.     // set the PDO error mode to exception
9.     $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
10.    $sql = "INSERT INTO MyGuests (firstname, lastname, email)
11.    VALUES ('John', 'Doe', 'john@example.com')";
12.    // use exec() because no results are returned
13.    $conn->exec($sql);
14.    echo "New record created successfully";
15.} catch(PDOException $e) {
16.    echo $sql . "<br>" . $e->getMessage();
17.}
18.
19.$conn = null;
20. ?>

```

3.5.2.2. Chọn dữ liệu từ cơ sở dữ liệu MySQL

Câu lệnh SELECT được sử dụng để chọn dữ liệu từ một hoặc nhiều bảng:

SELECT column_name(s) FROM table_name

Hoặc chúng ta có thể sử dụng ký tự * để chọn tất cả các cột từ một bảng:

SELECT * FROM table_name

* Sử dụng MySQLi hướng đối tượng

Ví dụ sau đây chọn các cột id, firstname và lastname từ bảng MyGuests và hiển thị trên trang:

```

1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";

```

```

6.
7. // Create connection
8. $conn = new mysqli($servername, $username, $password, $dbname);
9. // Check connection
10. if ($conn->connect_error) {
11.     die("Connection failed: " . $conn->connect_error);
12. }
13.
14. $sql = "SELECT id, firstname, lastname FROM MyGuests";
15. $result = $conn->query($sql);
16.
17. if ($result->num_rows > 0) {
18.     // output data of each row
19.     while($row = $result->fetch_assoc()) {
20.         echo "id: " . $row["id"]. " - Name: " . $row["firstname"]. " " .
            $row["lastname"]. "<br>";
21.     }
22. } else {
23.     echo "0 results";
24. }
25. $conn->close();
26. ?>

```

Đầu tiên, ta thiết lập một truy vấn SQL để chọn các cột id, firstname và lastname từ bảng MyGuests. Dòng mã tiếp theo chạy truy vấn và đặt dữ liệu kết quả vào một biến có tên \$result.

Sau đó, hàm num_rows() kiểm tra xem có nhiều hơn 0 hàng được trả về hay không. Nếu có nhiều hơn 0 hàng được trả về, thì hàm fetch_assoc() sẽ đặt tất cả kết quả vào một mảng kết hợp. Vòng lặp while() lặp qua tập kết quả và xuất dữ liệu từ các cột id, firstname và lastname.

* Sử dụng MySQLi hướng thủ tục

```

1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";
6.
7. // Create connection
8. $conn = mysqli_connect($servername, $username, $password, $dbname);
9. // Check connection
10. if (!$conn) {

```

```

11. die("Connection failed: " . mysqli_connect_error());
12.}
13.
14.$sql = "SELECT id, firstname, lastname FROM MyGuests";
15.$result = mysqli_query($conn, $sql);
16.
17.if (mysqli_num_rows($result) > 0) {
18. // output data of each row
19. while($row = mysqli_fetch_assoc($result)) {
20.     echo "id: " . $row["id"]. " - Name: " . $row["firstname"]. " " .
        $row["lastname"]. "<br>";
21. }
22.} else {
23. echo "0 results";
24.}
25.
26.mysql_close($conn);
27.}>

```

* Sử dụng PDO

```

1. <?php
2. echo "<table style='border: solid 1px black;'>";
3. echo "<tr><th>Id</th><th>Firstname</th><th>Lastname</th></tr>";
4.
5. class TableRows extends RecursiveIteratorIterator {
6.     function __construct($it) {
7.         parent::__construct($it, self::LEAVES_ONLY);
8.     }
9.
10.    function current() {
11.        return "<td style='width:150px;border:1px solid black;'>" .
            parent::current(). "</td>";
12.    }
13.
14.    function beginChildren() {
15.        echo "<tr>";
16.    }
17.
18.    function endChildren() {
19.        echo "</tr>" . "\n";
20.    }
21.}

```

```

22.
23.$servername = "localhost";
24.$username = "username";
25.$password = "password";
26.$dbname = "myDBPDO";
27.
28.try {
29.  $conn = new
        PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
30.  $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
31.  $stmt = $conn->prepare("SELECT id, firstname, lastname FROM MyGuests");
32.  $stmt->execute();
33.
34.  // set the resulting array to associative
35.  $result = $stmt->setFetchMode(PDO::FETCH_ASSOC);
36.  foreach(new TableRows(new
        RecursiveArrayIterator($stmt->fetchAll())) as $k=>$v) {
37.      echo $v;
38.  }
39.} catch(PDOException $e) {
40.  echo "Error: " . $e->getMessage();
41.}
42.$conn = null;
43.echo "</table>";
44.??>

```

Ví dụ trên sử dụng câu lệnh chọn các cột id, firstname và lastname từ bảng MyGuests và hiển thị nó trong bảng HTML.

3.5.2.3. Cập nhật dữ liệu trong cơ sở dữ liệu MySQL

Câu lệnh UPDATE được sử dụng để cập nhật các bản ghi hiện có trong một bảng như sau:

```

UPDATE table_name
    SET column1=value, column2=value2,...
    WHERE some_column=some_value

```

Lưu ý: Mệnh đề WHERE trong cú pháp UPDATE chỉ định bản ghi hoặc các bản ghi sẽ được cập nhật. Nếu bỏ qua mệnh đề WHERE thì tất cả các bản ghi sẽ được cập nhật.

Các ví dụ sau cập nhật bản ghi với id = 2 trong bảng "MyGuests".

* Sử dụng MySQLi hướng đối tượng

```

1. <?php
2. $servername = "localhost";

```

```

3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";
6. // Create connection
7. $conn = new mysqli($servername, $username, $password, $dbname);
8. // Check connection
9. if ($conn->connect_error) {
10.  die("Connection failed: " . $conn->connect_error);
11.}
12.$sql = "UPDATE MyGuests SET lastname='Doe' WHERE id=2";
13.
14.if ($conn->query($sql) === TRUE) {
15.  echo "Record updated successfully";
16.} else {
17.  echo "Error updating record: " . $conn->error;
18.}
19.$conn->close();
20.??>

```

* Sử dụng MySQLi hướng thủ tục

```

1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";
6. // Create connection
7. $conn = mysqli_connect($servername, $username, $password, $dbname);
8. // Check connection
9. if (!$conn) {
10.  die("Connection failed: " . mysqli_connect_error());
11.}
12.
13.$sql = "UPDATE MyGuests SET lastname='Doe' WHERE id=2";
14.
15.if (mysqli_query($conn, $sql)) {
16.  echo "Record updated successfully";
17.} else {
18.  echo "Error updating record: " . mysqli_error($conn);
19.}
20.
21mysqli_close($conn);
22.??>

```


* Sử dụng PDO

```
1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDBPDO";
6.
7. try {
8.     $conn = new
        PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
9.     // set the PDO error mode to exception
10.    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
11.
12.    $sql = "UPDATE MyGuests SET lastname='Doe' WHERE id=2";
13.
14.    // Prepare statement
15.    $stmt = $conn->prepare($sql);
16.
17.    // execute the query
18.    $stmt->execute();
19.
20.    // echo a message to say the UPDATE succeeded
21.    echo $stmt->rowCount() . " records UPDATED successfully";
22.} catch(PDOException $e) {
23.    echo $sql . "<br>" . $e->getMessage();
24.}
25.
26.$conn = null;
27.??>
```

3.5.2.4. Xóa dữ liệu trong cơ sở dữ liệu MySQL

Câu lệnh DELETE được sử dụng để xóa các bản ghi khỏi một bảng:

```
DELETE FROM table_name
        WHERE some_column = some_value
```

Lưu ý: Mệnh đề WHERE trong cú pháp DELETE chỉ định bản ghi hoặc các bản ghi sẽ bị xóa. Nếu bỏ qua mệnh đề WHERE thì tất cả các bản ghi sẽ bị xóa!

Các ví dụ sau xóa bản ghi có id = 3 trong bảng "MyGuests".

* Sử dụng MySQLi hướng đối tượng

```
1. <?php
2. $servername = "localhost";
3. $username = "username";
```

```

4. $password = "password";
5. $dbname = "myDB";
6. // Create connection
7. $conn = new mysqli($servername, $username, $password, $dbname);
8. // Check connection
9. if ($conn->connect_error) {
10.     die("Connection failed: " . $conn->connect_error);
11.}
12.
13.// sql to delete a record
14.$sql = "DELETE FROM MyGuests WHERE id=3";
15.
16.if ($conn->query($sql) === TRUE) {
17.    echo "Record deleted successfully";
18.} else {
19.    echo "Error deleting record: " . $conn->error;
20.}
21.
22.$conn->close();
23.??>

```

* Sử dụng MySQLi hướng thủ tục

```

1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDB";
6. // Create connection
7. $conn = mysqli_connect($servername, $username, $password, $dbname);
8. // Check connection
9. if (!$conn) {
10.     die("Connection failed: " . mysqli_connect_error());
11.}
12.
13.// sql to delete a record
14.$sql = "DELETE FROM MyGuests WHERE id=3";
15.
16.if (mysqli_query($conn, $sql)) {
17.    echo "Record deleted successfully";
18.} else {
19.    echo "Error deleting record: " . mysqli_error($conn);
20.}

```

```

21.
22. mysqli_close($conn);
23. ?>

* Sử dụng PDO

1. <?php
2. $servername = "localhost";
3. $username = "username";
4. $password = "password";
5. $dbname = "myDBPDO";
6.
7. try {
8.     $conn = new
        PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
9.     // set the PDO error mode to exception
10.    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
11.
12.    // sql to delete a record
13.    $sql = "DELETE FROM MyGuests WHERE id=3";
14.
15.    // use exec() because no results are returned
16.    $conn->exec($sql);
17.    echo "Record deleted successfully";
18.} catch(PDOException $e) {
19.    echo $sql . "<br>" . $e->getMessage();
20.}
21.
22. $conn = null;
23. ?>

```

3.5.3. Đóng kết nối

Kết nối sẽ tự động đóng khi tập lệnh kết thúc. Để đóng kết nối trước đó, sử dụng cú pháp sau:

- MySQLi hướng đối tượng:	<code>\$conn->close();</code>
- MySQLi hướng thủ tục:	<code>mysqli_close(\$conn);</code>
- PDO:	<code>\$conn = null;</code>

3.6. Lập trình hướng đối tượng

3.6.1. Hàm

a. Khái niệm

Đây là một trong những thuận lợi nhất trong phát triển ứng dụng. Các hàm cho phép phát triển việc sử dụng lại và dễ dàng chỉnh sửa các thành phần mà thật sự hữu

ích khi phát triển các ứng dụng web tương tự như trong quan niệm và tiện ích. Các kết quả của hàm rất ngắn gọn, dễ hiểu và dễ đọc.

Hàm là một đoạn các mã lệnh với một mục đích cụ thể và phải được gán một tên duy nhất. Tên hàm có thể được gọi tại bất kỳ đâu trong chương trình, cho phép các đoạn mã thể hiện bởi tên của nó được thực hiện lặp lại khi cần thiết. Đây là một thuận lợi bởi cùng một đoạn mã lệnh chỉ được viết một lần nhưng có thể được chỉnh sửa dễ dàng khi cần thiết. Hàm có hai loại: Hàm có sẵn và hàm tự định nghĩa.

- Hàm có sẵn: Là các hàm tích hợp sẵn trong PHP khi chúng ta cài đặt (Ví dụ: echo(), print(),...).

- Hàm tự định nghĩa: Là các hàm do người lập trình tự định nghĩa (tự tạo).

Có thể tạo một hàm tại bất kỳ nơi nào trong mã lệnh chương trình PHP. Tuy nhiên, với mục đích tổ chức mã lệnh thuận lợi thì đặt tất cả các hàm được dự định sẽ sử dụng trong mã lệnh tại đầu mỗi trang web. Một phương thức khác cho việc tổ chức hàm để giảm đi sự dư thừa mã lệnh và tăng việc sử dụng lại là đặt các hàm trong những file riêng (được xem như là một thư viện). Điều này rất thuận lợi bởi có thể dùng các hàm lặp đi lặp lại trong những ứng dụng khác nhau mà không tạo ra các mã lệnh dư thừa và do đó giảm bớt các nguy cơ gây lỗi do việc viết lại mã lệnh.

Một hàm thường bao gồm ba phần phân biệt:

- Tên của hàm.
- Cặp dấu ngoặc () chứa các tham số tùy chọn nhập vào.
- Phần thân của hàm, nằm trong cặp dấu {}.

Cú pháp:

```
function functionName(danh sách tham số) {  
    mã lệnh thực hiện trong thân hàm;  
}
```

Ví dụ khai báo một hàm sau:

```
1. function display_copyright($site_name) {  
2.     print "Copyright © 2000 $site_name. All Rights Reserved.";  
3. }
```

Để sử dụng hàm, chúng ta có thể được gọi ở bất kỳ đâu trong chương trình với tên của nó, theo sau là cặp dấu ngoặc đơn chứa (hoặc không chứa) các tham số của hàm và dấu chấm phẩy kết thúc dòng lệnh “;”.

** Hàm không có tham số*

Khi danh sách tham số không có thì hàm được gọi là hàm không có tham số.

Ví dụ:

```
1. <?php  
2. function MyFunction(){  
3.     echo "Hello World!";
```

```

4.     }
5.     MyFunction();
6. ?>

```

Trong ví dụ trên, một hàm có tên MyFunction được tạo để in ra dòng chữ Hello World!. Hàm không yêu cầu tham số để thực thi.

** Hàm có tham số*

Trong trường hợp khi khai báo hàm có danh sách tham số (được đặt trong cặp dấu ()) thì hàm gọi là có tham số. Tham số là một biến được sử dụng để truyền thông tin vào trong một hàm. Tham số của hàm còn được gọi là tham số hình thức.

Ví dụ 1:

```

1. <?php
2.     function MySum($x, $y){
3.         $z = $x + $y;
4.         echo "Sum of $x and $y is: $z.\n";
5.     }
6.
7.     $a = 15;
8.     $b = 10;
9.     MySum($a, $b);
10. ?>

```

Kết quả:

Sum of 15 and 10 is: 25

Trong ví dụ trên, hàm MySum() có hai tham số x và y , còn được gọi là tham số hình thức; a và b là hai biến còn gọi là đối số của hàm. Khi gọi hàm, hai đối số được truyền vào cho hàm thông qua hai tham số hình thức x và y . Có thể truyền trực tiếp giá trị cho hàm mà không phải thông qua đối số.

Ví dụ 1:

```

1. MySum(5, 10);

```

Ví dụ 2: Tạo trang tính diện tích hình chữ nhật.

```

1. <?php
2. //Khai báo hàm
3. function tinh_dt_hcn($d, $r)
4. {
5.     echo "Diện tích là: ". $d * $r;
6. }
7.
8. //Khai báo giá trị mặc định cho biến, nếu không sẽ báo lỗi ở
   dòng gọi hàm do biến không tồn tại.
9. $dai = 0;

```

```

10.$rong = 0;
11.
12.//Xử lý khi người dùng nhấn nút submit (nút Tính)
13.if (isset($_POST['btOk']))//Nếu người dùng nhấn nút Tính
14.{
15.    //Nếu tồn tại $_POST['dai'] thì gán biến $dai = $_POST['dai'],
        ngược lại $dai = 0.
16.    $dai = isset($_POST['dai']) ? $_POST['dai'] : 0;
17.    //Nếu tồn tại $_POST['rong'] thì gán biến $rong = $_POST['rong'],
        ngược lại $rong = 0.
18.    $rong = isset($_POST['rong']) ? $_POST['rong'] : 0;
19.}
20.??>
21.<!DOCTYPE html>
22.<html lang="en">
23.<head>
24.    <title>Tính diện tích</title>
25.    <meta charset="utf-8">
26.    <meta name="viewport" content="width=device-width, initial-scale=1">
27.    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap
        @4.6.2/dist/css/bootstrap.min.css">
28.    <script src="https://cdn.jsdelivr.net/npm/jquery@3.6.3/dist/jquery.
        slim.min.js"></script>
29.    <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd
        /popper.min.js"></script>
30.    <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/b
        ootstrap.bundle.min.js"></script>
31.</head>
32.<body>
33.
34.    <div class="container">
35.        <h2>Tính diện tích hình chữ nhật</h2>
36.        <form action="" method="post" class="was-validated my-3">
37.            <div class="form-group">
38.                <label>Chiều dài:</label>
39.                <input type="text" class="form-control" name="dai" required>
40.            </div>
41.            <div class="form-group">
42.                <label>Chiều rộng:</label>
43.                <input type="text" class="form-control" name="rong" required>
44.            </div>
45.

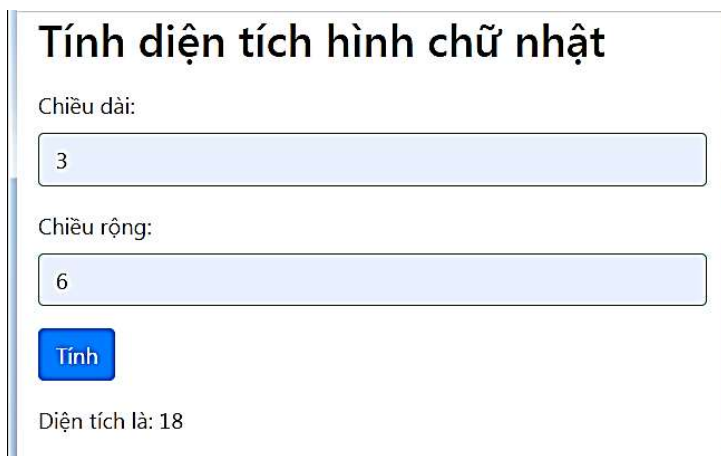
```

```

46.     <button type="submit" class="btn btn-primary"
        name="btOk">Tính</button>
47.     </form>
48.
49.     <?php
50.         //Hiển thị kết quả.
51.         tinh_dt_hcn($dai, $rong);
52.     ?>
53. </div>
54.</body>
55.</html>

```

Kết quả:



Hình 3.2. Giao diện trang tính diện tích hình chữ nhật

Giá trị mặc định:

Giá trị mặc định có thể được gán cho một tham số tại thời điểm tạo hàm. Khi hàm được gọi mà không chỉ rõ tham số thì nó sẽ sử dụng giá trị mặc định.

Trong ví dụ dưới đây, giá trị tham số mặc định được sử dụng để thực hiện phép tính tổng trên hai, ba hoặc bốn biến bằng cách sử dụng cùng một hàm có tên MySum(). Giá trị mặc định được đặt trong khi xác định hàm. Khi hàm được gọi mà không có tham số thì hàm đó sẽ sử dụng giá trị mặc định của tham số.

```

1. <?php
2.     function MySum($p, $q, $r=0, $s=0){
3.         echo $p+$q+$r+$s;
4.     }
5.     $a = 15; $b = 10; $c = 5; $d = 1;
6.     MySum($a,$b)."\n";    //Tham số r, s được lấy giá trị mặc định.
7.     MySum($a,$b,$c)."\n"; //Tham số s là mặc định.
8.     MySum($a,$b,$c,$d)."\n";
9. ?>

```

Kết quả:

25

30

31

Ví dụ 2: Hàm tính diện tích hình chữ nhật, sử dụng khai báo giá trị mặc định.

```
1. <?php
2. //Khai báo hàm
3. function tinh_dt_hcn($d=0, $r=0)
4. {
5.     echo "Diện tích là: ". $d * $r;
6. }
7. ?>
```

** Hàm trả về giá trị*

Việc trả về một giá trị từ hàm là rất hữu ích và được thực hiện bằng cách gán giá trị gọi hàm cho một biến. Bất kỳ kiểu dữ liệu nào cũng được trả về từ một hàm bao gồm cả kiểu mảng và danh sách. Để trả về giá trị từ hàm thì trong hàm phải có lệnh *return*.

Ví dụ 1:

```
1. <?php
2. function MySum($x, $y){
3.     $z = $x + $y;
4.     return $z;
5. }
6.
7. $a = 15;
8. $b = 10;
9. echo MySum($a, $b);
10. ?>
```

Kết quả:

25

Ví dụ 2: Tính diện tích hình chữ nhật, sử dụng hàm trả về giá trị.

```
1. <?php
2. //Khai báo hàm
3. function tinh_dt_hcn($d=0, $r=0)
4. {
5.     return $d * $r;
6. }
7.
8. //Khai báo giá trị mặc định cho biến,
   nếu không sẽ báo lỗi ở dòng hiển thị kết quả.
```



```

9. $dientich = 0;
10. //Xử lý khi người dùng nhấn nút submit (nút Tính)
11. if (isset($_POST['btOk'])) //Nếu người dùng nhấn nút Tính
12. {
13.     //Nếu tồn tại $_POST['dai'] thì gán biến $dai = $_POST['dai'],
        ngược lại $dai = 0.
14.     $dai = isset($_POST['dai']) ? $_POST['dai'] : 0;
15.     //Nếu tồn tại $_POST['rong'] thì gán biến $rong = $_POST['rong'],
        ngược lại $rong = 0.
16.     $rong = isset($_POST['rong']) ? $_POST['rong'] : 0;
17.     //Gọi hàm tính diện tích.
18.     $dientich = tinh_dt_hcn($dai, $rong);
19. }
20. ?>
21. <!DOCTYPE html>
22. <html lang="en">
23. <head>
24.     <title>Tính diện tích</title>
25.     <meta charset="utf-8">
26.     <meta name="viewport" content="width=device-width, initial-scale=1">
27.     <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap
        @4.6.2/dist/css/bootstrap.min.css">
28.     <script src="https://cdn.jsdelivr.net/npm/jquery@3.6.3/dist/jquery.
        slim.min.js"></script>
29.     <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.1/dist/umd
        /popper.min.js"></script>
30.     <script src="https://cdn.jsdelivr.net/npm/bootstrap@4.6.2/dist/js/b
        ootstrap.bundle.min.js"></script>
31. </head>
32. <body>
33.     <div class="container">
34.         <h2>Tính diện tích hình chữ nhật</h2>
35.         <form action="" method="post" class="was-validated my-3">
36.             <div class="form-group">
37.                 <label>Chiều dài:</label>
38.                 <input type="text" class="form-control" name="dai" required>
39.             </div>
40.             <div class="form-group">
41.                 <label>Chiều rộng:</label>
42.                 <input type="text" class="form-control" name="rong" required>
43.             </div>
44.

```

```

45.         <button type="submit" class="btn btn-primary"
           name="btOk">Tính</button>
46.     </form>
47.
48.     <?php
49.         //Hiển thị kết quả.
50.         echo "Diện tích là: ". $dientich;
51.     ?>
52. </div>
53.</body>
54.</html>

```

* Một số quy tắc khi tạo hàm

- Tên hàm phải bắt đầu với một ký tự hoặc dấu gạch dưới (gạch nối).
- Tên hàm không được bắt đầu bằng số.
- Tên của hàm phải là duy nhất.
- Tên của hàm không chứa dấu cách.
- Tên của hàm nên mô tả được công việc của nó.
- Hàm chấp nhận tham số và có thể trả về giá trị.

b. Khai báo kiểu dữ liệu của các tham số và của hàm

Trong PHP, không cần khai báo kiểu dữ liệu của các tham số trong một hàm. PHP tự động lấy kiểu dữ liệu của một biến, tùy thuộc vào giá trị của nó. Điều này có thể dẫn đến lỗi, ví dụ khi chuỗi được chuyển qua một hàm thay vì kiểu dữ liệu số. Do đó, trong PHP 7 trở đi, khai báo kiểu được thêm vào. Điều này cung cấp một tùy chọn để chỉ định kiểu dữ liệu của các tham số.

Trong ví dụ dưới đây, một hàm có tên MySum() được định nghĩa sẽ nhận kiểu dữ liệu số nguyên làm đối số.

```

1. <?php
2.     function MySum(int $x, int $y){
3.         $z = $x + $y;
4.         return $z;
5.     }
6.     $a = 15;
7.     $b = 10;
8.     echo MySum($a, $b);
9. ?>

```

Kết quả:

25

Cũng như trình bày ở trên, từ PHP 7 trở đi, PHP sẽ hỗ trợ khai báo kiểu dữ liệu trả về. Bằng cách thêm dấu hai chấm ":" sau khi khai báo tham số hàm.

Trong ví dụ dưới đây, một hàm có tên MySum() được định nghĩa. Nó được khai báo để trả về kiểu dữ liệu float.

```
1. <?php
2.     function MySum(float $x, int $y) : float
3.     {
4.         $z = $x * $y;
5.         return $z;
6.     }
7.     $a = 15.25;
8.     $b = 10;
9.     echo MySum($a, $b);
10. ?>
```

Kết quả:

152,5

c. Truyền tham chiếu cho hàm

Mặc định tất cả các đối số truyền vào hàm đều là truyền bằng giá trị. Điều này có nghĩa là khi các đối số được truyền đến hàm được gọi, giá trị được truyền thông qua các biến tạm (tham số hình thức). Mọi thao tác chỉ thực hiện trên biến tạm này nên nó không hề tác động đến biến chính. Điều này có nghĩa là nếu truyền bằng giá trị thì trong hàm nếu ta tác động đến giá trị biến truyền vào thì sau khi thoát khỏi hàm giá trị đó không thay đổi.

Ví dụ:

```
1. // Biến
2. $a = 1;
3. // Hàm tăng giá trị tham số truyền vào lên 1
4. function tang_len_1($a)
5. {
6.     return $a + 1;
7. }
8. // Xuất giá trị trả về của hàm
9. echo tang_len_1($a);
10. // Xuất giá trị của biến
11. echo $a;
```

Kết quả xuất ra màn hình sẽ là 2 và 1. Như thế biến \$a vẫn giữ nguyên giá trị bằng 1 sau khi hàm kết thúc. Còn trong hàm thì biến \$a có giá trị là 2.

Khi các đối số được truyền bằng giá trị thì giá trị của các đối số của hàm đang gọi không bị thay đổi. Tuy nhiên đôi khi chúng ta muốn những giá trị đó thay đổi theo thì lúc này phải truyền biến vào hàm dạng *truyền tham chiếu*.

Ví dụ:

```
1. // Biến
2. $a = 1;
3. // Hàm tăng giá trị tham số truyền vào lên 1
4. function tang_len_1(&$a)
5. {
6.     $a = $a + 1;
7.     return $a;
8. }
9. // Xuất giá trị trả về của hàm
10. echo tang_len_1($a);
11. // Xuất giá trị của biến
12. echo $a;
```

Kết quả xuất ra màn hình là 2 và 2. như vậy biến \$a đã bị thay đổi. Sự khác biệt ở đoạn mã này so với đoạn mã trên là biến \$a ở hàm tang_len_1() có dấu &, đây chính là cú pháp trong PHP báo cho trình biên dịch biết đó là một biến ở dạng tham chiếu.

d. Các hàm lồng nhau

Trong ví dụ *display_copyright* ở trên, có thể giới hạn nhu cầu chỉnh sửa ngày tháng cùng với việc dùng một hàm lồng định nghĩa trước của PHP trong hàm *display_copyright()*:

```
1. function display_copyright($site_name) {
2.     print "Copyright © ". $site_name. " All Rights Reserved.";
3. }
```

Cũng có thể lồng các khai báo hàm trong một hàm khác. Tuy nhiên, việc lồng một khai báo hàm không ngụ ý rằng nó sẽ được bảo vệ trong phạm vi giới hạn mà nó được khai báo. Hơn thế nữa, một hàm được lồng vào không thừa kế các tham số nhập vào của hàm "cha - mẹ", các tham số phải được chuyển sang hàm được lồng chỉ khi chúng được chuyển sang hàm khác. Tuy nhiên, việc lồng các khai báo hàm là thật sự hữu ích cho các lý do quản lý mã và đảm bảo sự rõ ràng của tổ chức mã.

Ví dụ:

```
1. function display_footer($site_name) {
2.     function display_copyright($site_name) {
3.         print "Copyright © ". $site_name. " All Rights Reserved.";
4.     }
5.     print "<center>
6.     <a href=\"\">home</a> | <a href = \"\">recipes</a> |
7.     <a href=\"\">events</a><br>
8.     <a href=\"\">tutorials</a> |
9.     <a href = \"\">about</a> |
10.    <a href = \"\">contact us</a><br>";
```

```

11. display_copyright($site_name);
12. print "</center>";
13.}
14.$site_name = "PHP Recipes";
15.display_footer($site_name);

```

Sau khi thực hiện, đoạn *script* trên sẽ cho kết quả như sau:

```

home | recipes | events
tutorials | about | contact us

```

Các hàm lồng nhau trong PHP không được bảo vệ bởi việc được gọi từ bất kỳ đoạn *script* nào trong chương trình, nhưng chúng không thể được gọi lại cho đến sau khi các hàm cha mẹ được gọi. Việc gọi thử một hàm lồng trước khi gọi hàm cha mẹ của nó sẽ sinh ra một thông báo lỗi.

e. Hàm đệ quy

Hoạt động của một hàm gọi lại bản thân chính nó nhiều lần để thỏa mãn một vài phép toán thật sự là một sức mạnh. Nếu được sử dụng một cách đúng đắn, các việc gọi hàm đệ quy có thể tiết kiệm được khoảng trống không đáng và dư thừa trong một *script* và đặc biệt hữu ích cho việc thực hiện các thủ tục lặp đi lặp lại.

Ví dụ: sử dụng một hàm lặp đệ quy để tính tổng một tập hợp các số *integer*.

```

1. function summation ($count)
2. {
3.     if ($count != 0)
4.         return $count + summation($count- 1);
5. }
6. $sum = summation(10);
7. print "Summation = $sum";

```

Kết quả của đoạn mã trên sẽ là *Summation=55*

Sử dụng hàm đệ quy có thể cải thiện tốc độ trong một chương trình nếu hàm được gọi đủ thường xuyên. Tuy nhiên, phải cẩn thận khi viết các thủ tục đệ quy, nếu mã không đúng sẽ dẫn đến việc lặp không dừng được.

f. Xây dựng các thư viện hàm

Các thư viện hàm là một trong những cách hữu ích để tiết kiệm thời gian khi xây dựng các ứng dụng. Ví dụ, có thể viết một loạt các hàm cho việc sắp xếp các mảng. Ta có thể sử dụng lại các hàm này trong các ứng dụng khác nhau. Hơn là việc thường xuyên viết lại hay copy và paste các hàm này vào các script mới, nó thật sự tiện lợi khi đặt tất cả các hàm liên quan đến việc sắp xếp trong cùng một file phân biệt. File này sẽ chứa các tiêu đề dễ dàng nhận ra, chẳng hạn như *array_sorting.inc*.

Ví dụ:

```
1. <?php
2. //file: array_sorting.inc
3. function merge_sort($array, $tmparray, $right, $left) {
4.     . . .
5. }
6. function bubble_sort($array, $n) {
7.     . . .
8. }
9. function quick_sort($array, $right, $left) {
10.    . . .
11.}
12. ?>
```

Hàm thư viện *array_sorting.inc* này sẽ hoạt động như là một chỗ chứa cho tất cả các hàm sắp xếp mảng. Điều này thật sự hữu ích bởi ta có thể tổ chức các hàm một cách hiệu quả theo mục đích cho phép dễ dàng tìm kiếm khi cần thiết. Một khi xây dựng cho chính mình một thư viện hàm, ta có thể sử dụng các câu lệnh *include()* và *require()* của PHP để chứa toàn bộ các file thư viện vào trong một *script* do đó làm cho tất cả các hàm đều có sẵn để sử dụng.

Giả sử nếu muốn dùng thư viện *array_sorting.inc* trong một *script* thì có thể thực hiện như sau:

```
include ("array_sorting.inc");
Bây giờ có thể sử dụng bất kỳ hàm nào trong array_sorting.inc
$some_array = (50, 42, 35, 46);
//sử dụng phương pháp sắp xếp bubble_sort()
$sorted_array = bubble_sort($some_array, 1);
```

3.6.2. Lớp và đối tượng

3.6.2.1. Khái niệm lớp

Lớp (Class) theo lập trình là một thành phần cơ bản chứa tất cả các thuộc tính và phương thức của 1 đối tượng sẽ được tạo từ class. Một lớp được định nghĩa bằng cách sử dụng từ khóa *class*, theo sau là tên của lớp và một cặp dấu ngoặc nhọn (*{}*). Tất cả các thuộc tính và phương thức của nó nằm bên trong dấu ngoặc nhọn.

Cú pháp:

```
<?php
class Name_of_Class{
    .....
}
?>
```

Giả sử có một lớp tên là Fruit (trái cây). Một trái cây có thể có các thuộc tính như tên, màu sắc, trọng lượng,... Có thể xác định các biến như \$name, \$color và \$weight để giữ các giá trị của các thuộc tính này.

Khi các đối tượng riêng lẻ (quả táo, chuối,...) được tạo, chúng kế thừa tất cả các thuộc tính và hành vi từ lớp, nhưng mỗi đối tượng sẽ có các giá trị khác nhau cho các thuộc tính.

Ví dụ:

```
1. <?php
2. class Fruit {
3.     // Properties
4.     public $name;
5.     public $color;
6.     // Methods
7.     function set_name($name) {
8.         $this->name = $name;
9.     }
10.    function get_name() {
11.        return $this->name;
12.    }
13.}
14. ?>
```

Lưu ý:

- Trong một lớp, các biến được gọi là thuộc tính và hàm được gọi là phương thức.
- Không thể khai báo hai phương thức trùng tên.
- Phương thức phải được khai báo ngay bên trong khai báo lớp.
- Dùng biến giả *\$this* để truy xuất các thuộc tính và phương thức trong lớp.

3.6.2.2. Đối tượng

Chúng ta có thể tạo nhiều đối tượng từ một lớp. Đối tượng chính là thể hiện của một lớp. Mỗi đối tượng có tất cả các thuộc tính và phương thức được định nghĩa trong lớp, nhưng chúng sẽ có các giá trị thuộc tính khác nhau. Các đối tượng của một lớp được tạo bằng cách sử dụng từ khóa *new*.

Cú pháp:

```
$tên_đối_tượng = new tên_lớp();
```

Trong ví dụ sau, \$apple và \$banana là các thể hiện của lớp Fruit:

```
1. <?php
2. class Fruit {
3.     // Properties
4.     public $name;
5.     public $color;
```

```

6.
7.  // Methods
8.  function set_name($name) {
9.      $this->name = $name;
10. }
11. function get_name() {
12.     return $this->name;
13. }
14.}
15.
16.$apple = new Fruit();
17.$banana = new Fruit();
18.$apple->set_name('Apple');
19.$banana->set_name('Banana');
20.
21.echo $apple->get_name();
22.echo "<br>";
23.echo $banana->get_name();
24.??>

```

Lưu ý: Để truy xuất đến thuộc tính hoặc phương thức của lớp thông qua đối tượng, chúng ta sử dụng tên biến đối tượng theo sau là toán tử “->”, tiếp theo là tên thuộc tính hoặc phương thức.

3.6.2.3. Phương thức khởi tạo và hủy

Phương thức khởi tạo cho phép khởi tạo các thuộc tính của đối tượng khi tạo đối tượng. Phương thức khởi tạo có tên được xác định bằng từ khóa `__construct`. Nếu tạo một phương thức `__construct()` thì PHP sẽ tự động gọi phương thức này khi tạo một đối tượng từ một lớp.

Cú pháp:

```

function __construct(danh sách tham số) {
    Các lệnh khởi tạo;
}

```

Lưu ý rằng phương thức khởi tạo bắt đầu bằng hai dấu gạch dưới (`__`).

Ví dụ sau minh họa việc sử dụng phương thức khởi tạo giúp ta không phải gọi phương thức `set_name()` từ đó giảm lượng mã lệnh:

```

1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.
6.     function __construct($name) {

```



```

7.     $this->name = $name;
8. }
9. function get_name() {
10.     return $this->name;
11. }
12.}
13.
14.$apple = new Fruit("Apple");
15.echo $apple->get_name();
16.??>

```

Phương thức hủy (dọn dẹp) được gọi khi đối tượng bị hủy hoặc tập lệnh bị dừng hoặc thoát. Phương thức hủy có tên được xác định bằng từ khóa `__destruct`. Nếu tạo một phương thức `__destruct()` trong lớp thì PHP sẽ tự động gọi phương thức này ở cuối tập lệnh.

Cú pháp:

```

function __destruct()
{
    //Các lệnh dọn dẹp;
}

```

Lưu ý rằng phương thức hủy bắt đầu bằng hai dấu gạch dưới (`__`).

Ví dụ sau có phương thức `__construct()` được gọi tự động khi tạo một đối tượng từ một lớp và phương thức `__destruct()` được gọi tự động ở cuối tập lệnh.

```

1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.
6.     function __construct($name) {
7.         $this->name = $name;
8.     }
9.     function __destruct() {
10.         echo "The fruit is {$this->name}.";
11.     }
12.}
13.
14.$apple = new Fruit("Apple");
15.??>

```

3.6.2.4. Phạm vi truy cập thuộc tính và phương thức

Các thuộc tính và phương thức có thể có các phạm vi truy cập khác nhau. Có ba loại phạm vi truy cập: *public*, *protected* và *private*. Trong đó:

- *public*: Thuộc tính hoặc phương thức có thể được truy cập từ mọi nơi. Đây là mặc định.

- *protected*: Thuộc tính hoặc phương thức có thể được truy cập trong lớp và bởi các lớp dẫn xuất từ lớp đó.

- *private*: Thuộc tính hoặc phương thức CHỈ có thể được truy cập trong lớp.

Trong ví dụ sau, ta đã thêm ba phạm vi truy cập khác nhau vào ba thuộc tính: *name*, *color*, *weight*. Khi thiết lập giá trị cho thuộc tính *name* thì mã lệnh sẽ hoạt động tốt vì thuộc tính *name* là công khai và có thể được truy cập từ mọi nơi. Tuy nhiên, nếu cố gắng thiết lập giá trị thuộc tính *color* hoặc *weight*, mã lệnh sẽ dẫn đến lỗi nghiêm trọng vì thuộc tính *color* và *weight* được bảo vệ và riêng tư.

```
1. <?php
2. class Fruit {
3.     public $name;
4.     protected $color;
5.     private $weight;
6. }
7.
8. $mango = new Fruit();
9. $mango->name = 'Mango'; // OK
10. $mango->color = 'Yellow'; // ERROR
11. $mango->weight = '300'; // ERROR
12. ?>
```

Trong ví dụ tiếp theo, ta đã thêm phạm vi truy cập vào hai phương thức. Khi gọi phương thức *set_color()* hoặc *set_weight()* thì sẽ dẫn đến một lỗi nghiêm trọng vì hai phương thức này được coi là được bảo vệ và riêng tư, ngay cả khi tất cả các thuộc tính đều công khai.

```
1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.     public $weight;
6.
7.     function set_name($n) { // a public function (default)
8.         $this->name = $n;
9.     }
10.     protected function set_color($n) { // a protected function
```

```

11.     $this->color = $n;
12. }
13. private function set_weight($n) { // a private function
14.     $this->weight = $n;
15. }
16.}
17.
18.$mango = new Fruit();
19.$mango->set_name('Mango'); // OK
20.$mango->set_color('Yellow'); // ERROR
21.$mango->set_weight('300'); // ERROR
22.??>

```

3.6.2.5. Kế thừa lớp

Khi một lớp kế thừa (được gọi là lớp con hay lớp dẫn xuất) từ một lớp khác (được gọi là lớp cha), lớp con sẽ kế thừa (inherited) tất cả các thuộc tính và phương thức có phạm vi *public* và *protected* từ lớp cha. Ngoài ra, lớp con có thể có các thuộc tính và phương thức riêng. Một lớp kế thừa được định nghĩa bằng cách sử dụng từ khóa *extends*.

Ví dụ:

```

1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.     public function __construct($name, $color) {
6.         $this->name = $name;
7.         $this->color = $color;
8.     }
9.     public function intro() {
10.         echo "The fruit is {$this->name} and the color is {$this->color}.";
11.     }
12.}
13.// Strawberry is inherited from Fruit
14.class Strawberry extends Fruit {
15.     public function message() {
16.         echo "Am I a fruit or a berry? ";
17.     }
18.}
19.$strawberry = new Strawberry("Strawberry", "red");
20.$strawberry->message();
21.$strawberry->intro();
22.??>

```

Trong ví dụ trên, lớp Strawberry được kế thừa từ lớp Fruit. Điều này có nghĩa là lớp Strawberry có thể sử dụng các thuộc tính public \$name và \$color cũng như các phương thức public __construct() và intro() từ lớp Fruit do tính kế thừa. Lớp Strawberry cũng có phương thức riêng: message().

Các thuộc tính hoặc phương thức được bảo vệ (protected) có thể được truy cập trong lớp và bởi các lớp dẫn xuất từ lớp đó.

Ví dụ:

```
1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.     public function __construct($name, $color) {
6.         $this->name = $name;
7.         $this->color = $color;
8.     }
9.     protected function intro() {
10.        echo "The fruit is {$this->name} and the color is {$this->color}.";
11.    }
12.}
13.
14.class Strawberry extends Fruit {
15.    public function message() {
16.        echo "Am I a fruit or a berry? ";
17.    }
18.}
19.
20.//Try to call all three methods from outside class
21.$strawberry = new Strawberry("Strawberry", "red");//__construct() is public
22.$strawberry->message(); //OK. message() is public
23.$strawberry->intro(); //ERROR. intro() is protected
24.}>
```

Trong ví dụ trên, ta thấy rằng nếu chúng ta cố gắng gọi một phương thức được bảo vệ (phương thức intro()) từ bên ngoài lớp thì sẽ gặp lỗi. Để khắc phục lỗi này, chỉ có thể gọi phương thức được bảo vệ của lớp cha bên trong lớp kế thừa (lớp con).

Ví dụ:

```
1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.     public function __construct($name, $color) {
```

```

6.     $this->name = $name;
7.     $this->color = $color;
8. }
9. protected function intro() {
10.     echo "The fruit is {$this->name} and the color is {$this->color}.";
11. }
12.}
13.
14.class Strawberry extends Fruit {
15.     public function message() {
16.         echo "Am I a fruit or a berry? ";
17.         // Call protected method from within derived class - OK
18.         $this -> intro();
19.     }
20.}
21.
22.$strawberry = new Strawberry("Strawberry", "red");
    // OK. __construct() is public
23.$strawberry->message(); // OK. message() is public and it calls intro()
    (which is protected) from within the derived class
24.??>

```

3.6.2.6. Ghi đè các phương thức kế thừa

Các phương thức kế thừa có thể được ghi đè (còn gọi là nạp chồng) bằng cách xác định lại các phương thức (sử dụng cùng tên) trong lớp con.

Trong ví dụ dưới đây. Các phương thức `__construct()` và `intro()` trong lớp con (Strawberry) sẽ ghi đè lên các phương thức `__construct()` và `intro()` trong lớp cha (Fruit):

```

1. <?php
2. class Fruit {
3.     public $name;
4.     public $color;
5.     public function __construct($name, $color) {
6.         $this->name = $name;
7.         $this->color = $color;
8.     }
9.     public function intro() {
10.         echo "The fruit is {$this->name} and the color is {$this->color}.";
11.     }
12.}
13.
14.class Strawberry extends Fruit {
15.     public $weight;

```

```

16. public function __construct($name, $color, $weight) {
17.     $this->name = $name;
18.     $this->color = $color;
19.     $this->weight = $weight;
20. }
21. public function intro() {
22.     echo "The fruit is {$this->name}, the color is {$this->
    >color}, and the weight is {$this->weight} gram.";
23. }
24.}
25.
26.$strawberry = new Strawberry("Strawberry", "red", 50);
27.$strawberry->intro();
28.??>

```

3.6.2.7. Chặn kế thừa, chặn ghi đè phương thức

Từ khóa *final* có thể được sử dụng để ngăn chặn kế thừa lớp hoặc ngăn chặn ghi đè phương thức.

Ví dụ:

```

1. <?php
2. final class Fruit {
3.     // some code
4. }
5. // will result in error
6. class Strawberry extends Fruit {
7.     // some code
8. }
9. ??>
10.<?php
11.class Fruit {
12. final public function intro() {
13.     // some code
14. }
15.}
16.class Strawberry extends Fruit {
17. // will result in error
18. public function intro() {
19.     // some code
20. }
21.}
22.??>

```

3.6.2.8. Hằng số trong lớp

Các hằng số của lớp có thể hữu ích nếu cần định nghĩa một số dữ liệu không đổi trong một lớp. Một hằng số lớp được khai báo bên trong một lớp với từ khóa *const*. Các hằng số lớp phân biệt chữ hoa chữ thường. Tuy nhiên, nên đặt tên hằng bằng chữ in hoa.

Có thể truy cập một hằng từ bên ngoài lớp bằng cách sử dụng tên lớp theo sau là toán tử phân giải phạm vi (::) và theo sau là tên hằng.

Ví dụ:

```
1. <?php
2. class Goodbye {
3.     const LEAVING_MESSAGE = "Thank you for visiting!";
4. }
5.
6. echo Goodbye::LEAVING_MESSAGE;
7. ?>
```

Hoặc có thể truy cập một hằng số từ bên trong lớp bằng cách sử dụng từ khóa *self* theo sau là toán tử phân giải phạm vi (::) theo sau là tên hằng số.

Ví dụ:

```
1. <?php
2. class Goodbye {
3.     const LEAVING_MESSAGE = "Thank you for visiting!";
4.     public function byebye() {
5.         echo self::LEAVING_MESSAGE;
6.     }
7. }
8.
9. $goodbye = new Goodbye();
10. $goodbye->byebye();
11. ?>
```

3.6.2.9. Lớp trừu tượng

Các lớp và phương thức trừu tượng (abstract) là khi lớp cha có một phương thức được đặt tên, nhưng cần (các) lớp con của nó để thực hiện các tác vụ. Lớp trừu tượng là lớp chứa ít nhất một phương thức trừu tượng. Một phương thức trừu tượng là một phương thức được khai báo, nhưng không được triển khai trong mã. Một lớp hoặc phương thức trừu tượng được định nghĩa bằng từ khóa *abstract*.

Khi kế thừa từ một lớp trừu tượng, phương thức của lớp con phải được định nghĩa có cùng tên và phạm vi truy cập ít bị hạn chế hơn. Vì vậy, nếu phương thức trừu tượng được định nghĩa là được bảo vệ thì phương thức của lớp con phải được định nghĩa là được bảo vệ hoặc công khai, nhưng không phải là riêng tư. Ngoài ra, loại và

số lượng đối số bắt buộc phải giống nhau. Tuy nhiên, các lớp con có thể có thêm các đối số tùy chọn. Vì vậy, khi một lớp con được kế thừa từ một lớp trừu tượng, ta có các quy tắc sau:

- Phương thức của lớp con phải được định nghĩa cùng tên và nó khai báo lại phương thức trừu tượng của lớp cha.

- Phương thức của lớp con phải được định nghĩa với cùng một công cụ sửa đổi truy cập ít bị hạn chế hơn.

- Số lượng đối số bắt buộc phải giống nhau. Tuy nhiên, lớp con có thể có thêm các đối số tùy chọn.

Ví dụ:

```
1. <?php
2. // Parent class
3. abstract class Car {
4.     public $name;
5.     public function __construct($name) {
6.         $this->name = $name;
7.     }
8.     abstract public function intro() : string;
9. }
10.// Child classes
11.class Audi extends Car {
12.    public function intro() : string {
13.        return "Choose German quality! I'm an $this->name!";
14.    }
15.}
16.class Volvo extends Car {
17.    public function intro() : string {
18.        return "Proud to be Swedish! I'm a $this->name!";
19.    }
20.}
21.class Citroen extends Car {
22.    public function intro() : string {
23.        return "French extravagance! I'm a $this->name!";
24.    }
25.}
26.// Create objects from the child classes
27.$audi = new audi("Audi");
28.echo $audi->intro() . "<br>";
29.$volvo = new volvo("Volvo");
30.echo $volvo->intro() . "<br>";
```



```

31.$citroen = new citroen("Citroen");
32.echo $citroen->intro();
33.??>

```

Các lớp Audi, Volvo và Citroen được kế thừa từ lớp Car. Điều này có nghĩa là các lớp Audi, Volvo và Citroen có thể sử dụng thuộc tính public \$name cũng như phương thức public __construct() từ lớp Car do tính kế thừa. Tuy nhiên, intro() là một phương thức trừu tượng nên được định nghĩa trong tất cả các lớp con và chúng sẽ trả về một chuỗi.

3.6.2.10. Phương thức tĩnh

Các phương thức tĩnh (Static methods) có thể được gọi trực tiếp mà không cần tạo một thể hiện (đối tượng) của lớp đó. Các phương thức tĩnh được khai báo với từ khóa *static*.

Ví dụ:

```

1. <?php
2. class ClassName {
3.     public static function staticMethod() {
4.         echo "Hello World!";
5.     }
6. }
7. ??>

```

Để truy cập một phương thức tĩnh ta sử dụng tên lớp, dấu hai chấm (::) và tên phương thức.

Ví dụ:

```

1. <?php
2. class greeting {
3.     public static function welcome() {
4.         echo "Hello World!";
5.     }
6. }
7. // Call static method
8. greeting::welcome();
9. ??>

```

3.6.2.11. Thuộc tính tĩnh

Các thuộc tính tĩnh (static properties) có thể được gọi trực tiếp mà không cần tạo một thể hiện của một lớp. Thuộc tính tĩnh được khai báo với từ khóa *static*.

Ví dụ:

```

1. <?php
2. class ClassName {
3.     public static $staticProp = "SaoDo";

```

```
4. }  
5. ?>
```

Để truy cập một thuộc tính tĩnh, hãy sử dụng tên lớp, dấu hai chấm (::) và tên thuộc tính.

Ví dụ:

```
1. <?php  
2. class pi {  
3.     public static $value = 3.14159;  
4. }  
5. // Get static property  
6. echo pi::$value;  
7. ?>
```

3.7. Làm việc với Session và Cookie

3.7.1. Session

Session (phiên) là một cách để lưu trữ thông tin (trong các biến – biến *session*) sẽ được sử dụng trên nhiều trang. Không giống như cookie, thông tin không được lưu trữ trên máy tính của người dùng.

Khi sử dụng một ứng dụng web, thực hiện một số thay đổi, rồi đóng nó. Điều này giống như một phiên làm việc. Máy tính sẽ biết người dùng là ai, biết khi nào người dùng bắt đầu sử dụng ứng dụng và khi nào người dùng kết thúc. Nhưng trên internet có một vấn đề: máy chủ web không biết người dùng là ai hoặc người dùng làm gì, vì địa chỉ HTTP không duy trì trạng thái.

Biến *session* giải quyết vấn đề này bằng cách lưu trữ thông tin người dùng sẽ được sử dụng trên nhiều trang (ví dụ: tên người dùng, màu yêu thích,...). Theo mặc định, các biến *session* có chu kỳ sống kéo dài cho đến khi người dùng đóng trình duyệt. Vì thế, các biến *session* chứa thông tin về một người dùng và có sẵn cho tất cả các trang trong một ứng dụng. Biến *session* sẽ mất đi khi đóng trình duyệt.

Lưu ý: Nếu cần lưu trữ dữ liệu vĩnh viễn, nên lưu trữ dữ liệu trong cơ sở dữ liệu.

a. Khởi tạo session

Một *session* được khởi tạo với hàm `session_start()`. Các biến *session* được đặt trong biến toàn cục PHP: `$_SESSION`.

Ví dụ: Tạo trang mới có tên "demo_session1.php". Trong trang này, khởi tạo một session và đặt một số biến session.

```
1. <?php  
2. // Start the session  
3. session_start();  
4. ?>  
5. <!DOCTYPE html>  
6. <html>
```

```

7. <body>
8. <?php
9. // Set session variables
10.$_SESSION["favcolor"] = "green";
11.$_SESSION["favanimal"] = "cat";
12.echo "Session variables are set.";
13.?'>
14.</body>
15.</html>

```

Lưu ý: Hàm `session_start()` phải được đặt ở dòng đầu tiên trong trang web. Trước bất kỳ thẻ HTML nào.

b. Truy cập các giá trị biến session

Để nhận giá trị của biến session, sử dụng cú pháp:

```
$_SESSION["tên biến"];
```

Ví dụ, tạo một trang khác gọi là "demo_session2.php". Từ trang này truy cập thông tin session đã đặt trên trang ("demo_session1.php").

Lưu ý:

- Các biến *session* không được truyền riêng lẻ cho từng trang, thay vào đó, chúng được truy xuất từ *session* chúng ta khởi tạo ở đầu mỗi trang (`session_start()`).

- Tất cả các giá trị của biến *session* được lưu trữ trong biến toàn cục `$_SESSION`.

Ví dụ:

```

1. <?php
2. session_start();
3. ?>
4. <!DOCTYPE html>
5. <html>
6. <body>
7. <?php
8. // Echo session variables that were set on previous page
9. echo "Favorite color is " . $_SESSION["favcolor"] . "<br>";
10.echo "Favorite animal is " . $_SESSION["favanimal"] . ".";
11.?'>
12.
13.</body>
14.</html>

```

Để thay đổi một biến session, chỉ cần ghi đè giá trị:

```
$_SESSION["tên biến"] = "giá trị mới";
```

c. Hủy session

Để xóa (hủy) tất cả các biến *session* chung và hủy *session*, hãy sử dụng hàm `session_unset()` và `session_destroy()`:

```
1. <?php
2. // remove all session variables
3. session_unset();
4. // destroy the session
5. session_destroy();
6. ?>
```

3.7.2. Cookie

Cookie thường được sử dụng để xác định người dùng. *Cookie* là một tệp nhỏ mà máy chủ nhúng vào máy tính của người dùng. Mỗi khi cùng một máy tính yêu cầu một trang bằng trình duyệt, nó cũng sẽ gửi *cookie*. Với PHP ta có thể tạo và truy xuất các giá trị cookie.

a. Tạo Cookie

Cookie được tạo bằng hàm `setcookie()`.

Cú pháp:

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

Ví dụ sau tạo cookie có tên "user" với giá trị "John Doe". Cookie sẽ hết hạn sau 30 ngày ($86400 * 30$). "/" có nghĩa là cookie có sẵn trong toàn bộ trang web (nếu không, hãy chọn thư mục muốn lưu trữ).

Sau đó, truy xuất giá trị của cookie "user" (sử dụng biến toàn cục `$_COOKIE`). Cũng sử dụng chức năng `isset()` để tìm hiểu xem cookie có được đặt hay không:

```
1. <?php
2. $cookie_name = "user";
3. $cookie_value = "John Doe";
4. setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/");
   // 86400 = 1 day
5. ?>
6. <html>
7. <body>
8. <?php
9. if(!isset($_COOKIE[$cookie_name])) {
10.  echo "Cookie named '" . $cookie_name . "' is not set!";
11.} else {
12.  echo "Cookie '" . $cookie_name . "' is set!<br>";
13.  echo "Value is: " . $_COOKIE[$cookie_name];
14.}
15. ?>
```

```
16.</body>
```

```
17.</html>
```

Lưu ý: Hàm `setcookie()` phải xuất hiện TRƯỚC thẻ `<html>`.

b. Sửa đổi giá trị cookie

Để sửa đổi cookie, chỉ cần đặt (lại) cookie bằng hàm `setcookie()`.

c. Xóa Cookie

Để xóa cookie, sử dụng hàm `setcookie()` với ngày hết hạn trong quá khứ.

Ví dụ:

```
1. <?php
2. // set the expiration date to one hour ago
3. setcookie("user", "", time() - 3600);
4. ?>
5. <html>
6. <body>
7. <?php
8. echo "Cookie 'user' is deleted.";
9. ?>
10.</body>
11.</html>
```

d. Kiểm tra xem Cookie đã được bật chưa

Ví dụ sau tạo một tập lệnh nhỏ để kiểm tra xem cookie có được bật hay không. Đầu tiên, hãy thử tạo một cookie thử nghiệm với hàm `setcookie()`, sau đó đếm biến mảng `$_COOKIE`.

```
1. <?php
2. setcookie("test_cookie", "test", time() + 3600, '/');
3. ?>
4. <html>
5. <body>
6. <?php
7. if(count($_COOKIE) > 0) {
8.     echo "Cookies are enabled.";
9. } else {
10.    echo "Cookies are disabled.";
11. }
12. ?>
13.</body>
14.</html>
```

CÂU HỎI ÔN TẬP CHƯƠNG 3

- Câu 1. So sánh sự giống và khác nhau của các phạm vi biến trong PHP.
- Câu 2. Trình bày các phương pháp truyền biến giữa các trang web.
- Câu 3. So sánh sự giống và khác nhau giữa các phương pháp sử dụng nhúng file vào trang web (include, include_once, require, require_once).
- Câu 4. Trình bày cú pháp một số hàm thường dùng đối với kiểu dữ liệu chuỗi ký tự trong PHP.
- Câu 5. Trình bày cách khai báo, truy xuất mảng chỉ số. Lấy ví dụ.
- Câu 6. Trình bày cách khai báo, truy xuất mảng kết hợp. Lấy ví dụ.
- Câu 7. Trình bày cách khai báo, truy xuất mảng hai chiều. Lấy ví dụ.
- Câu 8. Trình bày cú pháp, ý nghĩa câu lệnh If ... Else ... ElseIf. Lấy ví dụ.
- Câu 9. Trình bày cú pháp, ý nghĩa câu lệnh Switch. Lấy ví dụ.
- Câu 10. Trình bày cú pháp, ý nghĩa câu lệnh For. Lấy ví dụ.
- Câu 11. Trình bày cú pháp, ý nghĩa câu lệnh Foreach. Lấy ví dụ.
- Câu 12. Trình bày cú pháp, ý nghĩa câu lệnh While. Lấy ví dụ.
- Câu 13. Trình bày cú pháp, ý nghĩa câu lệnh Do.. While. Lấy ví dụ.
- Câu 14. Trình bày các bước thực hiện thao tác với cơ sở dữ liệu MySQL. Cho ví dụ minh họa.
- Câu 15. Minh họa cách xây dựng và sử dụng hàm (function) trong PHP.
- Câu 16. Minh họa phương pháp xây dựng và sử dụng lớp (class) và đối tượng (object) trong PHP.
- Câu 17. Trình bày các tính chất của lớp (class) trong PHP.
- Câu 18. Trình bày về Session trong PHP. Ý nghĩa và cách sử dụng. Lấy ví dụ.
- Câu 19. Trình bày về Cookie trong PHP. Ý nghĩa và cách sử dụng. Lấy ví dụ.
- Câu 20. So sánh ưu, nhược điểm khi sử dụng Session và Cookie.
- Câu 21. Lập trình hệ thống website thương mại điện tử bán hàng với các yêu cầu sau:
- Xây dựng các chức năng chính của hệ thống website.
 - Thiết kế giao diện người dùng, giao diện quản trị hệ thống.
 - Phân tích, thiết kế cơ sở dữ liệu.
 - Lập trình các trang quản trị, thực hiện quản trị cơ sở dữ liệu hệ thống. Yêu cầu kiểm tra đăng nhập hệ thống, kiểm tra tính phù hợp của dữ liệu nhập.
 - Lập trình các trang giao diện người dùng để giới thiệu sản phẩm, xem thông tin chi tiết sản phẩm, chọn mua và xem giỏ hàng sản phẩm, gửi thông tin đăng ký mua hàng của người dùng ...

Câu 22. Lập trình hệ thống website quản lý nhân sự với các yêu cầu sau:

- Xây dựng các chức năng chính của hệ thống website.
- Thiết kế giao diện người dùng, giao diện quản trị hệ thống.
- Phân tích, thiết kế cơ sở dữ liệu.
- Lập trình các trang quản trị, thực hiện quản trị cơ sở dữ liệu hệ thống. Yêu cầu kiểm tra đăng nhập hệ thống, kiểm tra tính phù hợp của dữ liệu nhập.
- Lập trình các trang giao diện người dùng để xem thông tin, tìm kiếm thông tin và một số yêu cầu tổng hợp khác đối với nghiệp vụ quản lý nhân sự.

Câu 23. Lập trình hệ thống website tin tức với các yêu cầu sau:

- Xây dựng các chức năng chính của hệ thống website.
- Thiết kế giao diện người dùng, giao diện quản trị hệ thống.
- Phân tích, thiết kế cơ sở dữ liệu.
- Lập trình các trang quản trị, thực hiện quản trị cơ sở dữ liệu hệ thống. Yêu cầu kiểm tra đăng nhập hệ thống, kiểm tra tính phù hợp của dữ liệu nhập.
- Lập trình các trang giao diện người dùng để xem thông tin, tìm kiếm thông tin và một số yêu cầu khác đối với một website tin tức.

Chương 4. MÔ HÌNH MVC

4.1. Giới thiệu mô hình MVC

Như đã trình bày trong chương 1, MVC là một mô hình thiết kế, giúp tổ chức code theo từng phần độc lập với nhau, và các phần tương tác với nhau theo một cách nhất định. Để đi sâu nghiên cứu mô hình này, ta sẽ lập trình một dự án web nhỏ có tương tác với cơ sở dữ liệu. Giả thiết dự án web thực hiện quản lý thông tin phản hồi (bài viết) của khách hàng theo mô hình MVC.

Trước khi lập trình, ta thiết lập thư mục lưu trữ web theo cấu trúc sau:

```
- demo_mvc
  |-- assets
  |   |-- fonts
  |   |-- images
  |   |-- javascripts
  |   |-- stylesheets
  |-- controllers
  |-- models
  |-- views
  |   |-- layouts
  |       |-- application.php
  |-- connection.php
  |-- index.php
  |-- routes.php
```

Hình 4.1. Cấu trúc thư mục web

Giải thích về cấu trúc thư mục trên:

- Thư mục demo_mvc là thư mục chứa project.
- Thư mục assets gồm các file font chữ, hình ảnh, javascript, css,...
- Thư mục controllers chứa các file định nghĩa các lớp controller, có các hàm trong đó tương tác với model và gọi ra view để trả về cho người dùng.
- Thư mục models chứa các file định nghĩa các lớp model, chịu trách nhiệm thao tác với CSDL.
- Thư mục views chứa thư mục layouts, layouts chứa template hiển thị chung của trang web trong file application.php
- ...

Tạo một cơ sở dữ liệu đơn giản có tên demo_mvc và có bảng posts với 3 trường: id (INT PRIMARY auto_increment), title (VARCHAR 255), content (TEXT).

Tiếp theo, tạo file index.php với nội dung như sau:

```
1. # index.php
2. <?php
3. require_once('connection.php');
4. if (isset($_GET['controller'])) {
```



```

5. $controller = $_GET['controller'];
6. if (isset($_GET['action'])) {
7.     $action = $_GET['action'];
8. } else {
9.     $action = 'index';
10. }
11.} else {
12. $controller = 'pages'; //Mặc định module pages được gọi đến.
13. $action = 'home';      //Mặc định action home được gọi đến.
14.}
15.require_once('routes.php'); //Nhúng routes để xử lý gọi controller
16.                             //phù hợp với yêu cầu của client.

```

File này sẽ nhận mọi yêu cầu truy vấn từ client để gửi lên server. Bởi vậy, mọi đường dẫn truy cập đều phải có dạng `/?param=value` hoặc `/index.php?param=value`. Param là các biến truyền qua URL. Trong dự án này sử dụng 2 biến chính có tên controller và action để xác định Controller nào được gọi và Action của nó.

Trong file index.php chạy nội dung file connection.php (dòng lệnh `require_once('connection.php')`) được dùng để kết nối và truy vấn đến cơ sở dữ liệu, sử dụng PDO.

File connection.php

```

1. # connection.php
2. <?php
3. class DB
4. {
5.     private static $instance = NULL;
6.     public static function getInstance() {
7.         if (!isset(self::$instance)) {
8.             try {
9.                 self::$instance =
10.                 new PDO('mysql:host=localhost;dbname=demo_mvc', 'root', '');
11.                 self::$instance->exec("SET NAMES 'utf8'");
12.             } catch (PDOException $ex) {
13.                 die($ex->getMessage());
14.             }
15.             return self::$instance;
16.         }
17. }

```

Cần chỉnh sửa lại phần mã lệnh `new PDO('mysql:host=<host name>;dbname=<database name>', '<username>', '<password>')` sao cho trùng với

thông tin kết nối tới CSDL của dự án. Sau khi chạy file connection.php, file index.php sẽ xử lý các tham số của đường dẫn, cụ thể là lấy ra 2 tham số *controller* và *action*, lưu giá trị của chúng vào các biến để sau này dùng cho việc quyết định sẽ làm việc gì hay hiển thị nội dung gì,... Mặc định nếu không có các tham số này thì chúng sẽ được gán giá trị là *controller* và trỏ đến *pages* (module pages), còn *action* thì trỏ đến *home*.

File routes.php sẽ chịu trách nhiệm phân tích 2 biến mà vừa lấy được ở bước trên sau đó xác định phần *view* nào sẽ được hiển thị.

```
1. # routes.php
2. <?php
3. $controllers = array(
4.     'pages' => ['home', 'error'],
5. );//Các controllers, action trong hệ thống
   có thể gọi ra từ controller đó.
   Mỗi controllers tương ứng với một module.
6.
7. //Nếu các tham số controller và action nhận được từ URL không hợp lệ
   (không thuộc danh sách controller và action khai báo trên thì
   trang báo lỗi sẽ được gọi ra.
8. if (!array_key_exists($controller, $controllers) ||
   !in_array($action, $controllers[$controller])) {
9.     $controller = 'pages';
10.    $action = 'error';
11.}
12.
13.//Nhúng file định nghĩa controller vào để có thể dùng được class định
   nghĩa trong file đó
14.include_once('controllers/' . $controller . '_controller.php');
15.//Tạo ra tên lớp controller từ các giá trị lấy được từ URL sau đó gọi
   ra để hiển thị trả về cho người dùng.
16.//Ví dụ $controller='pages' sẽ tạo ra tên lớp controller là PagesController
17.$klass = str_replace('_', '', ucwords($controller, '_')) . 'Controller';
18.$controller = new $klass;//Tạo đối tượng controller từ lớp controller
   tương ứng được khai báo trong file tên module_controller.php.
   Ví dụ file pages_controller.php.
19.$controller->$action();
   //Gọi hành động được chỉ định của đối tượng controller.
```

* Giao diện dùng chung

Mỗi một *action* sẽ gọi đến một view để hiển thị giao diện cho người dùng. Tuy nhiên trong một dự án, để có thể tạo ra các giao diện người dùng nhất quán thì cần sử dụng đến các layout. Có thể chia nhỏ layout thành các phần (part) rồi nhúng (include) trở lại view tương ứng (ví dụ: header, footer, main,...).

Tạo file application.php trong thư mục views/layouts với nội dung như sau:

```

1. # views/layouts/application.php
2.
3. <DOCTYPE html>
4. <html>
5.   <head>
6.     <meta charset="utf-8">
7.     <meta http-equiv="X-UA-Compatible" content="IE=edge">
8.     <meta content="width=device-width, initial-scale=1, maximum-
       scale=1, user-scalable=no" name="viewport">
9.     <title>Demo PHP MVC</title>
10.  </head>
11.  <body>
12.    <?= @$content ?>
13.  </body>
14.</html>

```

Trong mã trên, đoạn mã `<?= @$content ?>` tương tự `<?php echo $content; ?>` dùng để hiển thị giá trị của biến `content`. Giá trị của biến `content` sẽ được controller gọi hàm `render` (kế thừa trong lớp `BaseController`) để gán giá trị.

4.2. Tầng model

Tầng Model chịu trách nhiệm tương tác với cơ sở dữ liệu. Mỗi một bảng dữ liệu nên có một model tương ứng để xử lý dữ liệu của bảng, ngoài ra cũng có thể có các model thực hiện xử lý các nghiệp vụ khác.

Có thể đưa file `connection.php` vào trong tầng này để nhất quán về nguyên tắc phân chia trách nhiệm giữa các tầng theo mô hình MVC.

Trong dự án này, bảng `posts` trong cơ sở dữ liệu dùng để lưu thông tin bài viết phản hồi của khách hàng. Vì vậy cần xây dựng một model cho bảng này gọi là model `Post`. Hãy tạo file `post.php` trong thư mục `models` với nội dung sau:

```

1. # models/post.php
2. <?php
3. class Post
4. {
5.   //Khai báo các thuộc tính tương ứng với các trường của bảng trong CSDL.
6.   public $id;
7.   public $title;
8.   public $content;
9.
10.  //Phương thức khởi tạo dùng để gán giá trị truyền vào cho các thuộc
    tính mỗi khi tạo mới đối tượng của lớp Post.
11.  function __construct($id, $title, $content)
12.  {

```

```

13.     $this->id = $id;
14.     $this->title = $title;
15.     $this->content = $content;
16. }
17.
18. // Phương thức all() dùng để lấy toàn bộ dữ liệu của bảng posts.
19. static function all()
20. {
21.     $list = [];
22.     $db = DB::getInstance();
23.     $req = $db->query('SELECT * FROM posts');
24.
25.     foreach ($req->fetchAll() as $item) {
26.         $list[] = new Post($item['id'], $item['title'], $item['content']);
27.     }
28.     return $list;
29. }
30.
31. // Phương thức find() dùng để lấy thông tin 1 dòng theo id.
32. static function find($id)
33. {
34.     $db = DB::getInstance();
35.     $req = $db->prepare('SELECT * FROM posts WHERE id = :id');
36.     $req->execute(array('id' => $id));
37.
38.     $item = $req->fetch();
39.     if (isset($item['id'])) {
40.         return new Post($item['id'], $item['title'], $item['content']);
41.     }
42.     return null;
43. }
44.}

```

Trong đoạn mã trên gọi hàm *prepare* để chuẩn bị một câu lệnh SQL làm khung/mẫu được gọi là Prepared Statement với các Placeholder (gọi là tham số của các phương thức khi khai báo hàm).

Có 2 loại Placeholder trong Prepared Statement là Placeholder không định danh (Unnamed Placeholder) và Placeholder định danh (Named Placeholder).

Ví dụ:

```

1. $stmt = $conn->prepare('INSERT INTO users (name, email, age)
   values (?, ?, ?)');

```

```
2. $stmt = $conn->prepare('INSERT INTO users (name, email, age)
   values (:name, :mail, :age)');
```

Dòng lệnh thứ nhất sử dụng *Placeholder* không định danh là các dấu hỏi “?”. Dòng lệnh thứ 2 sử dụng *Placeholder* định danh: :name, :mail, :age (lưu ý dấu hai chấm và *Placeholder* không nhất thiết phải trùng với tên cột của bảng trong cơ sở dữ liệu).

Để truyền dữ liệu cho các *Placeholder* chúng ta có thể sử dụng hàm *bindParam()* hoặc mảng với lưu ý: nếu sử dụng *Placeholder* không định danh thì thứ tự sẽ tương ứng với các dấu ? trong câu lệnh, nếu dùng *Placeholder* định danh thì chỉ rõ tên định danh của *Placeholder*.

Ví dụ *Placeholder* không định danh:

```
1. $stmt = $conn->prepare('INSERT INTO users(name, email, age) values (?, ?, ?)');
2. //Gán các biến (lúc này chưa mang giá trị) vào các placeholder theo t
   hứ tự tương ứng
3. $stmt->bindParam(1, $name);
4. $stmt->bindParam(2, $mail);
5. $stmt->bindParam(3, $age);
6.
7. //Gán giá trị và thực thi
8. $name = "Nguyen Van A";
9. $mail = "nva@saodo.edu.vn";
10. $age = 22;
11. $stmt->execute();
12.
13. //Gán những giá trị khác và tiếp tục thực thi
14. $name = "Nguyen Van B";
15. $mail = "nvb@ saodo.edu.vn";
16. $age = 23;
17. $stmt->execute();
```

Hoặc :

```
1. $stmt = $conn->prepare('INSERT INTO users (name, email, age) values (?, ?, ?)');
2. $data = array('Nguyen Van A', 'nva@ saodo.edu.vn', 22);
3. //Phương thức execute() dưới đây sẽ gán lần lượt giá trị trong mảng
   vào các Placeholder theo thứ tự.
4. $stmt->execute($data);
```

Ví dụ *Placeholder* định danh:

Đối với *Placeholder* định danh, cách thực hiện cũng tương tự với *Placeholder* không định danh, chỉ khác là không dùng thứ tự *Placeholder* để gán giá trị (bind) mà dùng chính tên của *Placeholder*.

```

1. //Khởi tạo Prepared Statement từ biến $conn ở phần trước
2. $stmt = $conn->prepare('INSERT INTO users(name, email, age)
   values (:name, :mail, :age)');
3.
4. //Gán các biến (lúc này chưa mang giá trị) vào các placeholder theo t
   ên của chúng
5. $stmt->bindParam(':name', $name);
6. $stmt->bindParam(':mail', $mail);
7. $stmt->bindParam(':age', $age);
8.
9. //Gán giá trị và thực thi
10.$name = "Nguyen Van A";
11.$mail = "nva@saodo.edu.vn";
12.$age = 22;
13.$stmt->execute();

```

Hoặc:

```

1. //Lưu ý: Không cần thiết phải sử dụng dấu hai chấm cho các key
2. $data = array('name' => 'Nguyen Van A', 'mail' => 'nva@saodo.edu.vn',
   'age' => 22);

```

Cũng trong đoạn mã trên sử dụng hàm fetchAll() và fetch() của PDO. Hàm fetchAll – lấy ra tất cả các dòng dữ liệu, kết quả là 1 array. Hàm fetch – lấy ra 1 dòng dữ liệu, kết quả là 1 array.

4.3. Tầng controller

4.3.1. Xây dựng BaseController

Trong dự án này, trước khi controller gửi dữ liệu sang cho view rồi gọi view đó hiển thị giao diện cho người dùng, cần gọi đến phương thức render của controller dùng chung (lớp BaseController) được kế thừa để chỉ định view nào nhận dữ liệu và hiển thị thông tin.

Tạo 1 lớp BaseController để làm lớp cha cho các controller của hệ thống. Khi đó, có thể định nghĩa các hàm mà mọi controller đều có thể gọi ra mà không phải định nghĩa lại ở mỗi controller. Tạo file base_controller.php trong thư mục controllers.

```

1. # controllers/base_controller.php
2. <?php
3. class BaseController
4. {
5.     protected $folder;//Biến có giá trị là thư mục nào đó trong thư mục
       views, chứa các file view template của module đang truy cập.
6.
7.     //Hàm hiển thị kết quả ra cho người dùng. Biến file lưu tên file của
       view, mảng data lưu thông tin dữ liệu truyền cho view.
8.     function render($file, $data = array())

```

```

9.  {
10.    //Kiểm tra file gọi đến có tồn tại hay không?
11.    $view_file = 'views/' . $this->folder . '/' . $file . '.php';
12.    if (is_file($view_file)) {
13.        //Nếu tồn tại file đó thì tạo ra các biến chứa giá trị truyền vào
        lúc gọi hàm. Các khóa mảng làm tên biến và giá trị là giá trị biến.
14.        //$data là một mảng kết hợp (mảng có khóa và giá trị) nên sử dụng
        hàm extract để tạo các biến mới từ khóa của mảng và giá trị của mảng.
15.        extract($data);
16.        //Sau đó lưu giá trị trả về khi chạy file view template với các
        dữ liệu đó vào 1 biến mà chưa hiển thị luôn ra trình duyệt.
17.        ob_start(); // Tạo bộ đệm đầu ra.
18.        require_once($view_file);
19.        $content = ob_get_clean(); //trả về nội dung của bộ đệm đầu ra
        và sau đó xóa nội dung khỏi bộ đệm.
20.        // Sau khi có kết quả đã được lưu vào biến $content, gọi ra tem
        plate chung của hệ thống để hiển thị ra cho người dùng.
21.        require_once('views/layouts/application.php');
22.    } else {
23.        //Nếu file muốn gọi ra không tồn tại thì chuyển hướng đến trang
        báo lỗi.
24.        header('Location: index.php?controller=pages&action=error');
25.    }
26. }
27.}

```

4.3.2. Xây dựng *PagesController*

PagesController là lớp xử lý điều khiển cho module *Pages*, đặt trong file *pages_controller.php* lưu trong thư mục *controllers*.

```

1. # controllers/pages_controller.php
2.
3. <?php
4. require_once('controllers/base_controller.php');
5.
6. //Tạo lớp PagesController kế thừa từ lớp BaseController
7. class PagesController extends BaseController
8. {
9.     function __construct()
10.    {
11.        $this->folder = 'pages';
        //Chỉ định thư mục pages là thư mục lưu các file của view trong module Pages.
12.    }
13.

```

```

14. public function home()
15. {
16.     $data = array(
17.         'name' => 'Nguyễn Văn A',
18.         'age' => 22
19.     );
20.     $this->render('home', $data);
21. }
22.
23. public function error()
24. {
25.     $this->render('error');
26. }
27.}

```

4.3.3. Xây dựng PostsControllers

Tương ứng với module Posts, tạo file posts_controller.php trong thư mục controllers để xử lý các điều khiển của module này.

```

1. # controllers/posts_controller.php
2.
3. <?php
4. require_once('controllers/base_controller.php');
5. require_once('models/post.php');
6.
7. class PostsController extends BaseController
8. {
9.     function __construct()
10.    {
11.        $this->folder = 'posts';
12.        //Chỉ định thư mục posts là thư mục chứa các file view.
13.    }
14.    public function index()
15.    {
16.        $posts = Post::all(); //Gọi phương thức all() của model Post
17.        //lấy tất cả dữ liệu từ bảng.
18.        $data = array('posts' => $posts);
19.        $this->render('index', $data); //Gọi hàm render được kế thừa
20.    }
21.
22.    public function showPost()
23.    {

```



```

24.     $post = Post::find($_GET['id']); //Gọi phương thức find() của model
25.                                     //Post lấy dữ liệu có id từ bảng.
26.     $data = array('post' => $post);
27.     $this->render('show', $data);
28. }
29.
30.}

```

4.4. Tầng view

4.4.1. Module Pages

Trong thư mục views, tạo thư mục pages chứa 2 file home.php và error.php với nội dung như sau:

```

1. # views/pages/home.php
2. <?php
3.     echo "Tên tôi là: $name, năm nay tôi $age tuổi";
4.     //Giá trị biến $name và $age được truyền từ controller sang.
5. ?>
6.
7. # views/pages/error.php
8. <?php
9.     echo 'Có lỗi xảy ra!';
10. ?>

```

4.4.2. Module Posts

Module Posts có hai hành động là index và showPost tương ứng chúng ta tạo ra hai view cho các hành động này.

Tạo thư mục posts trong thư mục views, sau đó tạo file index.php với nội dung:

```

1. # views/posts/index.php
2. <?php
3. echo '<ul>';
4. //Mảng $posts được controller truyền sang.
   Duyệt mảng để hiển thị từng dòng thông tin.
5. foreach ($posts as $post) {
6.     echo '<li>
7.         <a href="index.php?controller=posts&action=showPost&id=' .
           $post->id . '">'. $post->title . '</a>
8.     </li>';
9.     //Thẻ a chỉ rõ hành động showPost khi người dùng kích chọn vào bài
       viết.
10.}
11.echo '</ul>';
12. ?>

```

Khi truy cập địa chỉ `/index.php?controller=posts` thì trình duyệt sẽ hiển thị trang báo lỗi. Cần bổ sung controller posts và các action tương ứng của module vào file `routes.php`

Tìm đến đoạn code và bổ sung như sau:

```
$controllers = array(
    'pages' => ['home', 'error'],
    'posts' => ['index', 'showPost'], // bổ sung thêm, chú ý dấu
phẩy “,” cuối mỗi dòng.
);
```

Để chạy thử module Posts, truy cập địa chỉ `/index.php?controller=posts`. Lưu ý, cần thêm dữ liệu cho bảng posts trong cơ sở dữ liệu để hiển thị kết quả.

Tiếp theo, tạo view cho hành động `showPost`. Tạo file `show.php` trong thư mục `views/posts` với nội dung:

```
1. # views/posts/show.php
2.
3. <?php
4.     echo "Tiêu đề: $post->title";
5.     echo "<br>";
6.     echo "Nội dung: $post->content";
7. ?>
```

Truy cập vào địa chỉ `/index.php?controller=posts` để kiểm tra kết quả thực hiện.

CÂU HỎI ÔN TẬP CHƯƠNG 4

Câu 1. Trình bày mô hình, vai trò các thành phần và cách thức hoạt động của mô hình MVC.

Câu 2. Phân tích từng dòng lệnh trong file kết nối cơ sở dữ liệu.

Câu 3. Phân tích từng dòng lệnh trong file phân tích luồng routes.

Câu 4. Phân tích từng dòng lệnh trong các file của thành phần Model.

Câu 5. Phân tích từng dòng lệnh trong các file của thành phần Controller.

Câu 6. Phân tích từng dòng lệnh trong các file của thành phần View.

Câu 7. Trong dự án của chương 4 sử dụng giao diện dùng chung chưa có các tùy chỉnh cho giao diện như CSS, JS,... Hãy bổ sung các thành phần này để giao diện thân thiện với người dùng (ví dụ: sử dụng template bootstrap).

Câu 8. Trong giao diện hệ thống thông thường sẽ có thành phần menu động. Hãy phân tích để bổ sung thành phần menu động (ví dụ: danh sách các nhóm thông tin) cho dự án.

Câu 9. Phát triển dự án bằng cách bổ sung các module quản lý nhóm sản phẩm, sản phẩm.

PHẦN 2. THỰC HÀNH

Bài 1. Cài đặt Xampp và cấu hình tên miền ảo

- Tải về Xampp tại địa chỉ: <https://www.apachefriends.org/download.html>
- Cài đặt Xampp.
- Cấu hình tên miền ảo cho dự án web: laptop247.saodo.edu.vn.

Bài 2. Quản trị cơ sở dữ liệu trong phpMyAdmin

1. Để quản lý Đồ án/khóa luận tốt nghiệp của sinh viên, người ta xây dựng một cơ sở dữ liệu có tên là DoAnKhoaLuan gồm các sơ đồ quan hệ sau:

- Khoa (makhoa char(10), tenkhoa char(30), dienthoai char(10)).
- GiangVien (magv int, hotengv char(30), luong decimal(5,2), makhoa char(10)).
- SinhVien (masv int, hotensv char(30), makhoa char(10), namsinh int, quequan char(30)).

- DeTai (madt char(10), tendt char(30), kinhphi int, NoiThucTap char(30)).

- HuongDan (masv int, madt char(10), magv int, ketqua decimal(5,2)).

a. Xây dựng cơ sở dữ liệu MySQL trên.

b. Tạo quan hệ cho các bảng trong cơ sở dữ liệu.

c. Nhập thông tin demo cho các bảng, mỗi bảng tối thiểu 5 bản ghi.

d. Sử dụng lệnh SQL trả lời các yêu cầu sau:

- Đưa ra thông tin gồm mã số, họ tên và tên khoa của tất cả các giảng viên.
- Đưa ra thông tin gồm mã số, họ tên và tên khoa của các giảng viên của một khoa cụ thể nào đó. Ví dụ khoa CNTT.
- Cho biết số sinh viên thực hiện đề tài của một khoa cụ thể nào đó.
- Đưa ra danh sách gồm mã số, họ tên và tuổi của các sinh viên của một khóa cụ thể nào đó.
- Cho biết số giảng viên của một khoa cụ thể nào đó.
- Đưa ra mã khoa, tên khoa và số giảng viên của mỗi khoa
- Cho biết số điện thoại của khoa mà sinh viên có tên ‘ten_sinh_vien’ thực hiện đề tài.
- Cho biết mã số và tên của các đề tài do giảng viên ‘ten_giang_vien’ hướng dẫn đề tài.
- Cho biết mã số, họ tên, tên khoa của các giảng viên hướng dẫn từ 3 sinh viên trở lên.
- Cho biết mã số, tên đề tài của đề tài có kinh phí cao nhất.
- Cho biết mã số và tên các đề tài có nhiều hơn 2 sinh viên thực hiện.
- Đưa ra mã số, họ tên và điểm của các sinh viên một khoa cụ thể nào đó.

- Đưa ra tên khoa, số lượng sinh viên của mỗi khoa.
- Hãy cho biết thông tin về những sinh viên chưa có điểm.
- Đưa ra danh sách gồm mã số, họ tên các sinh viên có điểm thực tập bằng 0.

2. Tạo cơ sở dữ liệu có tên laptop247 cho dự án web. Tạo các bảng dữ liệu sau:

- Taikhoan (tendangnhap varchar(100) PRIMARY KEY NOT NULL, matkhau varchar(50) NOT NULL, hoten varchar(100) NULL, email varchar(100) NULL).

- Nhomsanpham (nhomid int AUTOINCREMENT PRIMARY, tennhom varchar(100) NOT NULL).

- Sanpham (id int AUTOINCREMENT PRIMARY, tensp varchar(100) NOT NULL, mota text NULL, dongia int DEFAULT 0, anh varchar(100) NULL, hot boolean DEFAULT 0, nhomid int,...).

- Dathang (id int AUTOINCREMENT PRIMARY, hoten varchar(100) NOT NULL, dienthoai varchar(10) NOT NULL, email varchar(100) NOT NULL, diachi varchar(250) NULL, sanphamid int, soluong int, ngayOder datetime DEFAULT CURRENT_TIMESTAMP, ghichu text NULL).

a. Tạo quan hệ cho các bảng trên.



Hình b.1. Mô hình quan hệ giữa các bảng

b. Sử dụng phpMySQL thực hiện: Thêm, sửa, xóa dữ liệu cho các bảng. Lọc (truy vấn) dữ liệu theo các yêu cầu phức tạp có sử dụng mệnh đề Where, Group By, Join, Distinct, Order By ...

Bài 3. Lập trình các câu lệnh điều khiển

1. Thiết kế trang web tính diện tích hình chữ nhật sau:

Hình b.2. Form tính diện tích hình chữ nhật

Yêu cầu:

Stt	Đối tượng	Yêu cầu	Ghi chú
1	Form	- Đặt tên form. - Thiết lập phương thức của form Post. - Thiết lập action của form là tên trang.	
2	Điều khiển	- Sử dụng điều khiển Text và Button - Điều khiển Diện tích không cho phép nhập và chỉnh sửa dữ liệu.	
3	Nút lệnh “Tính”	Khi kích chọn, tính: - Diện tích = Chiều dài * Chiều rộng. - Nếu chiều dài hoặc chiều rộng ≤ 0 thì hiển thị thông báo lỗi.	

2. Thiết kế trang web tính toán tiền điện:

Hình b.3. Form thanh toán tiền điện

Yêu cầu:

Stt	Đối tượng	Yêu cầu	Ghi chú
1	Form	- Đặt tên form. - Thiết lập phương thức cho form là Post. - Thiết lập action của form là tên trang.	
2	Điều khiển	- Sử dụng điều khiển Text và Button - Điều khiển Số tiền thanh toán không cho phép nhập và chỉnh sửa dữ liệu. - Giá trị mặc định của đơn giá là 2000.	
3	Nút lệnh “Tính”	Khi kích chọn, tính: - Số tiền thanh toán = (Chỉ số mới – Chỉ số cũ) * Đơn giá. - Nếu chỉ số mới < chỉ số cũ hoặc đơn giá ≤ 0 thì hiển thị thông báo lỗi.	

3. Thiết kế trang web đổi ngoại tệ sau:

Hình b.4. Form đổi ngoại tệ

Yêu cầu:

Stt	Đối tượng	Yêu cầu	Ghi chú
1	Form	- Đặt tên form. - Thiết lập phương thức cho form là Post. - Thiết lập action của form là đường dẫn đến trang doi_ngoai_te.php.	
2	Điều khiển	- Sử dụng điều khiển Text, Select, Radio và Button. - Các loại ngoại tệ trong Select: Đô la Mỹ - USD, Đô la Úc – AUD, Yên Nhật – YPJ, Euro - EUR. - Các loại vàng thuộc Radio: SJC, AAA, PNJ.	
3	Nút lệnh “Quy đổi ngoại tệ/vàng”	Khi kích chọn: - Gửi thông tin sang trang doi_ngoai_te.php. - Trang doi_ngoai_te.php thực hiện tính toán, sau đó gửi kết quả trả lại trang chứa form. - Trang chứa Form tải lại và hiển thị kết quả. - Sử dụng cấu trúc if để xử lý giá trị loại ngoại tệ nhận được. - Sử dụng cấu trúc switch để xử lý giá trị loại vàng nhận được.	

Bài 4. Lập trình với kiểu chuỗi ký tự và mảng

1. Chuyển đổi các chữ cái trong chuỗi thành chữ hoa hoặc chữ thường.
2. Viết PHP script chia một chuỗi.
3. Kiểm tra xem chuỗi đã cho có chứa một chuỗi nào đó không.
4. Chuyển đổi giá trị của một biến thành chuỗi.
5. Lấy username từ địa chỉ email đã cho.

6. Lấy 3 ký tự cuối của một chuỗi.
7. Thay thế từ đầu tiên trong chuỗi bằng từ khác.
8. Xóa một phần của chuỗi bắt đầu từ đầu chuỗi.
9. Chèn một chuỗi vào một vị trí bất kì của chuỗi khác.
10. Lấy từ đầu tiên của một câu.
11. Tạo mảng kết hợp hiển thị tên các quốc gia và thủ đô.
12. Lấy phần tử đầu tiên trong mảng.
13. Chèn phần tử vào mảng.
14. Sắp xếp mảng kết hợp trong PHP
15. Tìm giá trị trung bình của các phần tử mảng và hiển thị 5 phần tử nhỏ nhất và lớn nhất.
15. Nhập hai mảng trong PHP
16. Tìm độ dài nhất, ngắn nhất của các phần tử (dạng chuỗi) trong mảng.
17. Tìm giá trị nhỏ nhất trong mảng PHP
18. Sắp xếp mảng theo chiều đảo ngược (còn gọi là đảo mảng)
19. Lấy key của giá trị lớn nhất trong mảng kết hợp.
20. Tìm giá trị trong một mảng kết hợp

Bài 5. Lập trình tương tác với cơ sở dữ liệu MySQL

1. Sử dụng template để xây dựng giao diện phần quản trị và phần người dùng cho dự án website laptop247.saodo.edu.vn (đã được cấu hình tên miền ảo trong bài 1). Yêu cầu sử dụng cơ chế phân chia từng phần (header, footer ...) để tách các phần của giao diện.

2. Xây dựng file database.php chứa các hàm tương tác với cơ sở dữ liệu (đã được xây dựng trong bài 2) bao gồm: kết nối, thực thi câu lệnh SQL, chuyển dữ liệu trả về của truy vấn thành một mảng, lấy số dòng trả về của truy vấn ...

3. Sử dụng các hàm trên thực hiện:

- Lập trình quản trị tài khoản, nhóm sản phẩm, sản phẩm, đặt hàng trong cơ sở dữ liệu.

- Lập trình đăng nhập, đăng xuất tài khoản quản trị.

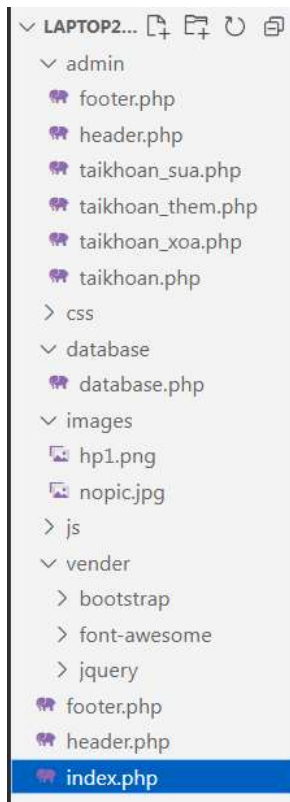
- Lập trình hiển thị danh sách, thông tin sản phẩm trên giao diện người dùng.

Yêu cầu có thể tìm kiếm theo nhóm sản phẩm.

- Lập trình giỏ hàng, đặt hàng của khách hàng.

Hướng dẫn:

Phần hướng dẫn này trình bày các thao tác cơ bản quản trị tài khoản và lập trình hiển thị giao diện người dùng (index.php - trang chủ website). Cấu trúc các file và thư mục sau khi xây dựng theo hướng dẫn như sau:



Hình b.5. Cấu trúc thư mục website.

* Tạo file tương tác cơ sở dữ liệu (database.php)

```

1. <?php
2. function ketnoi()
3. {
4.     try {
5.         $conn = new PDO("mysql:host=localhost;dbname=laptop247", 'root', '');
6.         // set the PDO error mode to exception
7.         $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
8.         //echo "Connected successfully";
9.     } catch(PDOException $e) {
10.        echo "Connection failed: " . $e->getMessage();
11.    }
12.    return $conn;
13.}
14.
15.//Hàm thực thi câu sql update, delete ...
16.function thucthisql($sql)
17.{
18.    $conn = ketnoi();
19.    return $conn->exec($sql);
20.}

```

```

21. //Hàm lấy dữ liệu của bảng, trả về một mảng (2 chiều)
22. function get_fetchall($sql)
23. {
24.     $conn = ketnoi();
25.     $result = $conn->query($sql)->fetchAll();
26.     $arr = array();
27.     foreach ($result as $row)
28.     {
29.         $arr[] = $row;
30.     }
31.     return $arr; //Trả về mảng các bản ghi
32. }
33. //Hàm lấy dữ liệu của bảng, trả về một mảng (1 chiều)
34. function get_fetch($sql)
35. {
36.     $conn = ketnoi();
37.     $result = $conn->query($sql);
38.     return $result->fetch();
39. }
40. ?>

```

* File quản lý tài khoản (taikhoan.php)

```

1. <?php include("header.php"); ?> //Nhúng file chứa phần đầu trang
2. <?php
3. //Nhúng file tương tác CSDL database.php. Nên đặt trong file header.php để
   các file có yêu cầu nhúng CSDL sau này không cần dòng lệnh nhúng này nữa.
4. include_once("../database/database.php");
5. $sql = "SELECT * FROM taikhoan";
6. //Thực thi câu truy vấn, sử dụng hàm get_fetchall() lấy dữ liệu trả
   về dạng mảng 2 chiều.
7. $results = get_fetchall($sql);
8. ?>
9. <div class="container">
10. <h2>Quản lý tài khoản</h2>
11. <table class="table table-bordered">
12.     <thead>
13.         <tr>
14.             <th></th>
15.             <th>Tên đăng nhập</th>
16.             <th>Mật khẩu</th>
17.             <th>Họ và tên</th>

```

```

18.         <th>Email</th>
19.     </tr>
20. </thead>
21. <tbody>
22. <?php foreach ($results as $row) { ?>
23.     <tr>
24.         <td>
25.             <a href="taikhoan_sua.php?tendangnhap=<?php echo
                $row['tendangnhap']; ?>">Sửa</a> |
26.             <a href="taikhoan_xoa.php?tendangnhap=<?php echo
                $row['tendangnhap']; ?>">Xóa</a>
27.         </td>
28.         <td><?php echo $row['tendangnhap']; ?></td>
29.         <td><?php echo $row['matkhau'] ?></td>
30.         <td><?php echo $row['hoten'] ?></td>
31.         <td><?php echo $row['email'] ?></td>
32.     </tr>
33. <?php } ?>
34. </tbody>
35. </table>
36. <a href="taikhoan_them.php?">Thêm mới</a>
37.</div>
38.<?php include("footer.php"); ?> //Nhúng file chứa phần cuối trang

```

Kết quả:

Quản lý tài khoản

	Tên đăng nhập	Mật khẩu	Họ và tên	Email
Sửa Xóa	admin	202cb962ac59075b964b07152d234b70	Phạm Văn Kiển	kienpvdesign@gmail.com
Sửa Xóa	kienpv	e10adc3949ba59abbe56e057f20f883e	Phạm Đăng Toàn	toanpv@gmail.com

[Thêm mới](#)

Hình b.6. Kết quả thực hiện

* File thêm mới tài khoản (taikhoan_them.php)

```

1. <?php include("header.php"); ?>
2. <?php
3. if (isset($_POST['add'])) {
4.     include_once("../database/database.php");
5.     $tendangnhap = isset($_POST['tendangnhap']) ? $_POST['tendangnhap'] : '';
6.     $matkhau     = isset($_POST['matkhau']) ? md5($_POST['matkhau']) : '';
        //Mã hóa MD5 cho mật khẩu
7.     $hoten       = isset($_POST['hoten']) ? $_POST['hoten'] : '';
8.     $email       = isset($_POST['email']) ? $_POST['email'] : '';

```

```

9.
10.     $sql = "INSERT INTO taikhoan(tendangnhap, matkhau, hoten, email)
        VALUES ('$tendangnhap', '$matkhau', '$hoten', '$email')";
11.
12.     $results = thucthisql($sql);
13.     if ($results) {
14.         echo "<script> alert('Thêm mới thành công.') </script>";
15.         echo "<script> window.location.href='taikhoan.php' </script>";
16.     } else {
17.         echo "<script> alert('Lỗi!') </script>";
18.         echo "<script> history.go(-1) </script>";
19.     }
20.}
21.??>
22.
23.<div class="container">
24.  <h2>Thêm mới tài khoản</h2>
25.  <form action="" method="post">
26.    <div class="form-group">
27.      <label>Tên đăng nhập:</label>
28.      <input type="text" class="form-control"
        id="tendangnhap" name="tendangnhap">
29.    </div>
30.    <div class="form-group">
31.      <label>Mật khẩu:</label>
32.      <input type="password" class="form-control"
        id="matkhau" name="matkhau">
33.    </div>
34.    <div class="form-group">
35.      <label>Họ và tên:</label>
36.      <input type="text" class="form-control" id="hoten" name="hoten">
37.    </div>
38.    <div class="form-group">
39.      <label>Email:</label>
40.      <input type="email" class="form-control" id="email" name="email">
41.    </div>
42.    <button type="submit" class="btn btn-primary"
        name="add">Thêm</button>
43.  </form>
44.</div>
45.
46.<?php include("footer.php"); ?>

```

* File sửa tài khoản (taikhoan_sua.php)

```
1. <?php include("header.php"); ?>
2. <?php
3. include_once("../database/database.php");
4.
5. //Lấy dữ liệu dòng cần sửa
6. $tendangnhap = isset($_GET['tendangnhap']) ? $_GET['tendangnhap'] : '';
   //Sử dụng phương thức truyền GET
7. $sql = "SELECT * FROM taikhoan WHERE tendangnhap='$tendangnhap' ";
8. $row = get_fetch($sql); //Sử dụng hàm get_fetch để lấy về 1 dòng,
   tương ứng là mảng 1 chiều.
9.
10. if (isset($_POST['edit'])) {
11.     $tendangnhap = isset($_POST['tendangnhap']) ? $_POST['tendangnhap'] : '';
        //Sử dụng phương thức truyền POST của form
12.     $matkhau     = isset($_POST['matkhau']) ? md5($_POST['matkhau']) : '';
        //Mã hóa MD5 cho mật khẩu
13.     $hoten       = isset($_POST['hoten']) ? $_POST['hoten'] : '';
14.     $email       = isset($_POST['email']) ? $_POST['email'] : '';
15.
16.     $sql = "UPDATE taikhoan SET
17.         matkhau = '$matkhau',
18.         hoten = '$hoten',
19.         email = '$email'
20.         WHERE tendangnhap = '$tendangnhap' ";
21.
22.     $results = thucthisql($sql);
23.     if ($results) {
24.         echo "<script> alert('Sửa thành công.') </script>";
25.         echo "<script> window.location.href='taikhoan.php' </script>";
26.     } else {
27.         echo "<script> alert('Lỗi!') </script>";
28.         echo "<script> history.go(-1) </script>";
29.     }
30. }
31. ?>
32.
33. <div class="container">
34.     <h2>Thêm mới tài khoản</h2>
35.     <form action="" method="post">
36.         <div class="form-group">
```

```

37.     <label>Tên đăng nhập:</label>
38.     <input type="text" class="form-control" id="ten"
        name="ten" value="<?php echo $row['tendangnhap'] ?>" disabled>
39.     </div>
40.     <div class="form-group">
41.         <label>Mật khẩu:</label>
42.         <input type="password" class="form-control" id="matkhau"
            name="matkhau" value="<?php echo $row['matkhau'] ?>">
43.     </div>
44.     <div class="form-group">
45.         <label>Họ và tên:</label>
46.         <input type="text" class="form-control" id="hoten"
            name="hoten" value="<?php echo $row['hoten'] ?>">
47.     </div>
48.     <div class="form-group">
49.         <label>Email:</label>
50.         <input type="email" class="form-control" id="email"
            name="email" value="<?php echo $row['email'] ?>">
51.     </div>
52.     <input type="hidden" name="tendangnhap"
        value="<?php echo $row['tendangnhap'] ?>">
53.     <button type="submit" class="btn btn-primary"
        name="edit">Sửa</button>
54. </form>
55.</div>
56.
57.<?php include("footer.php"); ?>

```

* File xóa tài khoản (taikhoan_xoa.php)

```

1. <?php
2. include_once("../database/database.php"); //trường hợp dòng lệnh nhúng có
   trong file header.php thì nhúng file header.php
3.
4. //Lấy biến xác định dòng cần xóa
5. $tendangnhap = isset($_GET['tendangnhap']) ? $_GET['tendangnhap'] : '';
   //Sử dụng phương thức truyền GET
6. $sql = "DELETE FROM taikhoan WHERE tendangnhap='$tendangnhap' ";
7. //Hàm thucthisql() để thực thi câu lệnh sql mà không lấy dữ liệu trả về.
8. $results = thucthisql($sql);
9.
10. if ($results) {
11.     echo "<script> alert('Xóa thành công.') </script>";

```

```

12.     echo "<script> window.location.href='taikhoan.php' </script>";
13. } else {
14.     echo "<script> alert('Lỗi!') </script>";
15.     echo "<script> history.go(-1) </script>";
16. }
17. ?>

```

* Trang chủ (index.php)

```

1. <?php include("header.php"); ?>
2. <?php
3. //Những file tương tác csdl
4. include_once("database/database.php");
5.
6. //Lấy danh sách nhóm sản phẩm
7. $sql = "SELECT * FROM nhomsanpham ORDER BY tennhom ASC";
8. $results_nhom = get_fetchall($sql);
9.
10. //Lấy danh sách sản phẩm, nếu tồn tại biến nhomid thì lấy sản phẩm
    có nhomid tương ứng.
11. //Biến nhomid có trong liên kết hiển thị từng nhóm sản phẩm
    trong cột bên trái giao diện.
12. $nhomid = isset($_GET['nhomid']) ? $_GET['nhomid'] : '';
    //Sử dụng phương thức truyền GET
13. if ($nhomid!='')
14.     $sql = "SELECT * FROM sanpham WHERE nhomid='$nhomid' ORDER BY id ASC";
15. else
16.     $sql = "SELECT * FROM sanpham ORDER BY id ASC";
17. $results = get_fetchall($sql);
18.
19. ?>
20. <div class="container mt-3">
21.     <div class="row">
22.         <!--Hiển thị menu danh mục nhóm sản phẩm trong cột bên trái-->
23.         <div class="col-sm-2">
24.             <ul class="list-group">
25.                 <?php foreach ($results_nhom as $row) { ?>
26.                     <li class="list-group-item">
27.                         <a href="index.php?nhomid=<?php echo $row['nhomid']; ?>">
28.                             <?php echo $row['tennhom']; ?></a></li>

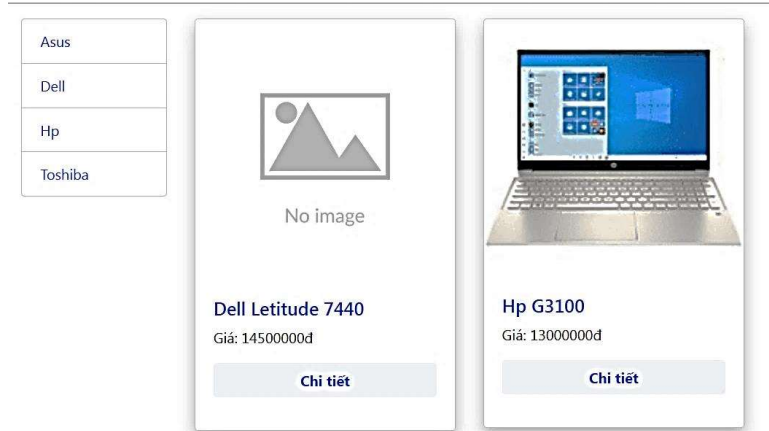
```

```

29.         </div>
30.         <!-- Hiển thị các sản phẩm trong cột bên phải -->
31.         <div class="col-sm-10">
32.             <div class="row">
33.                 <!-- Mỗi sản phẩm hiển thị chiếm 4 ô giao diện -->
34.                 <?php foreach ($results as $row) { ?>
35.                     <div class="col-sm-4">
36.                         <div class="card shadow">
37.                             <?php
38.                             $image_url = "";
39.                             if ($row['anh'] != '') {
40.                                 $image_url = "images/".$row['anh'];
41.                             }
42.                             //Clear cache and check file_exists again
43.                             clearstatcache();
44.                             if (file_exists($image_url)) {
45.                                 echo '';
46.                             } else {
47.                                 echo '';
48.                             }
49.                             ?>
50.                             <div class="card-body">
51.                                 <h5 class="text-primary">
<?php echo $row["tensp"] ?></h5>
52.                                 <p>Giá: <?php echo $row["dongia"] ?>đ</p>
53.                                 <p><a href="detail.php?id=<?php echo $row["id"] ?>"
class="btn btn-light text-primary w-100 font-weight-bold
stretched-link">Chi tiết</a></p>
54.                             </div>
55.                         </div>
56.                     </div>
57.                 <?php } ?>
58.             </div>
59.         </div>
60.     </div>
61.</div>
62.<?php include("footer.php"); ?>

```


Kết quả:



Hình b.7. Giao diện trang chủ

Bài 6. Lập trình hướng đối tượng

1. Kế thừa giao diện đã xây dựng cho dự án laptop247 trong bài 5.
 2. Xây dựng class Database tương tác với cơ sở dữ liệu (đã được xây dựng trong bài 2) bao gồm: kết nối, thực thi câu lệnh SQL, chuyển dữ liệu trả về của truy vấn thành một mảng, lấy số dòng trả về của truy vấn ...
 3. Sử dụng class Database trên và xây dựng các class khác thực hiện:
 - Lập trình quản trị tài khoản, nhóm sản phẩm, sản phẩm, đặt hàng trong cơ sở dữ liệu.
 - Lập trình đăng nhập, đăng xuất tài khoản quản trị.
 - Lập trình hiển thị danh sách, thông tin sản phẩm trên giao diện người dùng.
- Yêu cầu có thể tìm kiếm theo nhóm sản phẩm.
- Lập trình giỏ hàng, đặt hàng của khách hàng.

Hướng dẫn:

Nội dung hướng dẫn tập trung minh họa phương pháp sử dụng class để lập trình tương tác cơ sở dữ liệu. Cấu trúc thư mục tương tự bài 5, tuy nhiên thư mục database đổi tên thành class để gần gũi với phương pháp lập trình, các file chứa class sẽ có đuôi *.class.php.

* Xây dựng lớp DB trong file database.php

```
1. <?php
2. class DB
3. {
4.     private static $instance = NULL;
5.     //Khai báo phương thức static, khi sử dụng phải thông qua tên lớp.
6.     public static function getInstance() {
7.         if (!isset(self::$instance)) {
8.             try {
```

```

9.         self::$instance =
            new PDO('mysql:host=localhost;dbname=laptop247', 'root', '');
10.         self::$instance->exec("SET NAMES 'utf8'");
11.     } catch (PDOException $ex) {
12.         die($ex->getMessage());
13.     }
14. }
15. return self::$instance;
16. }
17.}
18.??>

```

Trong lớp trên chỉ có 1 phương thức getInstance() để tạo kết nối. Các phương thức khác không được xây dựng mà gọi trực tiếp các hàm PDO trong lớp thực thi trực tiếp tương ứng với từng bảng. Ví dụ hàm `query()`, `fetchAll()`, `fetch()`, `exec()`...

Việc sử dụng cách gọi trực tiếp các hàm của PDO trong các lớp thực thi tương ứng với các bảng giúp giảm bớt khai báo các phương thức của lớp DB như trên. Tuy nhiên cách gọi trực tiếp các hàm của PDO cũng gây khó khăn khi thay đổi cách thức hoạt động với cơ sở dữ liệu. Nếu thay đổi cách thức hoạt động với cơ sở dữ liệu khác PDO, ví dụ MySQLi chẳng hạn, phải chỉnh sửa các hàm của PDO trên (hàm `query()`, `fetchAll()`, `fetch()`, `exec()`...) thành các hàm của MySQLi trong các lớp thực thi. Điều đó sẽ mất rất nhiều công sức. Vì vậy cách tốt nhất là tạo các phương thức tương ứng với các hàm PDO ngay trong class DB, từ đó nếu có thay đổi cách sử dụng phương pháp hoạt động với cơ sở dữ liệu thì chỉ cần chỉnh sửa trong file chứa class DB là được.

* Xây dựng lớp TaiKhoan trong file taikhoan.class.php

```

1. <?php
2. class TaiKhoan
3. {
4.     // Khai báo các thuộc tính tương ứng với các trường của bảng trong
        CSDL.
5.     public $tendangnhap;
6.     public $matkhau;
7.     public $hoten;
8.     public $email;
9.
10.    // Phương thức khởi tạo.
11.    function __construct()
12.    {
13.
14.    }
15.

```

```

16. //Phương thức all() dùng để lấy toàn bộ dữ liệu của bảng.
    Mặc định không chỉ rõ phạm vi sẽ là public
17. function get_all()
18. {
19.     $db = DB::getInstance();//Lớp DB có phương thức getInstance()
        là static nên khi gọi phương thức phải thông qua tên lớp.
20.     $req = $db->query('SELECT * FROM taikhoan');
21.     return $req->fetchAll(); //Trả về mảng các bản ghi
22. }
23.
24. //Phương thức get_byid() dùng để lấy thông tin 1 dòng theo tendangnhap.
25. function get_byid($tendangnhap)
26. {
27.     $db = DB::getInstance();
28.     $req = $db->prepare('SELECT * FROM taikhoan
        WHERE tendangnhap = :tendangnhap');
29.     $req->execute(array('tendangnhap' => $tendangnhap));
30.     return $req->fetch();
31. }
32.
33. //Thêm mới
34. function insert()
35. {
36.     $db = DB::getInstance();
37.     $sql = "INSERT INTO taikhoan(tendangnhap, matkhau, hoten, email)
38.         VALUES ('$this->tendangnhap', '$this->matkhau',
        '$this->hoten', '$this->email')";
39.     return $db->exec($sql);
40. }
41.
42. //Sửa
43. function update()
44. {
45.     $db = DB::getInstance();
46.     $sql = "UPDATE taikhoan SET
47.         matkhau = '$this->matkhau',
48.         hoten = '$this->hoten',
49.         email = '$this->email'
50.         WHERE tendangnhap = '$this->tendangnhap' ";
51.     return $db->exec($sql);
52. }
53.

```

```

54. //Xóa
55. function delete($tendangnhap)
56. {
57.     $db = DB::getInstance();
58.     $sql = "DELETE FROM taikhoan WHERE tendangnhap='$tendangnhap' ";
59.     return $db->exec($sql);
60. }
61.
62.}
63.??>

```

* Xây dựng trang quản lý tài khoản (taikhoan.php)

```

1. <?php include("header.php"); ?>
2. <?php
3. require_once("../class/database.class.php");
   //Nhúng file class DB. Nên đặt trong file header.php
4. require_once("../class/taikhoan.class.php"); //Nhúng file class TaiK
   hoan.
5.
6. $obj = new TaiKhoan(); //Tạo đối tượng của lớp TaiKhoan.
7. $results = $obj->get_all();
   //Gọi phương thức get_all() để lấy toàn bộ dữ liệu bảng.
8. ?>
9.
10.<div class="container">
11. <h2>Quản lý tài khoản</h2>
12. <table class="table table-bordered">
13.     <thead>
14.         <tr>
15.             <th></th>
16.             <th>Tên đăng nhập</th>
17.             <th>Mật khẩu</th>
18.             <th>Họ và tên</th>
19.             <th>Email</th>
20.         </tr>
21.     </thead>
22.     <tbody>
23.         <?php foreach ($results as $row) { ?>
24.             <tr>
25.                 <td>
26.                     <!-- Liên kết có truyền biến tendangnhap -->

```

```

27.         <a href="taikhoan_sua.php?tendangnhap=<?php echo
           $row['tendangnhap']; ?>">Sửa</a> |
28.         <a href="taikhoan_xoa.php?tendangnhap=<?php echo
           $row['tendangnhap']; ?>">Xóa</a>
29.     </td>
30.     <td><?php echo $row['tendangnhap']; ?></td>
31.     <td><?php echo $row['matkhau']; ?></td>
32.     <td><?php echo $row['hoten']; ?></td>
33.     <td><?php echo $row['email']; ?></td>
34. </tr>
35. <?php } ?>
36. </tbody>
37. </table>
38. <a href="taikhoan_them.php?">Thêm mới</a>
39.</div>
40.<?php include("footer.php"); ?>

```

* Xây dựng trang thêm mới tài khoản (taikhoan_them.php)

```

1. <?php include("header.php"); ?>
2. <?php
3. if (isset($_POST['add'])) {
4.     require_once("../class/database.class.php");//Nhúng file class DB.
       Nên đặt trong file header.php
5.     require_once("../class/taikhoan.class.php");//Nhúng file class TaiKhoan.
6.
7.     //Tạo đối tượng của lớp TaiKhoan.
8.     $obj = new TaiKhoan();
9.
10.    //Gán giá trị cho các thuộc tính
11.    $obj->tendangnhap =
        isset($_POST['tendangnhap']) ? $_POST['tendangnhap'] : '';
12.    $obj->matkhau =
        isset($_POST['matkhau']) ? md5($_POST['matkhau']) : '';
        //Mã hóa MD5 cho mật khẩu
13.    $obj->hoten = isset($_POST['hoten']) ? $_POST['hoten'] : '';
14.    $obj->email = isset($_POST['email']) ? $_POST['email'] : '';
15.
16.    //Gọi hàm insert() để thêm dữ liệu bảng.
17.    $results = $obj->insert();
18.
19.    //Kiểm tra kết quả thực hiện.
20.    if ($results) {

```

```

21.     echo "<script> alert('Thêm mới thành công.') </script>";
22.     echo "<script> window.location.href='taikhoan.php' </script>";
23. } else {
24.     echo "<script> alert('Lỗi!') </script>";
25.     echo "<script> history.go(-1) </script>";
26. }
27.}
28.??>
29.<div class="container">
30. <h2>Thêm mới tài khoản</h2>
31. <form action="" method="post">
32.     <div class="form-group">
33.         <label>Tên đăng nhập:</label>
34.         <input type="text" class="form-
35.             control" id="tendangnhap" name="tendangnhap">
36.     </div>
37.     <div class="form-group">
38.         <label>Mật khẩu:</label>
39.         <input type="password" class="form-
40.             control" id="matkhau" name="matkhau">
41.     </div>
42.     <div class="form-group">
43.         <label>Họ và tên:</label>
44.         <input type="text" class="form-control" id="hoten" name="hoten">
45.     </div>
46.     <div class="form-group">
47.         <label>Email:</label>
48.         <input type="email" class="form-control" id="email" name="email">
49.     </div>
50.     <button type="submit" class="btn btn-
51.         primary" name="add">Thêm</button>
52. </form>
53.</div>
54.<?php include("footer.php"); ?>

```

* Xây dựng trang sửa tài khoản (taikhoan_sua.php)

```

1. <?php include("header.php"); ?>
2. <?php
3. require_once("../class/database.class.php"); //Nhúng file class DB.
   Nên đặt trong file header.php
4. require_once("../class/taikhoan.class.php"); //Nhúng file class TaiKhoan.

```

```

5. //Tạo đối tượng của lớp TaiKhoan.
6. $obj = new TaiKhoan();
7. //Lấy dữ liệu cần sửa, biến tendangnhap được truyền qua URL khi người dùng
8. //kích chọn nút Sửa trên giao diện trang tài khoản (taikhoan.php)
9. $tendangnhap = isset($_GET['tendangnhap']) ? $_GET['tendangnhap'] : '';
10.$row = $obj->get_byid($tendangnhap);
11.
12.if (isset($_POST['edit'])) {
13.    //Gán giá trị cho các thuộc tính của đối tượng
14.    $obj->tendangnhap =
        isset($_POST['tendangnhap']) ? $_POST['tendangnhap'] : '';
        //Sử dụng phương thức truyền POST của form
15.    $obj->matkhau =
        isset($_POST['matkhau']) ? md5($_POST['matkhau']) : '';
        //Mã hóa MD5 cho mật khẩu
16.    $obj->hoten = isset($_POST['hoten']) ? $_POST['hoten'] : '';
17.    $obj->email = isset($_POST['email']) ? $_POST['email'] : '';
18.
19.    //Thực thi update()
20.    $results = $obj->update();
21.    if ($results) {
22.        echo "<script> alert('Sửa thành công.') </script>";
23.        echo "<script> window.location.href='taikhoan.php' </script>";
24.    } else {
25.        echo "<script> alert('Lỗi!') </script>";
26.        echo "<script> history.go(-1) </script>";
27.    }
28.}
29.??>
30.<div class="container">
31.    <h2>Thêm mới tài khoản</h2>
32.    <form action="" method="post">
33.        <div class="form-group">
34.            <label>Tên đăng nhập:</label>
35.            <input type="text" class="form-control" id="ten"
                name="ten" value="<?php echo $row['tendangnhap'] ?>" disabled>
36.        </div>
37.        <div class="form-group">
38.            <label>Mật khẩu:</label>
39.            <input type="password" class="form-control" id="matkhau"
                name="matkhau" value="<?php echo $row['matkhau'] ?>">
40.        </div>

```

```

41.     <div class="form-group">
42.         <label>Họ và tên:</label>
43.         <input type="text" class="form-control" id="hoten"
           name="hoten" value="<?php echo $row['hoten'] ?>">
44.     </div>
45.     <div class="form-group">
46.         <label>Email:</label>
47.         <input type="email" class="form-control" id="email"
           name="email" value="<?php echo $row['email'] ?>">
48.     </div>
49.     <input type="hidden" name="tendangnhap"
           value="<?php echo $row['tendangnhap'] ?>">
50.     <button type="submit" class="btn btn-primary" name="edit">
           Sửa</button>
51. </form>
52.</div>
53.<?php include("footer.php"); ?>

```

* Xây dựng trang xóa tài khoản (taikhoan_xoa.php)

```

1. <?php
2. require_once("../class/database.class.php");//Nhúng file class DB.
   Nên đặt trong file header.php
3. require_once("../class/taikhoan.class.php");//Nhúng file class TaiKhoan.
4. $obj = new TaiKhoan(); //Tạo đối tượng của lớp TaiKhoan.
5.
6. //Lấy biến xác định dòng cần xóa
7. $tendangnhap = isset($_GET['tendangnhap']) ? $_GET['tendangnhap'] : '';
8.
9. //Thực hiện xóa
10.$results = $obj->delete($tendangnhap);
11.
12.//Kiểm tra
13.if ($results) {
14.    echo "<script> alert('Xóa thành công.') </script>";
15.    echo "<script> window.location.href='taikhoan.php' </script>";
16.} else {
17.    echo "<script> alert('Lỗi!') </script>";
18.    echo "<script> history.go(-1) </script>";
19.}
20.?>

```


* Xây dựng trang chủ (index.php)

Trong việc xây dựng trang chủ có sử dụng đến việc truy xuất lấy ra danh sách nhóm sản phẩm để hiển thị menu lọc sản phẩm theo nhóm sản phẩm, truy xuất lấy ra danh sách các sản phẩm để hiển thị trên giao diện trang chủ. Vì vậy cần tạo các class truy xuất bảng nhóm sản phẩm (nhomsanpham.class.php) và bảng sản phẩm (sanpham.class.php) tương tự như tạo class truy xuất bảng tài khoản (taikhoan.class.php). Lưu ý bổ sung phương thức lọc danh sách sản phẩm theo nhóm (get_bynhomid(\$nhomid)) trong class sản phẩm.

```
1. //Phương thức get_bynhomid() dùng để lấy thông tin sản phẩm theo nhomid.
2. function get_bynhomid($nhomid)
3. {
4.     $db = DB::getInstance();
5.     $req = $db->prepare('SELECT * FROM sanpham
        WHERE nhomid = :nhomid ORDER BY id DESC');
6.     $req->execute(array('nhomid' => $nhomid));
7.     return $req->fetchAll();//Chú ý dùng hàm fetchAll()
8. }
```

Tiếp theo xây dựng trang chủ như sau:

```
1. <?php include("header.php"); ?>
2. <?php
3. //Nhúng file tương tác csdl
4. include_once("class/database.class.php");
5.
6. //Nhúng file class Nhóm sản phẩm.
7. require_once("class/nhomsanpham.class.php");
8. //Tạo đối tượng của lớp NhomSP.
9. $obj_nhom = new NhomSP();
10. //Gọi phương thức get_all() để lấy toàn bộ dữ liệu bảng.
11. $results_nhom = $obj_nhom->get_all();
12.
13. //Lấy danh sách sản phẩm, nếu tồn tại biến nhomid thì lấy sản phẩm
    có nhomid tương ứng
14. //Biến nhomid có trong liên kết hiển thị từng nhóm sản phẩm trong
    cột bên trái giao diện
15. $nhomid = isset($_GET['nhomid']) ? $_GET['nhomid'] : ''; //Sử dụng phương
    thức truyền GET do truyền biến qua URL.
16. //Nhúng file class Sản phẩm.
17. require_once("class/sanpham.class.php");
18. //Tạo đối tượng của lớp SanPham.
19. $obj = new SanPham();
```

```

20. if ($nhomid!='')
21.     $results = $obj->get_bynhomid($nhomid);
22. else
23.     $results = $obj->get_all();
24.
25. ?>
26.
27. <div class="container mt-3">
28.     <div class="row">
29.         <!-- Hiển thị menu danh mục nhóm sản phẩm -->
30.         <div class="col-sm-2">
31.             <ul class="list-group">
32.                 <?php foreach ($results_nhom as $row) { ?>
33.                     <li class="list-group-item"><a href="index.php?nhomid=
34.                         <?php echo $row['nhomid']; ?>"><?php echo $row['tennhom']; ?></a></li>
35.                 <?php } ?>
36.             </ul>
37.         </div>
38.         <!-- Hiển thị các sản phẩm -->
39.         <div class="col-sm-10">
40.             <div class="row">
41.                 <!-- Mỗi sản phẩm hiển thị 4 cột -->
42.                 <?php foreach ($results as $row) { ?>
43.                     <div class="col-sm-4">
44.                         <div class="card shadow">
45.                             <?php
46.                             $image_url = "";
47.                             if ($row['anh']!='') {
48.                                 $image_url = "images/".$row['anh'];
49.                             }
50.                             //Clear cache and check file_exists again
51.                             clearstatcache();
52.                             if (file_exists($image_url)) {
53.                                 echo '';
55.                             } else {
56.                                 echo '';
58.                             }
59.                         </div>
60.                     </div>
61.                 <?php } ?>
62.             </div>
63.         </div>
64.     </div>
65. </div>

```

```

58.         <div class="card-body">
59.             <h5 class="text-primary">
        <?php echo $row["tensp"] ?></h5>
60.             <p>Giá: <?php echo $row["dongia"] ?>đ</p>
61.             <p><a href="detail.php?id=<?php echo $row["id"] ?>"
        class="btn btn-light text-primary w-100 font-weight-bold stretched-link">
        <i class="fa fa-envelope"></i> Chi tiết</a></p>
62.             </div>
63.         </div>
64.     </div>
65.     <?php } ?>
66. </div>
67. </div>
68. </div>
69.</div>
70.<?php include("footer.php"); ?>

```

Trong các bước thực hiện trên, thông tin truyền qua các trang là các biến hoặc mảng các bản ghi. Ví dụ phương thức `get_all()` có dữ liệu trả về là mảng các bản ghi. Để hiển thị dữ liệu ta dùng cú pháp truy xuất `$tên_biến_mảng['tên_trường']` (Ví dụ: `$row['tendangnhap']`). Có thể tùy chỉnh để dữ liệu truyền giữa các trang dùng các mảng đối tượng thay cho mảng bản ghi, từ đó mô hình lập trình sẽ phù hợp hơn, dễ kiểm soát, chỉnh sửa hơn. Khi đó, để truy xuất dữ liệu ta dùng cú pháp `$tên_biến_đối_tượng->tên_thuộc_tính` (Ví dụ: `$obj->tendangnhap`).

Chú ý: Nên sử dụng phương thức static để gọi trực tiếp thông qua tên lớp thay vì phải tạo đối tượng rồi mới gọi phương thức thông qua đối tượng.

Mã lớp `TaiKhoan` được chỉnh sửa như sau:

```

1. <?php
2. class TaiKhoan
3. {
4.     //Khai báo các thuộc tính tương ứng với các trường của bảng trong CSDL.
5.     public $tendangnhap;
6.     public $matkhau;
7.     public $hoten;
8.     public $email;
9.
10.    //Phương thức khởi tạo dùng để gán giá trị truyền vào cho các thuộc
        tính mỗi khi tạo mới đối tượng của lớp.
11.    function __construct($tendangnhap, $matkhau, $hoten, $email)
12.    {
13.        $this->tendangnhap = $tendangnhap;
14.        $this->matkhau     = $matkhau;

```

```

15.     $this->hoten      = $hoten;
16.     $this->email      = $email;
17. }
18.
19. //Phương thức all() dùng để lấy toàn bộ dữ liệu của bảng. Mặc định
    không chỉ rõ phạm vi sẽ là public
20. //Dùng static để truy xuất trực tiếp qua lớp thay vì phải tạo đối
    tượng rồi mới gọi phương thức.
21. static function get_all()
22. {
23.     $list = [];
24.     $db = DB::getInstance();
25.     $req = $db->query('SELECT * FROM taikhoan');
26.
27.     //Duyệt mảng các bản ghi, chuyển bản ghi thành đối tượng và gán
        vào mảng list.
28.     foreach ($req->fetchAll() as $item) {
29.         $list[] = new TaiKhoan($item['tendangnhap'], $item['matkhau'],
            $item['hoten'], $item['email']);
30.     }
31.
32.     return $list; //Trả về mảng các đối tượng TaiKhoan
33. }
34.
35. //Phương thức get_byid() dùng để lấy thông tin 1 dòng theo tendangnhap.
36. //Dùng static để truy xuất trực tiếp qua lớp thay vì phải tạo đối
    tượng rồi mới gọi phương thức.
37. static function get_byid($tendangnhap)
38. {
39.     $db = DB::getInstance();
40.     $req = $db->prepare('SELECT * FROM taikhoan
        WHERE tendangnhap = :tendangnhap');
41.     $req->execute(array('tendangnhap' => $tendangnhap));
42.     $item = $req->fetch();
        //Lấy ra 1 bản ghi có tên đăng nhập = giá trị biến $tendangnhap
43.     if (isset($item['tendangnhap'])) {
44.         return new TaiKhoan($item['tendangnhap'], $item['matkhau'],
            $item['hoten'], $item['email']); //Trả về đối tượng TaiKhoan
45.     }
46.     return null;
47. }
48.

```

```

49. //Thêm mới
50. function insert()
51. {
52.     $db = DB::getInstance();
53.     $sql = "INSERT INTO taikhoan(tendangnhap, matkhau, hoten, email)
54.         VALUES ('$this->tendangnhap', '$this->matkhau',
55.             '$this->hoten', '$this->email')";
56.     return $db->exec($sql);
57. }
58. //Sửa
59. function update()
60. {
61.     $db = DB::getInstance();
62.     $sql = "UPDATE taikhoan SET
63.         matkhau = '$this->matkhau',
64.         hoten = '$this->hoten',
65.         email = '$this->email'
66.         WHERE tendangnhap = '$this->tendangnhap' ";
67.     return $db->exec($sql);
68. }
69.
70. //Xóa
71. //Dùng static sẽ gọi trực tiếp thay vì phải tạo đối tượng
    rồi mới gọi phương thức.
72. static function delete($tendangnhap)
73. {
74.     $db = DB::getInstance();
75.     $sql = "DELETE FROM taikhoan WHERE tendangnhap='$tendangnhap' ";
76.     return $db->exec($sql);
77. }
78.
79. }
80. ?>

```

Trong phần mã trên, lớp TaiKhoan có thêm phương thức khởi tạo (`__construct()`) để gán giá trị truyền vào cho đối tượng khi tạo đối tượng. Một số phương thức sử dụng *static* để gọi trực tiếp qua lớp (`get_all()`, `get_byid()`, `delete()`); phương thức `insert()`, `update()` được gọi đến thông qua đối tượng của lớp (tức là phải tạo đối tượng mới sử dụng được phương thức này).

Trang quản lý tài khoản được chỉnh sửa như sau:

```

1. <?php include("header.php"); ?>
2. <?php
3. require_once("../class/database.class.php");
   //Nhúng file class DB. Nên đặt trong file header.php
4. require_once("../class/taikhoan.class.php");//Nhúng file class TaiKhoan.
5. //Gọi phương thức get_all() lấy toàn bộ dữ liệu bảng. Phương thức get_all()
   là static nên gọi qua class mà không phải tạo và gọi qua đối tượng của class.
6. $results = TaiKhoan::get_all();
7. ?>
8. <div class="container">
9.   <h2>Quản lý tài khoản</h2>
10.  <table class="table table-bordered">
11.    <thead>
12.      <tr>
13.        <th></th>
14.        <th>Tên đăng nhập</th>
15.        <th>Mật khẩu</th>
16.        <th>Họ và tên</th>
17.        <th>Email</th>
18.      </tr>
19.    </thead>
20.    <tbody>
21.      <?php foreach ($results as $taikhoan) { ?>
22.        <tr>
23.          <td>
24.            <!-- Liên kết có truyền biến tendangnhap -->
25.            <a href="taikhoan_sua.php?tendangnhap=<?php echo
               $taikhoan->tendangnhap; ?>">Sửa</a> |
26.            <a href="taikhoan_xoa.php?tendangnhap=<?php echo
               $taikhoan->tendangnhap; ?>">Xóa</a>
27.          </td>
28.          <td><?php echo $taikhoan->tendangnhap ?></td>
29.          <td><?php echo $taikhoan->matkhau ?></td>
30.          <td><?php echo $taikhoan->hoten ?></td>
31.          <td><?php echo $taikhoan->email ?></td>
32.        </tr>
33.      <?php } ?>
34.    </tbody>
35.  </table>
36.  <a href="taikhoan_them.php?">Thêm mới</a>
37.</div>
38.<?php include("footer.php"); ?>

```

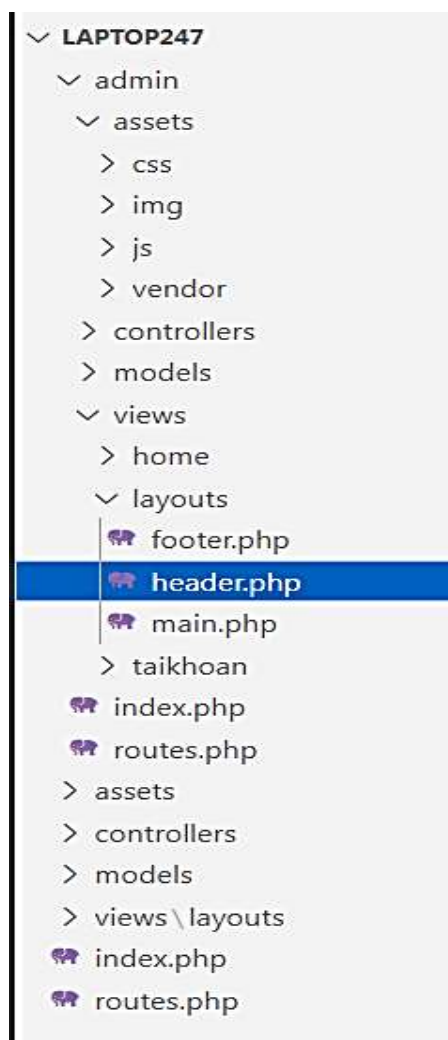
Trong phần mã trên, chú ý cách truy vấn dữ liệu hiển thị. Thay vì sử dụng cú pháp \$tên_biến_bản_ghi['tên_trường'] thì sử dụng cú pháp \$tên_đối_tượng->thuộc_tính (Ví dụ: \$taikhoan->tendangnhap). Cách làm tương tự cho các trang còn lại.

Bài 7. Lập trình theo mô hình MVC

1. Xây dựng giao diện cho dự án website laptop247 (có thể kế thừa giao diện đã xây dựng trong bài 5). Cấu trúc các thư mục theo mẫu dự án web MVC.
2. Lập trình dự án website laptop247 theo mô hình MVC.

Hướng dẫn:

1. Xây dựng cấu trúc thư mục

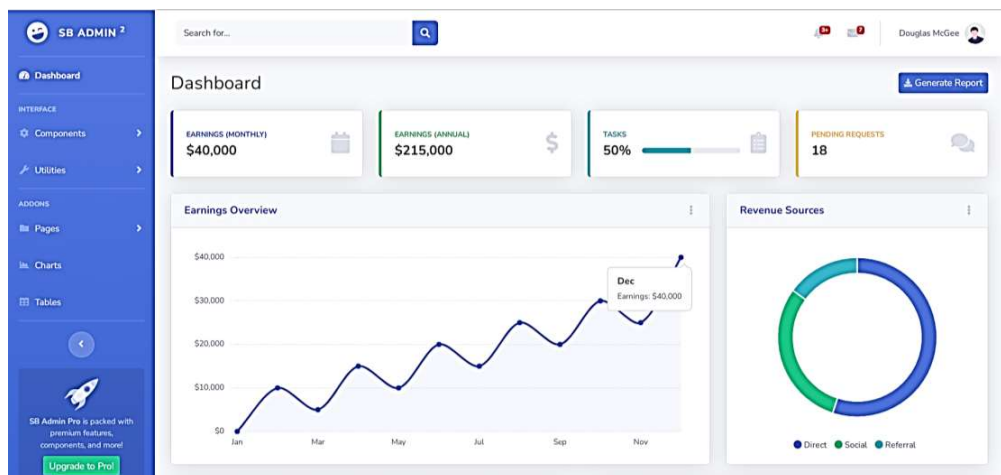


Hình b.8. Cấu trúc thư mục website

Theo hướng dẫn trong chương 4, ta xây dựng cấu trúc thư mục trong đó phân chia giao diện thành hai phần: phần quản trị (backend) và phần người dùng (frontend). Cả hai phần này đều có layout riêng và xây dựng theo cấu trúc MVC do đó sẽ có cấu trúc tương tự nhau và cũng gồm các thư mục models, controllers, views. Phần quản trị sẽ đặt trong thư mục admin trong thư mục gốc của dự án. Cấu trúc thư mục được mô tả như sau:

* Phần quản trị

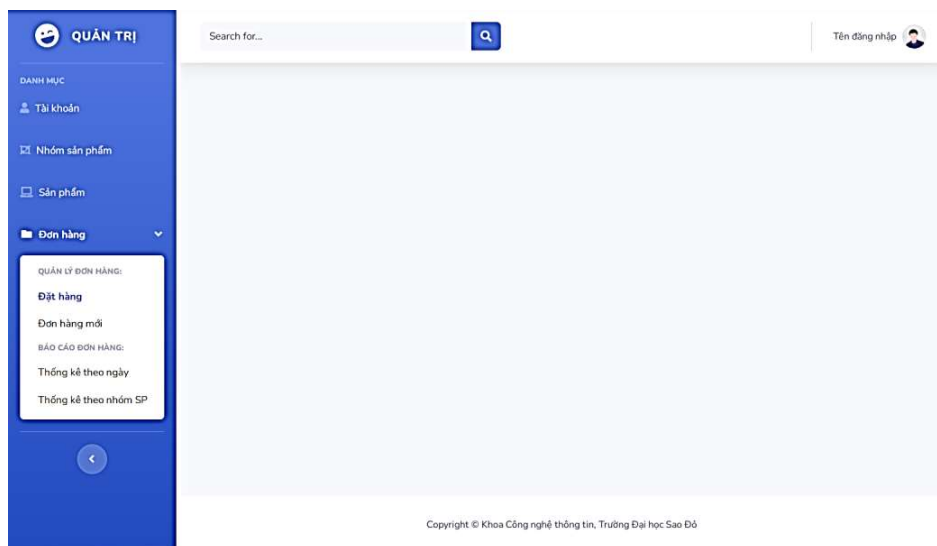
Trong thiết kế giao diện quản trị của dự án này, chúng ta sử dụng giao diện SB Admin có mẫu sau:



Hình b.9. Giao diện SB Admin

Sao chép các thư mục của mẫu vào thư mục `admin\assets\` của dự án. Sau đó mở trang `blank.html` của mẫu, tách mã thành 2 phần: phần header đặt trong trang `admin\views\layouts\header.php`; phần footer đặt trong trang `admin\views\layouts\footer.php`. Tiếp theo tạo trang `main.php` đặt trong thư mục `admin\views\layouts`. Trang `main.php` sẽ gọi nhúng trực tiếp trang `header.php` và trang `footer.php`.

Cần chỉnh sửa đường dẫn đến các file thư viện (css, js, img...) của các trang layout để đảm bảo không xảy ra lỗi đường dẫn. Đồng thời cũng cần chỉnh sửa các liên kết cho phù hợp với dự án. Cuối cùng giao diện trang quản trị dùng chung như sau:



Hình b.10. Giao diện trang quản trị hệ thống

Chú ý kiểm tra lỗi giao diện (lỗi đường dẫn) trong bảng error bên phải cửa sổ trình duyệt.

* Phần người dùng

Để xây dựng giao diện phần người dùng, thực hiện tương tự như trên.

Trong các hướng dẫn tiếp theo sẽ tập trung hướng dẫn lập trình xử lý cho phần quản trị theo mô hình MVC.

2. Xây dựng các trang điều khiển luồng thông tin

Mặc định, để truy cập vào tài khoản quản trị, quản trị cần truy nhập đường dẫn <http://laptop247.saodo.edu.vn/admin> (trong đó laptop247.saodo.edu.vn là tên miền ảo đã thiết lập trước đó). Ngay khi truy cập đường dẫn trên, trang index.php đặt trong thư mục admin sẽ được gọi đến. Mã nguồn trang như sau:

```
1. <?php
2. require_once('models/db_model.php'); //Nhúng trang kết nối csdl.
3. if (isset($_GET['controller'])) {
4.     $controller = $_GET['controller'];
5.     if (isset($_GET['action'])) {
6.         $action = $_GET['action'];
7.     } else {
8.         $action = 'index'; //Mỗi controller phải có một action
           có tên index làm mặc định nếu không chỉ rõ action.
9.     }
10.} else {
11.    $controller = 'home'; //Mặc định module home được gọi đến.
12.    $action = 'index'; //Mặc định action index của home được gọi đến.
13.}
14.require_once('routes.php'); //Nhúng routes để xử lý gọi controller
    phù hợp với yêu cầu của client.
```

Tiếp theo xây dựng trang routes.php đã được nhúng trong trang index.php như phần trình bày bên trên. Mã nguồn trang routes.php như sau:

```
1. <?php
2. $controllers = array(
3.     'home'      => ['index', 'error'],
4.     'taikhoan'  => ['index', 'add', 'add_submit', 'edit', 'edit_sub
        mit', 'del', 'detail'],
5.     'nhomsanpham' => ['index', 'add', 'add_submit', 'edit', 'edit_sub
        mit', 'del', 'detail'],
6.     'sanpham'   => ['index', 'add', 'add_submit', 'edit', 'edit_sub
        mit', 'del', 'detail'],
7.     'dathang'   => ['index', 'add', 'add_submit', 'edit', 'edit_sub
        mit', 'del', 'detail'],
8. ); // Các controllers trong hệ thống và các action có thể gọi ra
    từ controller đó. Mỗi controllers tương ứng với một module.
9.
```

```

10.// Nếu các tham số controller và action nhận được từ URL không hợp lệ
11.// (không thuộc list controller và action khai báo bên trên) thì trang
    báo lỗi sẽ được gọi ra.
12.if (!array_key_exists($controller, $controllers)
    || !in_array($action, $controllers[$controller])) {
13.    $controller = 'home';
14.    $action = 'error';
15.    echo "<script> alert('Không tồn tại controller hoặc action!') </script>";
16.}
17.
18.// Nhúng file định nghĩa controller vào để có thể dùng
    được class định nghĩa trong file đó
19.include_once('controllers/' . $controller . '_controller.php');
20.// Tạo ra tên lớp controller từ các giá trị lấy được từ URL
    sau đó gọi ra để hiển thị trả về cho người dùng.
21.//Ví dụ $controller='pages' thì tạo ra tên lớp controller là PagesController.
22.$klass = str_replace('_', '', ucwords($controller, '_')) . 'Controller';
23.
24.// Tạo đối tượng controller từ lớp controller tương ứng được khai báo
25.// trong file tên-module_controller.php. Ví dụ file pages_controller.php.
26.$controller = new $klass;
27.
28.// Gọi hành động được chỉ định của đối tượng controller.
29.$controller->$action();

```

Tiếp theo xây dựng lớp kết nối cơ sở dữ liệu.

3. Xây dựng lớp kết nối cơ sở dữ liệu

Kế thừa lớp kết nối đã trình bày trong các phần trước, tuy nhiên trong dự án này, ta đặt file lớp kết nối trong thư mục models của phần quản trị ứng dụng, có tên file db_model.php.

```

1. <?php
2. class DB
3. {
4.     private static $instance = NULL;
5.
6.     //Khai báo phương thức static, khi sử dụng phải thông qua tên lớp.
7.     public static function getInstance() {
8.         if (!isset(self::$instance)) {
9.             try {
10.                 self::$instance =
                    new PDO('mysql:host=localhost;dbname=laptop247', 'root', '');
11.                 self::$instance->exec("SET NAMES 'utf8'");

```

```

12.     } catch (PDOException $ex) {
13.         die($ex->getMessage());
14.     }
15.     }
16.     return self::$instance;
17. }
18.}
19.??>

```

4. Xây dựng module home

Như nội dung đã trình bày trong trang routes.php. Nếu không có controller nào được chỉ định thì sẽ gọi đến controller home (module home). Do đó cần xây dựng module home. Module home không tương tác đến cơ sở dữ liệu nên các bước thực hiện như sau.

Đầu tiên ta xây dựng lớp BaseController. Lớp BaseController chứa các phương thức dùng chung cho tất cả các controller của các module khác. Trong dự án này, nó chứa phương thức render dùng để gọi trang chứa view tương ứng và hiển thị.

* Lớp BaseController (trang controllers\base_controller.php).

```

1. <?php
2. class BaseController
3. {
4.     protected $folder; //Biến có giá trị là thư mục trong thư mục
        views\tên_module, chứa các file view template của module đang truy cập.
5.     // Hàm hiển thị kết quả ra cho người dùng. Biến file lưu tên file
        của view, mảng data lưu thông tin dữ liệu truyền cho view.
6.     function render($file, $data = array())
7.     {
8.         // Kiểm tra file gọi đến có tồn tại hay không?
9.         $view_file = 'views/' . $this->folder . '/' . $file . '.php';
10.        if (is_file($view_file)) {
11.            // Nếu tồn tại file đó thì tạo ra các biến chứa giá trị truyền
                vào lúc gọi hàm. Các khóa mảng làm tên biến và giá trị là giá trị biến.
12.            // $data là một mảng kết hợp (mảng có khóa và giá trị). Khi sử dụng
                hàm extract sẽ tạo ra các biến mới từ khóa của mảng và giá trị của mảng.
13.            extract($data);
14.            //Sau đó lưu giá trị trả về khi chạy file view template v
                ới các dữ liệu đó vào 1 biến mà chưa hiển thị luôn ra trình duyệt.
15.            ob_start(); // Tạo bộ đệm đầu ra.
16.            require_once($view_file);
17.            $content = ob_get_clean(); // trả về nội dung của bộ đệm
                đầu ra và sau đó xóa nội dung khỏi bộ đệm.

```

```

18.         // Sau khi có kết quả đã được lưu vào biến $content, gọi
           ra template chung của hệ thống để hiển thị ra cho người dùng.
19.         require_once('views/layouts/main.php');
20.     } else {
21.         // Nếu file muốn gọi ra không tồn tại thì chuyển hướng đến
           trang báo lỗi.
22.         header('Location: index.php?controller=home&action=error');
23.     }
24. }
25.}

```

* Xây dựng Controller (trang controllers\home_controller.php)

```

1. <?php
2. require_once('controllers\base_controller.php');
3. //Tạo lớp HomeController kế thừa từ lớp BaseController
4. class HomeController extends BaseController
5. {
6.     function __construct()
7.     {
8.         $this->folder = 'home';
           //Chỉ định thư mục home là thư mục lưu các file của view trong module home.
9.     }
10.
11.     public function index()
12.     {
13.         $data = array();
14.         $this->render('index', $data); //Gọi đến index view.
15.     }
16.
17.     public function error()
18.     {
19.         $this->render('error'); //Gọi đến error view.
20.     }
21.
22.}

```

* Các trang View

+ Trang views\home\index.php

```

1. <?php
2.     //Trang index của module Home
3. ?>

```

+ Trang views\home\error.php

```
1. <?php
2.     echo 'Có lỗi xảy ra!';
3. ?>
```

5. Xây dựng module chức năng quản trị tài khoản (module taikhoan)

* Xây dựng Model (trang models\taikhoan_model.php)

```
1. <?php
2. class Taikhoan //Chữ cái đầu viết hoa, còn lại chữ thường
3. {
4.     //Khai báo các thuộc tính tương ứng với các trường của bảng trong CSDL.
5.     public $tendangnhap;
6.     public $matkhau;
7.     public $hoten;
8.     public $email;
9.
10.    //Phương thức khởi tạo dùng để gán giá trị truyền vào cho các thuộc
        tính mỗi khi tạo mới đối tượng của lớp.
11.    function __construct($tendangnhap, $matkhau, $hoten, $email)
12.    {
13.        $this->tendangnhap = $tendangnhap;
14.        $this->matkhau     = $matkhau;
15.        $this->hoten       = $hoten;
16.        $this->email       = $email;
17.    }
18.
19.    //Phương thức get_all() dùng để lấy toàn bộ dữ liệu của bảng.
        Theo mặc định khi không chỉ rõ phạm vi thì phạm vi sẽ là public
20.    //Dùng static để truy xuất trực tiếp qua lớp thay vì phải tạo
        đối tượng rồi mới gọi phương thức.
21.    static function get_all()
22.    {
23.        $list = [];
24.        $db = DB::getInstance();
25.        $req = $db->query('SELECT * FROM taikhoan');
26.
27.        //Duyệt mảng các bản ghi, chuyển bản ghi thành đối tượng
            và gán vào mảng list.
28.        foreach ($req->fetchAll() as $item) {
29.            $list[] = new Taikhoan($item['tendangnhap'], $item['matkhau'],
                $item['hoten'], $item['email']);
        }
```

```

30.     }
31.
32.     return $list; //Trả về mảng các đối tượng Taikhoan
33. }
34.
35. //Phương thức get_byid() dùng để lấy thông tin 1 dòng theo tendangnhap.
36. //Dùng static để truy xuất trực tiếp qua lớp thay vì phải tạo đối
    tượng rồi mới gọi phương thức.
37. static function get_byid($tendangnhap)
38. {
39.     $db = DB::getInstance();
40.     $req = $db->prepare('SELECT * FROM taikhoan
        WHERE tendangnhap = :tendangnhap');
41.     $req->execute(array('tendangnhap' => $tendangnhap));
42.
43.     $item = $req->fetch();
        //Lấy ra 1 bản ghi có tên đăng nhập = giá trị biến $tendangnhap
44.     if (isset($item['tendangnhap'])) {
45.         return new Taikhoan($item['tendangnhap'], $item['matkhau'],
            $item['hoten'], $item['email']); //Trả về đối tượng Taikhoan
46.     }
47.     return null;
48. }
49.
50. //Thêm mới
51. function insert()
52. {
53.     $db = DB::getInstance();
54.     $sql = "INSERT INTO taikhoan(tendangnhap, matkhau, hoten, email)
55.         VALUES ('$this->tendangnhap', '$this->matkhau',
            '$this->hoten', '$this->email')";
56.     return $db->exec($sql);
57. }
58.
59. //Sửa
60. function update()
61. {
62.     $db = DB::getInstance();
63.     $sql = "UPDATE taikhoan SET
64.         matkhau = '$this->matkhau',
65.         hoten = '$this->hoten',
66.         email = '$this->email'

```

```

67.         WHERE tendangnhap = '$this->tendangnhap' ";
68.     return $db->exec($sql);
69. }
70.
71. //Xóa
72. //Dùng static sẽ gọi trực tiếp thay vì phải tạo đối tượng
    rồi mới gọi phương thức.
73. static function delete($tendangnhap)
74. {
75.     $db = DB::getInstance();
76.     $sql = "DELETE FROM taikhoan WHERE tendangnhap='$tendangnhap' ";
77.     return $db->exec($sql);
78. }
79.
80.}
81. ?>

```

* Xây dựng Controller (trang controllers\taikhoan_controller.php)

```

1. <?php
2. require_once('controllers/base_controller.php');
3. require_once('models/taikhoan_model.php');
4.
5. class TaikhoanController extends BaseController
6. {
7.     function __construct()
8.     {
9.         $this->folder = 'taikhoan'; //Chỉ định thư mục chứa các file view.
10.    }
11.
12.    public function index()
13.    {
14.        $taikhoans = Taikhoan::get_all(); //Gọi phương thức get_all() của
            model Taikhoan lấy tất cả dữ liệu từ bảng.
15.
16.        $data = array('taikhoans' => $taikhoans);
17.        $this->render('index', $data); //Gọi hàm render được kế thừa
18.    }
19.
20.    public function detail()
21.    {

```

```

22.     $taikhoan = Taikhoan::get_byid($_GET['id']);
        //Gọi phương thức get_byid() của model Taikhoan lấy dữ liệu có id từ bảng
23.
24.     $data = array('taikhoan' => $taikhoan);
25.     $this->render('detail', $data);
26. }
27.
28. public function add()
29. {
30.     $this->render('add');
31. }
32.
33. public function add_submit()
34. {
35.     //Gán giá trị của form cho các biến
36.     $tendangnhap = isset($_POST['tendangnhap']) ? $_POST['tendangnhap'] : '';
37.     $matkhau      = isset($_POST['matkhau']) ? md5($_POST['matkhau']) : '';
        //Mã hóa MD5 cho mật khẩu
38.     $hoten        = isset($_POST['hoten']) ? $_POST['hoten'] : '';
39.     $email        = isset($_POST['email']) ? $_POST['email'] : '';
40.
41.     //Tạo đối tượng của lớp Taikhoan.
42.     //phương thức khởi tạo của lớp Taikhoan sẽ được gọi đến khi tạo đối tượng.
43.     if (($tendangnhap!='') && ($matkhau!='')) {
44.         $taikhoan = new Taikhoan($tendangnhap, $matkhau, $hoten, $email);
45.
46.         //Gọi phương thức insert() để thêm dữ liệu vào bảng. Phương thức
        insert() không dùng static nên phải gọi thông qua đối tượng của class.
47.         $results = $taikhoan->insert();
48.
49.         //Hiển thị thông báo
50.         if ($results) {
51.             echo "<script> alert('Thêm mới thành công.') </script>";
52.         } else {
53.             echo "<script> alert('Lỗi thêm dữ liệu!') </script>";
54.         }
55.
56.         //Trở về trang chủ
57.         $this->index();
58.     } else {
59.         //Hiển thị thông báo

```



```

60.         echo "<script> alert('Lỗi: Tên đăng nhập và mật khẩu không
        được để trống!') </script>";
61.
62.         //Trở về trang thêm mới
63.         $this->add();
64.     }
65. }
66.
67.
68. public function edit()
69. {
70.     $taikhoan = Taikhoan::get_byid($_GET['id']); //Gọi phương thức
        get_byid() của model Taikhoan lấy dữ liệu có id từ bảng.
71.
72.     $data = array('taikhoan' => $taikhoan);
73.     $this->render('edit', $data);
74. }
75.
76. public function edit_submit()
77. {
78.     //Gán giá trị của form cho các biến
79.     $tendangnhap = isset($_POST['tendangnhap']) ? $_POST['tendangnhap'] : '';
80.     $matkhau      = isset($_POST['matkhau']) ? md5($_POST['matkhau']) : '';
        //Mã hóa MD5 cho mật khẩu
81.     $hoten        = isset($_POST['hoten']) ? $_POST['hoten'] : '';
82.     $email        = isset($_POST['email']) ? $_POST['email'] : '';
83.
84.     //Tạo đối tượng của lớp Taikhoan.
85.     //phương thức khởi tạo của lớp Taikhoan sẽ được gọi đến khi tạo đối tượng
86.     if (($tendangnhap!='') && ($matkhau!='')) {
87.         $taikhoan = new Taikhoan($tendangnhap, $matkhau, $hoten, $email);
88.
89.         //Gọi phương thức update() để thêm dữ liệu vào bảng.
90.         //Phương thức update() không dùng static nên phải gọi thông
        qua đối tượng của class.
91.         $results = $taikhoan->update();
92.
93.         //Hiển thị thông báo
94.         if ($results) {
95.             echo "<script> alert('Sửa thành công.') </script>";
96.         } else {
97.             echo "<script> alert('Lỗi sửa dữ liệu!') </script>";

```

```

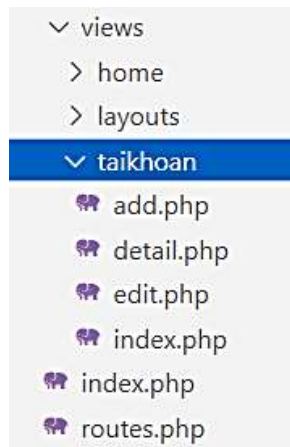
98.     }
99.
100.    //Trở về trang chủ
101.    $this->index();
102.    } else {
103.    //Hiển thị thông báo
104.    echo "<script> alert('Lỗi: Tên đăng nhập và mật khẩu không
    được để trống!') </script>";
105.
106.    //Trở về trang sửa
107.    //$this->edit();//Nếu dùng cách gọi phương thức edit() sẽ không
    truyền được biến Id sang phương thức edit() để lọc dữ liệu cần edit.
108.    echo "<script> history.go(-1) </script>";
109.    }
110.    }
111.
112.    public function del()
113.    {
114.    $taikhoan = Taikhoan::delete($_GET['id']); //Gọi phương thức
    delete() của model Taikhoan.
115.
116.    //Trở về trang chủ
117.    $this->index();
118.    }
119.    } //End class

```

* Xây dựng View (các trang nằm trong thư mục views\taikhoan)

Khi cần sử dụng đến giao diện cho các chức năng quản trị, các giao diện sẽ được đặt trong thư mục views\taikhoan. Thường mỗi action của controller sẽ gọi đến view tương ứng bởi phương thức render của lớp controller. Ví dụ: `$this->render('detail', $data)`; trong đó detail là tên của view cũng tương ứng với tên file (detail.php), `$data` là mảng dữ liệu truyền từ controller sang cho view xử lý để hiển thị.

Từ controller của tài khoản, chúng ta thấy có các action: `index()`, `detail()`, `add()`, `add_submit()`, `edit()`, `edit_submit()`, `del()`. Trong đó các action `index()`, `detail()`, `add()`, `edit()` sử dụng đến giao diện view. Do đó chúng ta có các trang view tương ứng cho các action đó.



Hình b.11. Các trang view của chức năng quản lý tài khoản

+ Trang views\taikhoan\index.php:

```

1. <h4 class="text-primary font-weight-bold">Quản lý tài khoản</h4>
2. <table class="table table-bordered">
3.   <thead>
4.     <tr>
5.       <th></th>
6.       <th>Tên đăng nhập</th>
7.       <th>Mật khẩu</th>
8.       <th>Họ và tên</th>
9.       <th>Email</th>
10.    </tr>
11.  </thead>
12.  <tbody>
13.    <!-- Duyệt mảng các đối tượng được controller truyền sang -->
14.    <?php foreach ($taikhoans as $taikhoan) { ?>
15.      <tr>
16.        <td>
17.          <!-- Liên kết có truyền biến -->
18.          <a href="index.php?controller=taikhoan&action=edit&id=<?php
19.            echo $taikhoan->tendangnhap ?>">Sửa</a> |
20.          <a href="index.php?controller=taikhoan&action=del&id=<?php
21.            echo $taikhoan->tendangnhap ?>"
22.            onclick="return confirm('Bạn chắc chắn muốn xóa?');">Xóa</a>
23.        </td>
24.        <td><?php echo $taikhoan->matkhau ?></td>
25.        <td><?php echo $taikhoan->hoten ?></td>

```

```

25.         <td><?php echo $taikhoan->email ?></td>
26.     </tr>
27. <?php } ?>
28. </tbody>
29.</table>
30.<a href="index.php?controller=taikhoan&action=add">Thêm mới</a>

```

+ Trang views\taikhoan\detail.php:

```

1. <?php
2.     echo "Tên đăng nhập: $taikhoan->tendangnhap <br>";
3.     echo "Mật khẩu: $taikhoan->matkhau <br>";
4.     echo "Họ và tên: $taikhoan->hoten <br>";
5.     echo "Email: $taikhoan->email <br>";
6. ?>

```

Có thể tùy chỉnh trang này để hiển thị thông tin thân thiện hơn.

+ Trang views\taikhoan\add.php:

```

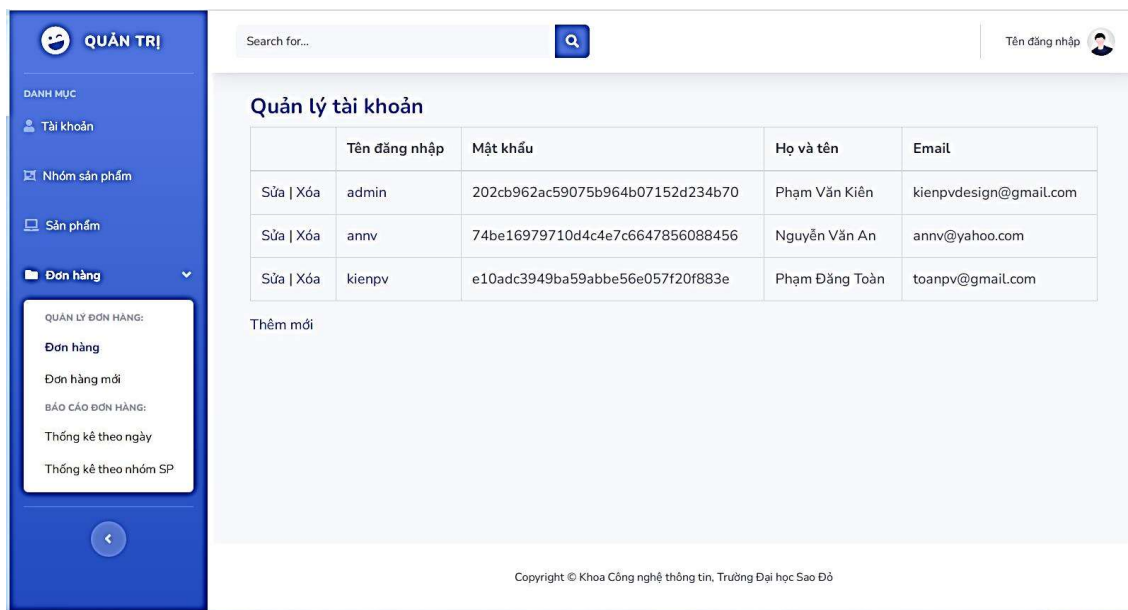
1. <h4 class="text-primary font-weight-bold">Thêm mới tài khoản</h4>
2. <form action="index.php?controller=taikhoan&action=add_submit"
   method="post" class="was-validated">
3.     <div class="form-group">
4.         <label>Tên đăng nhập:</label>
5.         <input type="text" class="form-control" id="tendangnhap"
   name="tendangnhap" required>
6.     </div>
7.     <div class="form-group">
8.         <label>Mật khẩu:</label>
9.         <input type="password" class="form-control" id="matkhau"
   name="matkhau" required>
10.    </div>
11.    <div class="form-group">
12.        <label>Họ và tên:</label>
13.        <input type="text" class="form-control" id="hoten" name="hoten">
14.    </div>
15.    <div class="form-group">
16.        <label>Email:</label>
17.        <input type="email" class="form-control" id="email" name="email">
18.    </div>
19.    <button type="submit" class="btn btn-primary" name="add">Thêm</button>
20.</form>

```

+ Trang views\taikhoan\edit.php

```
1. <h4 class="text-primary font-weight-  
bold">Sửa thông tin tài khoản</h4>  
2. <form action="index.php?controller=taikhoan&action=edit_submit"  
method="post" class="was-validated">  
3.   <div class="form-group">  
4.     <label>Tên đăng nhập:</label>  
5.     <input type="text" class="form-control" id="ten"  
name="ten" value="<?php echo $taikhoan->tendangnhap ?>" disabled>  
6.   </div>  
7.   <div class="form-group">  
8.     <label>Mật khẩu:</label>  
9.     <input type="password" class="form-control" id="matkhau"  
name="matkhau" value="<?php echo $taikhoan->matkhau ?>" required>  
10.  </div>  
11.  <div class="form-group">  
12.    <label>Họ và tên:</label>  
13.    <input type="text" class="form-control" id="hoten"  
name="hoten" value="<?php echo $taikhoan->hoten ?>">  
14.  </div>  
15.  <div class="form-group">  
16.    <label>Email:</label>  
17.    <input type="email" class="form-control" id="email"  
name="email" value="<?php echo $taikhoan->email ?>">  
18.  </div>  
19.  <input type="hidden" name="tendangnhap"  
value="<?php echo $taikhoan->tendangnhap ?>">  
20.  <button type="submit" class="btn btn-  
primary" name="edit">Sửa</button>  
21.</form>
```

Đến đây đã hoàn thành chức năng quản trị tài khoản. Các chức năng quản trị khác được xây dựng tương tự.



Hình b.12. Giao diện quản trị tài khoản

Bài 8. Lập trình website thương mại điện tử

Lập trình website thương mại điện tử theo chủ đề tự chọn, yêu cầu sử dụng mô hình hướng đối tượng hoặc MVC.

TÀI LIỆU THAM KHẢO

Tài liệu tiếng Việt

[1]. Joel Murach, Ray Harris (2019), *Lập trình cơ bản PHP và MySQL*, Nhà xuất bản Bách khoa Hà Nội.

[2]. Joel Murach, Ray Harris (2019), *Lập trình nâng cao PHP và MYSQL*, Nhà xuất bản Bách khoa Hà Nội.

Website

[3]. <https://www.w3schools.com>.